

International Cybersecurity and Digital Forensics Academy (ICDFA)

Basic Computer Skills for Digital Forensics

WEP40 Wireless Packet Decryption and Aircrack Forensics

Student Name: Kafayat Omolara Animashawun

Student ID: 2025/FWSD/11317

Instructor Name: Aminu Idris

Date of Submission: 22/09/2026

EXECUTIVE SUMMARY

This assignment presents a forensic examination of a historical WEP40 wireless network capture supplied as an offline training dataset. The examination followed a controlled forensic workflow beginning with preservation and cryptographic hashing of the original compressed evidence, followed by creation of a working copy, decompression, wireless traffic analysis, WEP initialization vector examination, offline WEP key recovery, decryption, network communication analysis and HTTP object recovery.

The supplied capture contained 45,169 wireless packets associated with the historical network identified as cgnetwork. Analysis identified WEP-protected traffic associated with BSSID 00:26:66:55:97:D6, including repeated WEP initialization vector values. Aircrack-ng successfully recovered the WEP40 key A4:3D:F6:F3:74, after which Airdecap-ng successfully decrypted all 15,477 WEP data packets identified for decryption, with zero corrupted WEP packets reported.

Analysis of the decrypted capture identified 15,477 Ethernet frames containing ARP, IPv4, IPv6, TCP and UDP communications, together with application-layer protocols including HTTP, TLS, DNS, DHCP, mDNS and SSDP. The primary internal endpoint identified was 192.168.0.15, which participated in extensive external HTTP communications.

HTTP object extraction recovered 222 objects occupying approximately 4.3 MB. File-type examination showed that the recovered material consisted predominantly of normal web resources such as HTML, CSS, JavaScript, PNG and GIF images, XML, text and web fonts. Although the capture contained web activity involving a PhishMe webpage titled "Decoding Zeus Disguised as an .RTF File", the recovered objects examined did not identify an RTF document, executable binary or other direct malware payload. Therefore, the evidence supports historical access to the webpage and its associated resources but does not independently establish that a Zeus sample was downloaded or executed.

The examination demonstrates both the forensic value of offline wireless packet analysis and the security weaknesses of legacy WEP encryption, particularly the impact of initialization vector reuse and the feasibility of recovering a WEP40 key from captured traffic.

PURPOSE OF THE ASSIGNMENT

The purpose of this assignment is to perform a controlled digital forensic examination of a historical WEP wireless packet capture and demonstrate the process of preserving, analysing, decrypting and interpreting wireless network evidence.

The assignment is designed to provide practical experience in examining IEEE 802.11 traffic, identifying WEP-protected communications, analysing initialization vectors, recovering a WEP40 key from an approved historical capture and validating the resulting decryption. It further examines the application-layer information that becomes available after decryption, including network endpoints, communications, HTTP activity and recoverable network objects.

The exercise also provides an opportunity to evaluate the security weaknesses of WEP and relate the forensic observations to modern wireless security practices.

OBJECTIVES OF THE ASSIGNMENT

The objectives of the assignment are to:

- Preserve and verify the integrity of the supplied historical wireless evidence using cryptographic hashing.

- Create and work from a controlled forensic copy of the original evidence.
- Identify the structure and characteristics of the IEEE 802.11 traffic contained in the capture.
- Identify WEP-protected frames and associated wireless network information.
- Examine WEP initialization vector values and identify evidence of IV reuse.
- Recover and validate the WEP40 key using Aircrack-ng against the supplied offline historical capture.
- Decrypt the WEP-protected traffic using Airdecap-ng and verify the success of the decryption.
- Hash and preserve the resulting decrypted capture as a forensic derivative.
- Analyse the decrypted network traffic, including protocols, endpoints, MAC addresses and network conversations.
- Identify and examine HTTP requests and responses contained within the decrypted traffic.
- Extract recoverable HTTP objects from the decrypted capture.
- Identify and hash the recovered objects to establish their types and integrity.
- Determine whether the recovered evidence contains identifiable executable, RTF or other potentially significant document objects.
- Explain the security weaknesses demonstrated by the historical WEP configuration and identify appropriate modern wireless security controls.

METHODOLOGY OF THE ASSIGNMENT

The examination was conducted as an offline forensic analysis of the supplied historical wireless capture. The original compressed evidence was first acquired and preserved without modifying the source file. A SHA-256 hash was calculated to establish an initial integrity reference, after which a preserved working copy was created for analysis.

The working copy was decompressed and its file type was verified before packet analysis commenced. The wireless capture was then examined using TShark to establish the protocol hierarchy, IEEE 802.11 frame types and subtypes, protected-frame characteristics, BSSID information and WEP initialization vector values. Repeated IV values were analysed as part of the assessment of the historical WEP implementation.

The capture was subsequently processed with Aircrack-ng using only the supplied historical dataset to recover the WEP40 key. The recovered key was validated through Airdecap-ng, which produced a decrypted Ethernet PCAP. The decrypted derivative was independently hashed to establish an integrity reference before further analysis.

The decrypted capture was examined to identify protocols, IPv4 and MAC-address communications, ARP relationships and TCP/UDP conversations. HTTP requests and responses were then analysed to identify significant web activity. HTTP objects were exported from the decrypted capture into a separate evidence directory, after which the recovered objects were hashed and examined using file-type identification. A targeted search was also performed for potentially significant formats including RTF documents, executable binaries, archives, PDF files and Microsoft/OLE documents.

The methodology maintained a distinction between the original evidence, working copies and derived forensic artifacts. Findings were based on observable packet, protocol, object and cryptographic evidence rather than assumptions derived solely from filenames, URLs or webpage titles.

TOOLS AND RESOURCES USED

The following tools and resources were used during the examination:

Tool / Resource	Purpose
Kali Linux	Forensic analysis and examination environment
TShark 4.6.6	Packet filtering, protocol analysis, frame classification, endpoint and conversation analysis, HTTP examination and object extraction
Aircrack-ng 1.7	Offline WEP40 key recovery from the supplied historical wireless capture
Airdecap-ng	Offline decryption and validation of WEP-protected traffic
Wireshark	Wireless packet inspection and supporting packet-level analysis
SHA-256 (sha256sum)	Integrity verification of original, working, decrypted and recovered evidence

Tool / Resource	Purpose
file utility	File-signature and recovered-object type identification
find / du	Recovered-object inventory and storage-size measurement
xz / unxz	Decompression of the supplied XZ-compressed evidence
wget	Acquisition of the supplied historical evidence file
Foremost	Installed as an available forensic file-carving utility
Binwalk	Installed as an available forensic file-analysis utility
Supplied historical PCAP/XZ dataset	Primary evidence examined during the practical
Linux terminal and forensic workspace	Controlled environment for evidence processing, hashing and report generation

All wireless key-recovery and decryption activities were performed against the supplied historical/offline capture as part of the authorised laboratory exercise. No live wireless network was targeted or interacted with during the examination.

Evidence Preparation and Analysis Environment

Before beginning analysis of the historical WEP wireless capture, a dedicated forensic workspace was created within the Kali Linux virtual machine. The workspace was divided into separate directories for original evidence, working copies, exported objects, reports, screenshots and supporting scripts. This separation provides an organized working environment and helps prevent accidental modification of the original evidence during subsequent analysis.

```
VMware Workstation
kali-linux-2025.2-vmware...
File Actions Edit View Help
(kali@kali)-[~]
$ mkdir -p ~/SBT-DF203-Lab9/{evidence,working,exported,reports,screenshots,scripts}
cd ~/SBT-DF203-Lab9
pwd
/home/kali/SBT-DF203-Lab9
find . -maxdepth 1 -type d -print
./
./screenshots
./scripts
./working
./reports
./evidence
./exported
```

Installation and Verification of Required Tools

The Lab 9 workflow requires several tools for different stages of the forensic examination. Aircrack-ng is required for WEP key recovery or validation, while Airdecap-ng is used to decrypt the historical WEP capture. Wireshark and TShark provide packet-level and command-line analysis capabilities, while Foremost and Binwalk support recovery and identification of transferred objects. XZ utilities are required to decompress the supplied .xz evidence file.

```
VMware Workstation
kali-linux-2025.2-vmware...
File Actions Edit View Help
(kali@kali)-[~/SBT-DF203-Lab9]
$ sudo apt install -y aircrack-ng wireshark tshark foremost binwalk xz-utils
[sudo] password for kali:
wireshark is already the newest version (4.6.6-1).
tshark is already the newest version (4.6.6-1).
The following packages were automatically installed and are no longer required:
aspnetcore-runtime-6.0 libgl1-mesa-dev libqt6test6
aspnetcore-targeting-pack-6.0 libglx-dev libqt6xml6
bloodhound.py libpgmep6t64 libraw23t64
dotnet-apphost-pack-6.0 libgssrpc4t64 libbsd12-image-2.0-0
dotnet-host libhdf4-0-alt libbsd12-mixer-2.0-0
dotnet-hostfxr-6.0 libice-dev libbsd12-ttf-2.0-0
dotnet-runtime-6.0 libjs-jquery-ui libwireshark18
dotnet-runtime-deps-6.0 libjs-underscore libwiretap15
dotnet-sdk-6.0 libkadm5clnt-mit12 libwsutil16
dotnet-targeting-pack-6.0 libkadm5srv-mit12 libx11-dev
gir1.2-girepository-2.0 libkdb5-10t64 libxar1
hyphen-en-us libkmlbase1t64 libxau-dev
lib32gcc-s1 libkmlcore1t64 libxcb1-dev
lib32stdc++6 libkmlengine1t64 libxdmcp-dev
libann0 liblab-gamut1 libxext-dev
libarmadillo14 libllvml18 libxmp4
libavc1394-0 libllvmspirvlib18.1 libxxhash-dev
libc6-i386 liblz4-dev libz3-dev
libcapstone-dev liblzma-dev libzxing3
libcdt5 libmagic-dev llvm-18
libcgraph6 libmbedcrypto16 llvm-18-linker-tools
libclang-common-18-dev libmjpegutils-2.1-0t64 llvm-18-runtime
libclang-common-19-dev libmpeg2encpp-2.1-0t64 llvm-18-tools
libclang-cpp18 libmpeg2-2.1-0t64 llvm-19
libclang-cpp19 libobjc-14-dev llvm-19-linker-tools
libclang-rt-18-dev libobjc4 llvm-19-runtime
libclang-rt-19-dev libogdi4.1 llvm-19-tools
libclang1-18 libopencl-dev llvm-spirv-18
libclang1-19 libopusfile0 netstandard-targeting-pack-2.1
libcrypt-dev libpfm4 python3-aardwolf
libffi-dev libpoc12-common python3-aesedb
libgavl-1 libpoppler147 python3-aioasm
libgc1 python3-aiohttp python3-aiohttp-libs
libgdal36 python3-aiohttp-jinja2 python3-aiohttp-jinja2-libs
libgeos3.13.1 libqt6openglwidgets6 python3-altgraph
```

**Relogin or restarts required!**
Your session is running obsolete binaries or libraries listed below.
Please consider a relogin or restart of the affected processes!
apt[44054], xfce4-panel[12565], xfce4-session[12565]

python3-asciitree	python3-pyppkatz
python3-asn1tools	python3-pyqt6.sip
python3-asyauth	python3-pyvnc
python3-asysocks	python3-secretsocks
python3-bitstruct	python3-sqlalchemy-utc
python3-cssselect2	python3-stix2
python3-docopt	python3-stix2-patterns
python3-donut	python3-stone
python3-dploit	python3-terminaltables
python3-dropbox	python3-unicrypto
python3-fs	python3-websocket
python3-gpg	python3-websockify
python3-humanize	python3-winacl
python3-jq	python3-xldr
python3-jwcrypto	python3-xmlutils
python3-libnmap	python3-xlwt
python3-lsassy	python3-xmltodict
python3-macholib	python3-zlib-wrapper
python3-markdown2	python3.13
python3-masky	python3.13-minimal
python3-md2pdf	python3.13-tk
python3-minidump	python3.13-venv
python3-minikerberos	rpcsvc-proto
python3-msldap	samba-ad-dc
python3-neo4j	samba-ad-provision
python3-neobolt	samba-dsdb-modules
python3-neotime	starkiller
python3-obfuscator	weasyprint
python3-oscrypto	x11proto-dev
python3-pydispatch	xar
python3-pydf	xorg-sgml-doctools
python3-pygame	xtrans-dev
python3-pylnk3	zipcmp
python3-pymysql	zipmerge
python3-pynfsclient	ziptool

```
nd64 - VMware Workstation
Help [Icons]
kali-linux-2025.2-vmware... X
File Actions Edit View Help
libgl-dev libqt6sql6-sqlite python3-arc4 python3-pyp
Use 'sudo apt autoremove' to remove them.

Upgrading:
  aircrack-ng binwalk liblzma-dev liblzma5 python3-binwalk xz-utils

Installing:
  foremost

Summary:
  Upgrading: 6, Installing: 1, Removing: 0, Not Upgrading: 2275
  Download size: 727 kB / 2,168 kB
  Space needed: 266 kB / 12.5 GB available

Get:1 https://kali.download/kali kali-rolling/main amd64 aircrack-ng amd64 1:1.7+git20230807.4bf83f1a-3 [557 kB]
Get:2 https://kali.download/kali kali-rolling/main amd64 python3-binwalk all 2.4.3+dfsg1-3 [122 kB]
Get:3 https://kali.download/kali kali-rolling/main amd64 binwalk all 2.4.3+dfsg1-3 [6,008 B]
Get:4 https://kali.download/kali kali-rolling/main amd64 foremost amd64 1.5.7-12+b1 [41.8 kB]
Fetched 727 kB in 13s (57.0 kB/s)
(Reading database... 417986 files and directories currently installed.)
Preparing to unpack .../liblzma-dev_5.8.3-1_amd64.deb...
Unpacking liblzma-dev:amd64 (5.8.3-1) over (5.8.1-1)...
Preparing to unpack .../liblzma5_5.8.3-1_amd64.deb...
Unpacking liblzma5:amd64 (5.8.3-1) over (5.8.1-1)...
Setting up liblzma5:amd64 (5.8.3-1)...
(Reading database... 417983 files and directories currently installed.)
Preparing to unpack .../xz-utils_5.8.3-1_amd64.deb...
Unpacking xz-utils (5.8.3-1) over (5.8.1-1)...
Preparing to unpack .../aircrack-ng_1%3a1.7+git20230807.4bf83f1a-3_amd64.deb...
Unpacking aircrack-ng (1:1.7+git20230807.4bf83f1a-3) over (1:1.7+git20230807.4bf83f1a-2)...
Preparing to unpack .../python3-binwalk_2.4.3+dfsg1-3_all.deb...
Unpacking python3-binwalk (2.4.3+dfsg1-3) over (2.4.3+dfsg1-2)...
Preparing to unpack .../binwalk_2.4.3+dfsg1-3_all.deb...
Unpacking binwalk (2.4.3+dfsg1-3) over (2.4.3+dfsg1-2)...
Selecting previously unselected package foremost.
Preparing to unpack .../foremost_1.5.7-12+b1_amd64.deb...
Unpacking foremost (1.5.7-12+b1)...
Setting up foremost (1.5.7-12+b1)...
Setting up aircrack-ng (1:1.7+git20230807.4bf83f1a-3)...
Setting up xz-utils (5.8.3-1)...
Setting up liblzma-dev:amd64 (5.8.3-1)...
Setting up python3-binwalk (2.4.3+dfsg1-3)...
Setting up binwalk (2.4.3+dfsg1-3)...
```

Acquisition of the Historical WEP Capture

The authorised historical WEP training capture supplied for Lab 9 was downloaded into the dedicated evidence directory of the forensic workspace. The download initially encountered a read/decryption error after approximately 65% of the file had been transferred. The download utility automatically resumed the transfer using an HTTP partial-content response and subsequently completed the remaining portion. The final output reported that evidence/file.xz was saved with a total size of **12,837,660 bytes**.

```
(kali@kali)-[~/SBT-DF203-Lab9]
$ wget -O evidence/file.xz \
'https://raw.githubusercontent.com/ctfs/write-ups-2015/master/codegate-ctf-2015/programming/good-crypto/file.xz'
--2026-09-25 09:54:05-- https://raw.githubusercontent.com/ctfs/write-ups-2015/master/codegate-ctf-2015/programming/good-crypto/file.xz
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.110.133, 185.199.111.133, 185.199.109.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.110.133|:443 ... connected.
HTTP request sent, awaiting response... 200 OK
Length: 12837660 (12M) [application/octet-stream]
Saving to: 'evidence/file.xz'

evidence/file.xz          65%[=====] 8.04M --.-KB/s in 10m 0s

2026-09-25 10:04:14 (13.7 KB/s) - Read error at byte 8431616/12837660 (Decryption has failed.). Retrying.

--2026-09-25 10:04:15-- (try: 2) https://raw.githubusercontent.com/ctfs/write-ups-2015/master/codegate-ctf-2015/programming/good-crypto/file.xz
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.110.133|:443 ... connected.
HTTP request sent, awaiting response... 206 Partial Content
Length: 12837660 (12M), 4406044 (4.2M) remaining [application/octet-stream]
Saving to: 'evidence/file.xz'

evidence/file.xz          100%[=====] 12.24M 95.0KB/s in 62s

2026-09-25 10:05:22 (69.4 KB/s) - 'evidence/file.xz' saved [12837660/12837660]

(kali@kali)-[~/SBT-DF203-Lab9]
$
```

SHA-256 Hashing of the Original Evidence

After successfully acquiring the historical WEP capture, a SHA-256 cryptographic hash was calculated for the original compressed evidence file. The hash provides a unique integrity reference for evidence/file.xz and establishes a baseline against which the file can be verified throughout the forensic examination. The hash output was also saved to reports/file_xz_sha256.txt to maintain a persistent record within the investigation workspace.

```
(kali@kali)-[~/SBT-DF203-Lab9]
$ sha256sum evidence/file.xz | tee reports/file_xz_sha256.txt
dd54144caef34f228bfb4b87a9101ca7173969376f7ed357bd068061d4f4b8d6 evidence/file.xz

(kali@kali)-[~/SBT-DF203-Lab9]
$
```

The SHA-256 value serves as the integrity baseline for the original compressed evidence. If the contents of evidence/file.xz were altered after this point, recalculating its SHA-256 hash would produce a different value. Recording the hash before creating or analysing derived working files therefore provides an auditable reference for the original evidence.

The original historical WEP capture was successfully hashed, producing the SHA-256 digest dd54144caef34f228bfb4b87a9101ca7173969376f7ed357bd068061d4f4b8d6. The integrity value was preserved in the laboratory reports directory, establishing the original compressed capture as the reference evidence for subsequent analysis.

Creation of the Working Copy

Following the SHA-256 verification of the original evidence, a separate working copy of the compressed capture was created in the working directory. The analysis was deliberately separated from the preserved source file so that subsequent decompression and forensic examination would not modify the original evidence. The `--preserve=timestamps` option was used to retain the original file timestamps on the working copy.

```
(kali㉿kali)-[~/SBT-DF203-Lab9]
$ cp --preserve=timestamps evidence/file.xz working/file_working.xz

(kali㉿kali)-[~/SBT-DF203-Lab9]
$
```

The separation between the evidence and working directories establishes a controlled evidence-handling process. The original `evidence/file.xz` remains preserved as the source artefact, while `working/file_working.xz` becomes the file used for subsequent decompression and analysis. Preserving the timestamps also helps retain relevant file metadata when creating the working derivative.

A working copy of the original compressed capture was successfully created at `working/file_working.xz`. No error was reported during the copy operation, allowing subsequent processing to be performed on the working copy without directly modifying the preserved source evidence.

Decompression of the Working Evidence

The compressed working copy was decompressed to produce the network capture required for packet-level forensic analysis. Decompression was performed on `working/file_working.xz`, rather than on the preserved original evidence, maintaining the separation between source evidence and derived analysis files. The `-k` option was used to retain the compressed working copy after decompression.

```
(kali㉿kali)-[~/SBT-DF203-Lab9]
$ unxz -k working/file_working.xz

(kali㉿kali)-[~/SBT-DF203-Lab9]
$
```

The uncompressed capture is the derivative that will be used for subsequent wireless packet analysis. Retaining the .xz working copy alongside the decompressed file also preserves a clear relationship between the compressed working derivative and the file used for analysis.

The historical wireless capture was successfully decompressed from the working copy without an error being reported. The resulting uncompressed capture is now available in the working directory for format verification and subsequent 802.11/WEP analysis.

Verification of the Decompressed Capture Format

After decompression, the resulting file was examined using the Linux file utility to verify its format and confirm that it was suitable for subsequent packet-level forensic analysis. This verification establishes the type of network capture, its timestamp format, byte ordering, capture format version and the maximum captured frame length before further analysis is performed.

```
(kali@kali)-[~/SBT-DF203-Lab9]
$ file working/file_working
working/file_working: pcap capture file, microsecond ts (little-endian) - version 2.4 (802.11, capture length 65535)
(kali@kali)-[~/SBT-DF203-Lab9]
```

The format verification confirms that the decompressed derivative is a valid PCAP network capture containing 802.11 traffic. This is significant because the remainder of the investigation requires wireless-frame analysis, including identification of WEP-protected traffic, BSSID and station addresses, IV values and subsequent offline decryption. The microsecond timestamp resolution also provides precise timing information for analysing the sequence of captured wireless frames.

The decompressed evidence was confirmed to be a PCAP version 2.4 capture containing 802.11 wireless traffic, with microsecond timestamps, little-endian byte ordering and a capture length of 65,535 bytes. The file is therefore suitable for the planned Wireshark, TShark and Aircrack-ng analysis.

SHA-256 Hashing of the Working Evidence and Decompressed Capture

After verifying the format of the decompressed capture, SHA-256 hashes were calculated for both the compressed working copy and its decompressed derivative. This provides integrity references for the artefacts that will be used during the subsequent forensic

analysis. The hash of the compressed working copy can also be compared with the previously recorded hash of the original evidence to confirm that the working copy was transferred without alteration.

The compressed working copy, `working/file_working.xz`, produced the hash:

`dd54144caef34f228bfb4b87a9101ca7173969376f7ed357bd068061d4f4b8d6`

The decompressed capture, `working/file_working`, produced the hash:

`c17a3f9b955e84f5befd476dbd55c67286d1e3eea9ab402d5359cac0874ebb2d`

The output was also saved to `reports/working_hashes.txt` for inclusion in the laboratory evidence record.

Integrity comparison

There is an important observation here is that the SHA-256 hash of `working/file_working.xz` is **identical** to the SHA-256 hash previously calculated for the original evidence/`file.xz`:

Original: `dd54144caef34f228bfb4b87a9101ca7173969376f7ed357bd068061d4f4b8d6`

Working copy:

`dd54144caef34f228bfb4b87a9101ca7173969376f7ed357bd068061d4f4b8d6`

This confirms that the compressed working copy has the same cryptographic content as the preserved source file.

The decompressed file necessarily has a different hash, because decompression changes the representation and contents of the file from the compressed XZ format into the raw PCAP format. That is expected and does not indicate corruption.

```
(kali@kali)~[~/SBT-DF203-Lab9]
$ sha256sum evidence/file.xz | tee reports/file_xz_sha256.txt
dd54144caef34f228bfb4b87a9101ca7173969376f7ed357bd068061d4f4b8d6  evidence/file.xz

(kali@kali)~[~/SBT-DF203-Lab9]
$ The SHA-256 value serves as the integrity baseline for the original compressed evidence. If the contents of evidence/file.xz were altered after this point, recalculating its SHA-256 hash would produce a different value. Recording the hash before creating or analysing derived working files therefore provides an auditable reference for the original evidence.

Finding
The original historical WEP capture was successfully hashed, producing the SHA-256 digest dd54144caef34f228bfb4b87a9101ca7173969376f7ed357bd068061d4f4b8d6. The integrity value was preserved in the laboratory reports directory, establishing the original compressed capture as the reference evidence for subsequent analysis.

(kali@kali)~[~/SBT-DF203-Lab9]
$ cp --preserve-timestamps evidence/file.xz working/file_working.xz

(kali@kali)~[~/SBT-DF203-Lab9]
$ unxz -k working/file_working.xz

(kali@kali)~[~/SBT-DF203-Lab9]
$ file working/file_working
working/file_working: pcap capture file, microsecond ts (little-endian) - version 2.4 (802.11, capture length 65535)

(kali@kali)~[~/SBT-DF203-Lab9]
$ sha256sum working/file_working.xz working/file_working | tee reports/working_hashes.txt
dd54144caef34f228bfb4b87a9101ca7173969376f7ed357bd068061d4f4b8d6  working/file_working.xz
c17a3f9b955e84f5befd476dbd55c67286d1e3eea9ab402d5359cac0874ebb2d  working/file_working

(kali@kali)~[~/SBT-DF203-Lab9]
$
```

Matching SHA-256 values between the original evidence/file.xz and working/file_working.xz provide evidence that the working copy is an exact cryptographic match to the preserved source. The separate hash for working/file_working establishes an integrity baseline for the decompressed PCAP derivative that will be subjected to packet analysis. Recording these values creates an auditable chain between the preserved source evidence and the analysis copy.

The compressed working copy was verified against the original evidence using SHA-256 and produced the same digest, dd54144caef34f228bfb4b87a9101ca7173969376f7ed357bd068061d4f4b8d6. The decompressed PCAP derivative was independently hashed as c17a3f9b955e84f5befd476dbd55c67286d1e3eea9ab402d5359cac0874ebb2d. The working evidence is therefore ready for subsequent offline wireless traffic analysis.

At this point we have established:

Evidence stage	Artefact	SHA-256
Original source	evidence/file.xz	dd54144caef34f228bfb4b87a9101ca7173969376f7ed357bd068061d4f4b8d6

Evidence stage	Artefact	SHA-256
Verified working copy	working/file_working.xz	dd54144caef34f228bfb4b87a9101ca7173969376f7ed357bd068061d4f4b8d6
Decompressed derivative	working/file_working	c17a3f9b955e84f5befd476dbd55c67286d1e3eea9ab402d5359cac0874ebb2d

Initial 802.11 Protocol Hierarchy Analysis

Following preservation, hashing and format verification of the historical capture, an initial read-only protocol hierarchy analysis was performed using TShark. The purpose of this step was to establish the overall composition of the capture before examining individual wireless frames, WEP protection, BSSID information, IV values and endpoint activity. The analysis was performed against the decompressed working copy, working/file_working, ensuring that the preserved original evidence remained unchanged.

```
(kali@kali)-[~/SBT-DF203-Lab9]
$ sha256sum working/file_working.xz working/file_working | tee reports/working_hashes.txt
dd54144caef34f228bfb4b87a9101ca7173969376f7ed357bd068061d4f4b8d6  working/file_working.xz
c17a3f9b955e84f5befd476dbd55c67286d1e3eea9ab402d5359cac0874ebb2d  working/file_working

(kali@kali)-[~/SBT-DF203-Lab9]
$ tshark -r working/file_working -q -z io,phs
Warning: program compiled against libxml 215 using older 214

Protocol Hierarchy Statistics
Filter:

frame                               frames:45169 bytes:13511274
 wlan                               frames:45169 bytes:13511274
  data                              frames:15713 bytes:13104128

(kali@kali)-[~/SBT-DF203-Lab9]
$
```

The screenshot above presents the protocol hierarchy statistics generated from the historical capture. TShark identified a total of 45,169 frames, representing approximately 13.51 MB of captured traffic. All identified frames are classified at the WLAN layer, confirming that the capture consists of IEEE 802.11 wireless traffic.

Within the WLAN traffic, 15,713 frames are classified as data frames, accounting for approximately 13.10 MB of the captured bytes. The protocol hierarchy output therefore

establishes that the capture contains a substantial volume of wireless data traffic rather than merely management or control traffic.

The important output is:

```
frame  frames:45169 bytes:13511274
```

```
wlan  frames:45169 bytes:13511274
```

```
data frames:15713 bytes:13104128
```

The protocol hierarchy shows that all 45,169 captured frames are associated with the WLAN protocol layer. Of these, 15,713 frames are identified as data frames. This indicates that the capture contains significant wireless data traffic suitable for further examination of protected communications and WEP characteristics.

The remaining frames are not identified as data by this particular protocol-hierarchy view and will need to be examined through specific 802.11 frame-type filters rather than assumed to be a particular category.

Establishing the protocol hierarchy provides an initial baseline for the wireless investigation. It confirms that the evidence is an 802.11 capture and quantifies the volume of traffic available for further analysis. The presence of 15,713 WLAN data frames is particularly relevant because WEP protection is applied to wireless data traffic and these frames can be examined for protected-frame indicators, initialization vectors and other characteristics associated with the WEP protocol.

802.11 Frame Category Analysis

After establishing the overall protocol hierarchy of the capture, the next stage was to identify the IEEE 802.11 frame types and subtypes present in the evidence. Wireless LAN traffic is broadly divided into management, control and data frames, with each category serving a different purpose in wireless communication. Examining the frame-type distribution provides an initial understanding of the activity represented in the historical capture and helps identify the traffic relevant to subsequent WEP analysis. The analysis was performed using TShark against the decompressed working capture.

```
(kali@kali)-[~/SBT-DF203-Lab9]
$ tshark -r working/file_working -Y 'wlan.fc.type' \
-T fields -e wlan.fc.type -e wlan.fc.subtype \
| sort | uniq -c
Warning: program compiled against libxml 215 using older 214
  1 0      0
  1 0      1
  3 0     11
 219 0      5
  1 0      8
10938 1     11
  76 1     12
17127 1     13
  3 1     14
 468 1      8
  35 1      9
 117 2      0
  3 2     12
 582 2      4
15595 2      8
```

This shows the number of occurrences of each 802.11 frame type and subtype identified in the capture. The first value represents the IEEE 802.11 frame type, while the second value represents the associated subtype.

The three principal frame types are:

wlan.fc.type	Category	Meaning
0	Management	Frames used to establish, maintain and manage wireless communication
1	Control	Frames used to coordinate and control wireless transmission
2	Data	Frames carrying data between wireless stations

The capture contains all three categories.

Observed Management Frames

The output contains:

1 0 0
1 0 1
3 0 11
219 0 5

1 0 8

These are management-frame subtypes. The significant observation is that 225 management frames were identified in total.

For example:

- 0 / 0 - Association Request
- 0 / 1 - Association Response
- 0 / 5 - Probe Response
- 0 / 8 - Beacon
- 0 / 11 - Authentication

This shows that the capture contains wireless network management activity, including authentication and network discovery/advertisement activity.

Observed Control Frames

The capture contains:

10938 1 11

76 1 12

17127 1 13

3 1 14

468 1 8

35 1 9

These represent 28,647 control frames.

The observed subtypes include:

- 1 / 8 - Block Acknowledgment
- 1 / 9 - Block Acknowledgment Request

- 1 / 11 - Request to Send (RTS)
- 1 / 12- Clear to Send (CTS)
- 1 / 13 - Acknowledgment (ACK)
- 1 / 14 - Contention-Free End (CF-End)

The particularly large counts for RTS/ACK-related control traffic indicate substantial wireless transmission-control activity within the historical capture.

Observed Data Frames

The output contains:

117 2 0

3 2 12

582 2 4

15595 2 8

This gives 16,297 data frames.

The observed data subtypes include:

- 2 / 0 - Data
- 2 / 4 - Null Function
- 2 / 8 - QoS Data
- 2 / 12 - QoS Null

The large number of 2 / 8 frames indicates that QoS Data frames make up a substantial portion of the captured data traffic.

The frame-type analysis confirms that the historical capture contains management, control and data traffic. Management frames provide evidence of wireless network establishment and authentication activity, control frames demonstrate transmission coordination between wireless stations, and data frames represent the traffic most

relevant to examining protected WEP communications. This classification establishes the wireless protocol context required for the subsequent identification of WEP-protected frames and repeated initialization vectors.

The capture contains all three principal IEEE 802.11 frame categories. The observed traffic includes management activity such as association, authentication, probe responses and beacons; control activity including RTS, CTS, ACK and Block Acknowledgment frames; and data activity including standard, QoS and null-data frames. The presence of substantial data traffic provides the basis for proceeding to the next stage of the investigation: identifying and quantifying WEP-protected frames

WEP-Protected Frames and Initialization Vector Values

Following the initial 802.11 frame classification, the capture was examined specifically for frames marked as protected using the IEEE 802.11 Protected Frame flag. The analysis was performed with TShark against the verified decompressed working capture. The wlan.wep.iv field was used to extract the Initialization Vector (IV), as this TShark version identifies the WEP IV using the wlan.wep.iv field.

The resulting output provides the frame number, relative capture time, source MAC address, destination MAC address, BSSID, and WEP Initialization Vector for each protected frame returned by the filter.

```
(kali@kali)-[~/SBT-DF203-Lab9]
$ tshark -r working/file_working -Y 'wlan.fc.protected = 1' \
-T fields \
-e frame.number \
-e frame.time_relative \
-e wlan.sa \
-e wlan.da \
-e wlan.bssid \
-e wlan.wep.iv \
| head -20
Warning: program compiled against libxml 215 using older 214
9432  4.868456000  00:26:66:55:97:d4  ff:ff:ff:ff:ff:ff  00:26:66:55:97:d6  0x67ffec
10130  5.773726000  f0:f6:1c:68:96:7c  00:26:66:55:97:d6  00:26:66:55:97:d6  0xc2cbeb
10155  5.812130000  f0:f6:1c:68:96:7c  04:8d:38:48:e5:b4  00:26:66:55:97:d6  0xc3cbeb
10157  5.812642000  f0:f6:1c:68:96:7c  04:8d:38:48:a8:b5  00:26:66:55:97:d6  0xc4cbeb
10159  5.812642000  f0:f6:1c:68:96:7c  08:10:77:92:7c:2f  00:26:66:55:97:d6  0xc5cbeb
10161  5.812642000  f0:f6:1c:68:96:7c  00:26:66:55:97:d4  00:26:66:55:97:d6  0xc6cbeb
10163  5.812642000  f0:f6:1c:68:96:7c  c8:3a:35:58:c4:ba  00:26:66:55:97:d6  0xc7cbeb
10165  5.812641000  f0:f6:1c:68:96:7c  ff:ff:ff:ff:ff:ff  00:26:66:55:97:d6  0xc8cbeb
10167  5.813160000  00:26:66:55:97:d4  f0:f6:1c:68:96:7c  00:26:66:55:97:d6  0x68ffec
10170  5.816234000  f0:f6:1c:68:96:7c  ff:ff:ff:ff:ff:ff  00:26:66:55:97:d6  0x69ffec
10171  5.817762000  f0:f6:1c:68:96:7c  33:33:00:00:00:02  00:26:66:55:97:d6  0xc9cbeb
10172  5.822891000  00:26:66:55:97:d4  ff:ff:ff:ff:ff:ff  00:26:66:55:97:d6  0x6affec
10173  5.822882000  f0:f6:1c:68:96:7c  33:33:00:00:00:02  00:26:66:55:97:d6  0xc9cbeb
10175  5.823393000  f0:f6:1c:68:96:7c  ff:ff:ff:ff:ff:ff  00:26:66:55:97:d6  0xcacbeb
10177  5.824426000  f0:f6:1c:68:96:7c  33:33:00:00:00:02  00:26:66:55:97:d6  0x6bffec
10178  5.825450000  f0:f6:1c:68:96:7c  ff:ff:ff:ff:ff:ff  00:26:66:55:97:d6  0x6cffec
10183  5.827490000  f0:f6:1c:68:96:7c  ff:ff:ff:ff:ff:ff  00:26:66:55:97:d6  0xcbcbeb
10185  5.828522000  f0:f6:1c:68:96:7c  ff:ff:ff:ff:ff:ff  00:26:66:55:97:d6  0x6dffec
10224  5.894562000  f0:f6:1c:68:96:7c  ff:ff:ff:ff:ff:ff  00:26:66:55:97:d6  0xcccbeb
10228  5.898154000  f0:f6:1c:68:96:7c  ff:ff:ff:ff:ff:ff  00:26:66:55:97:d6  0x6effec
```

The screenshot shows multiple protected wireless frames associated with BSSID 00:26:66:55:97:d6. The first displayed protected frame is frame 9432, originating from 00:26:66:55:97:d4 and addressed to the broadcast MAC address ff:ff:ff:ff:ff:ff, with an IV of 0x67ffec.

From frame 10130 onward, a significant sequence of protected traffic is visible. The station f0:f6:1c:68:96:7c appears repeatedly as the source, communicating with the access point identified by BSSID 00:26:66:55:97:d6. The destination addresses include individual MAC addresses as well as broadcast and multicast addresses such as ff:ff:ff:ff:ff:ff and 33:33:00:00:00:02.

The IV values are also clearly exposed in the decrypted packet metadata. For example, frames 10130 through 10167 show sequential IV values including 0xc2cbeb, 0xc3cbeb, 0xc4cbeb, 0xc5cbeb, 0xc6cbeb, 0xc7cbeb, and 0xc8cbeb. This demonstrates that the capture contains substantial WEP-protected traffic and provides the IV information required for subsequent offline WEP analysis.

The output also establishes that the protected traffic involves several wireless endpoints rather than a single source-destination pair. The repeated BSSID 00:26:66:55:97:d6 provides an important reference point for subsequent endpoint and access-point analysis. The presence of protected 802.11 frames together with exposed 24-bit WEP IV values is significant because WEP relies on the combination of a secret key and a short IV to generate the RC4 keystream used for encryption. The limited IV space can result in IV reuse as traffic volume increases, which is one of the fundamental weaknesses investigated in this laboratory. The extracted IV values therefore provide the basis for the next stage of the examination: determining the volume and characteristics of the WEP traffic and assessing IV reuse/weak-IV conditions before attempting offline key recovery with Aircrack-ng.

Analysis of WEP Initialization Vector Reuse Analysis

To determine whether Initialization Vector (IV) values were reused within the protected wireless traffic, the extracted WEP IV values were grouped and counted using TShark, sort, and uniq. The results were then ordered by frequency so that the most frequently repeated IV values appeared first.

```

(kali㉿kali)-[~/SBT-DF203-Lab9]
$ tshark -r working/file_working -Y 'wlan.fc.protected = 1' \
-T fields -e wlan.wep.iv \
| sort | uniq -c | sort -nr | head -20
Warning: program compiled against libxml 215 using older 214
 7 0x42e8eb
 7 0x28d3eb
 6 0x95dfeb
 5 0x9415ed
 5 0x86e5eb
 5 0x7ae5eb
 5 0x28e7eb
 5 0x19e5eb
 4 0xd5d7eb
 4 0xd3e8eb
 4 0xc915ed
 4 0xc7d8eb
 4 0xade4eb
 4 0x73e5eb
 4 0x65d8eb
 4 0x4ad5eb
 4 0x31e7eb
 4 0x2723ed
 4 0x10e7eb
 3 0xffe5eb

```

The screenshot demonstrates that several WEP Initialization Vector values occur multiple times within the historical capture. The most frequently observed values include:

IV Value	Number of Occurrences
0x42e8eb	7
0x28d3eb	7
0x95dfeb	6
0x9415ed	5
0x86e5eb	5
0x7ae5eb	5
0x28e7eb	5
0x19e5eb	5

IV Value	Number of Occurrences
0xd5d7eb	4
0xd3e8eb	4

The highest-frequency IV values, 0x42e8eb and 0x28d3eb, each occur seven times in the captured protected traffic. Other IVs occur between three and six times in the displayed top-20 results.

This establishes that the capture contains reused WEP IV values rather than every protected frame having a unique IV. WEP uses a 24-bit Initialization Vector together with the configured secret key when generating the RC4 keystream. Because the IV space is limited, the same IV can occur again during sufficiently large volumes of wireless traffic. Reuse is particularly significant in WEP because observations sharing the same IV and key can provide information that can be exploited by WEP-cracking techniques.

The repeated IV values identified in this capture therefore provide direct evidence of a weakness in the historical WEP implementation and explain why the supplied capture is suitable for the subsequent authorised offline Aircrack-ng analysis. The repeated values themselves do not constitute the recovered WEP key; rather, they represent the captured evidence that supports the next stage of key-recovery analysis.

Protected Frame Volume and BSSID Identification and Distribution

After confirming the presence of repeated WEP Initialization Vectors, the protected traffic was quantified to establish the size and scope of the encrypted portion of the historical wireless capture. TShark was used to count all frames matching the IEEE 802.11 Protected Frame condition and then to determine the BSSID associated with those frames.

The first command returned 15,713 protected frames. This is consistent with the data frame count previously observed in the protocol hierarchy statistics, providing an additional view of the protected traffic volume.

```
(kali@kali)-[~/SBT-DF203-Lab9]
$ tshark -r working/file_working -Y 'wlan.fc.protected == 1' \
-T fields -e frame.number \
| wc -l
Warning: program compiled against libxml 215 using older 214
15713

(kali@kali)-[~/SBT-DF203-Lab9]
$ tshark -r working/file_working -Y 'wlan.fc.protected == 1' \
-T fields -e wlan.bssid \
| sort | uniq -c | sort -nr
Warning: program compiled against libxml 215 using older 214
15713 00:26:66:55:97:d6
```

The BSSID analysis produced a single result:

BSSID	Protected Frames
00:26:66:55:97:d6	15,713

This demonstrates that all 15,713 protected frames identified by the filter are associated with the same BSSID, 00:26:66:55:97:d6.

The result establishes 00:26:66:55:97:d6 as the principal access-point/BSSID identifier for the protected wireless network examined in this laboratory. It also provides a consistent network identifier for subsequent Aircrack-ng analysis and endpoint investigation.

The concentration of all identified protected traffic under a single BSSID indicates that the WEP-protected activity in the supplied historical capture is associated with one identifiable wireless network. This simplifies the subsequent forensic examination because the protected traffic can be analysed as a single WEP network rather than having to separate multiple protected BSSIDs.

The combination of the findings from the preceding analyses is significant: the capture contains 15,713 protected frames, those frames are associated with BSSID 00:26:66:55:97:d6, and repeated WEP IV values have already been observed. Together, these findings establish that the supplied capture contains a substantial volume of WEP-protected traffic suitable for the authorised offline key-recovery stage.

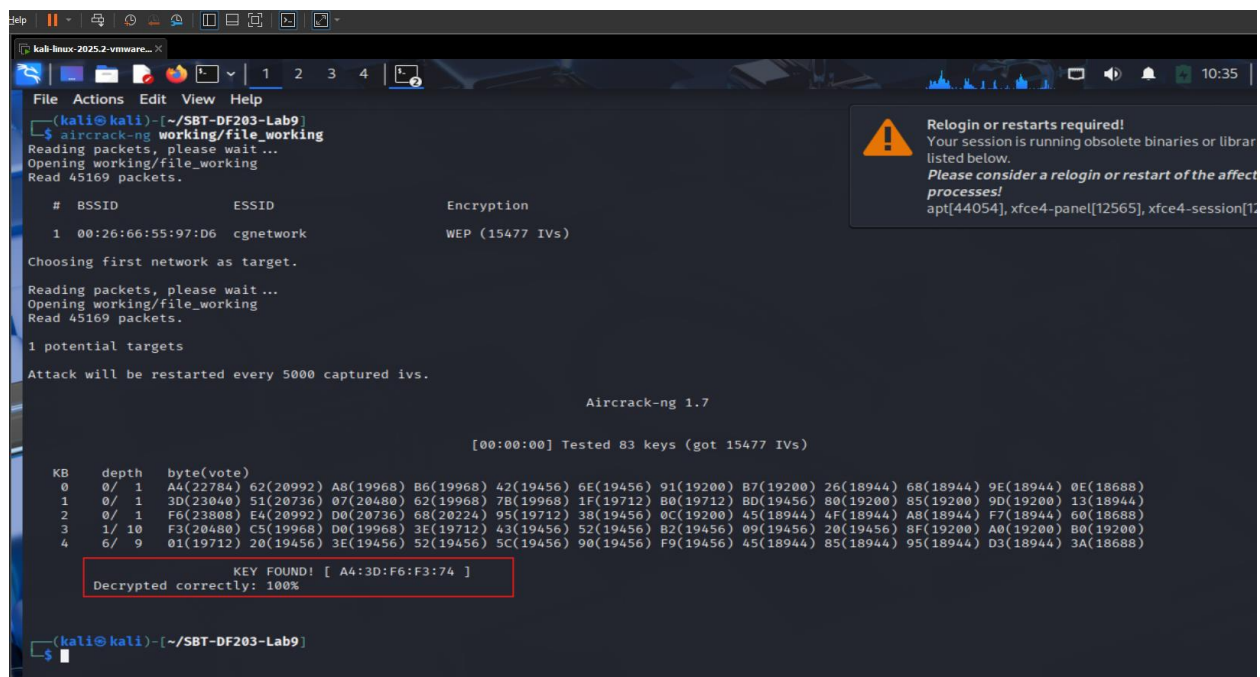
Aircrack-ng Offline WEP Key Recovery Analysis

Following the identification of the protected traffic, repeated Initialization Vectors, and the primary BSSID, Aircrack-ng was used to perform offline analysis of the supplied historical capture. The analysis was conducted directly against the previously verified working PCAP and did not involve interaction with a live wireless network.

Aircrack-ng first examined the capture and identified 45,169 packets. It identified one wireless target:

- BSSID: 00:26:66:55:97:D6
- ESSID: cgnetwork
- Encryption: WEP
- Captured IVs: 15,477

The tool reported one potential target and proceeded with the WEP key-recovery process. During the analysis, Aircrack-ng tested candidate key values using the captured IVs and progressively ranked possible byte values according to their votes.



The screenshot shows the Aircrack-ng terminal interface. It displays the command `aircrack-ng working/file_working` and the subsequent analysis of 45,169 packets. The output identifies a target network with BSSID 00:26:66:55:97:D6, ESSID cgnetwork, and WEP encryption using 15,477 IVs. The tool then proceeds to test candidate key values, showing a table of results for the first four keys. The final output indicates that the key was found: `KEY FOUND! [ A4:3D:F6:F3:74 ]` and that the decryption was successful (100%).

```
File Actions Edit View Help
(kali@kali)-[~/SBT-DF203-Lab9]
$ aircrack-ng working/file_working
Reading packets, please wait ...
Opening working/file_working
Read 45169 packets.

# BSSID      ESSID      Encryption
1 00:26:66:55:97:D6  cgnetwork  WEP (15477 IVs)

Choosing first network as target.
Reading packets, please wait ...
Opening working/file_working
Read 45169 packets.

1 potential targets
Attack will be restarted every 5000 captured ivs.

Aircrack-ng 1.7

[00:00:00] Tested 83 keys (got 15477 IVs)

KB  depth  byte(vote)
0/  1  A4(22784) 62(20992) A8(19968) B6(19968) 42(19456) 6E(19456) 91(19200) B7(19200) 26(18944) 68(18944) 9E(18944) 0E(18688)
1  0/  1  3D(23040) 51(20736) 07(20480) 62(19968) 7B(19968) 1F(19712) B0(19712) BD(19456) 80(19200) 85(19200) 9D(19200) 13(18944)
2  0/  1  F6(23808) E4(20992) D0(20736) 68(20224) 95(19712) 38(19456) 0C(19200) 45(18944) 4F(18944) A8(18944) F7(18944) 60(18688)
3  1/ 10  F3(20480) C5(19968) D0(19968) 3E(19712) 43(19456) 52(19456) B2(19456) 89(19456) 20(19456) A0(19200) B0(19200)
4  6/  9  01(19712) 20(19456) 3E(19456) 52(19456) 5C(19456) 90(19456) F9(19456) 45(18944) 85(18944) 95(18944) D3(18944) 3A(18688)

KEY FOUND! [ A4:3D:F6:F3:74 ]
Decrypted correctly: 100%

(kali@kali)-[~/SBT-DF203-Lab9]
$
```

The screenshot shows that Aircrack-ng successfully recovered the WEP key:

Recovered WEP40 Key: A4:3D:F6:F3:74

Most importantly, Aircrack-ng reported: Decrypted correctly: 100%

This provides direct validation that the recovered five-byte key successfully decrypts the protected traffic in the supplied historical capture.

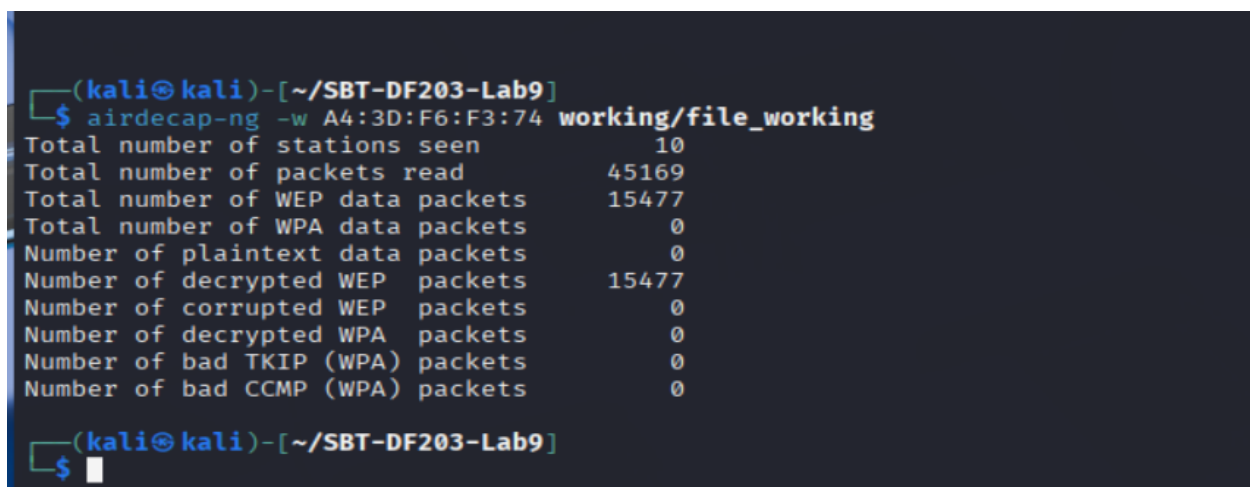
The result confirms that the historical wireless network identified as cgnetwork, using BSSID 00:26:66:55:97:D6, was protected using WEP and that the supplied capture contained sufficient evidence for successful offline recovery of its WEP40 key.

The successful recovery is supported by several preceding findings. The capture contained 15,713 protected frames, a substantial number of WEP IVs were available for analysis, and repeated IV values were identified. Aircrack-ng subsequently recovered the five-byte WEP40 key and independently reported 100% successful decryption.

Offline WEP Decryption and Validation Using Airdecap-ng

After Aircrack-ng successfully recovered the WEP40 key, Airdecap-ng was used to decrypt the historical wireless capture using the recovered key. This provided an independent validation step and produced a decrypted derivative of the working capture for subsequent forensic examination.

The recovered key A4:3D:F6:F3:74 was supplied to Airdecap-ng against working/file_working.

A terminal window with a dark background and light blue text. The prompt is (kali@kali)-[~/SBT-DF203-Lab9]. The command airdecap-ng -w A4:3D:F6:F3:74 working/file_working has been executed. The output shows statistics for the decryption process, including the number of stations, packets, and WEP/WPA data packets. The results indicate that 15,477 WEP data packets were successfully decrypted, while all other categories show zero.

```
(kali@kali)-[~/SBT-DF203-Lab9]
$ airdecap-ng -w A4:3D:F6:F3:74 working/file_working
Total number of stations seen          10
Total number of packets read          45169
Total number of WEP data packets      15477
Total number of WPA data packets       0
Number of plaintext data packets       0
Number of decrypted WEP packets       15477
Number of corrupted WEP packets        0
Number of decrypted WPA packets        0
Number of bad TKIP (WPA) packets       0
Number of bad CCMP (WPA) packets       0

(kali@kali)-[~/SBT-DF203-Lab9]
$
```

The Airdecap-ng output reports the following results:

Metric	Result
Total stations seen	10
Total packets read	45,169
WEP data packets	15,477
WPA data packets	0
Plaintext data packets	0
Decrypted WEP packets	15,477
Corrupted WEP packets	0
Decrypted WPA packets	0
Bad TKIP packets	0
Bad CCMP packets	0

The results show that Aircap-ng identified 10 wireless stations in the capture and processed all 45,169 packets. Of the packets classified as WEP data, all 15,477 packets were successfully decrypted. No WEP packets were reported as corrupted.

The output also confirms that the capture did not contain WPA data packets, while the plaintext data packet count was zero before decryption. The Aircap-ng results provide an independent validation of the WEP key recovered during the Aircrack-ng stage. The successful decryption of all 15,477 WEP data packets, combined with zero corrupted WEP packets, demonstrates that the recovered key was valid for the protected traffic processed from the historical capture.

This result also establishes that the capture can now be examined at higher protocol layers using the decrypted derivative. This is important because the original protected frames cannot provide the same level of visibility into IP-level communications and transferred application data without successful decryption.

The original historical evidence remains preserved separately. The decrypted capture is treated as a forensic derivative created for analysis, allowing subsequent examination and hashing without altering the source evidence.

Verification of the Decrypted Capture Derivative

Following the Airdecap-ng decryption process, the contents of the working directory were examined to confirm that the decrypted derivative had been created successfully and to distinguish it from the preserved compressed evidence and original 802.11 capture.

The directory listing identified three relevant files:

- file_working.xz - the preserved compressed working copy of the historical evidence.
- file_working- the decompressed original capture containing 802.11 wireless frames.
- file_working-dec- the decrypted capture generated by Airdecap-ng.

The file command was then used to verify the format of each file.

```
(kali@kali)-[~/SBT-DF203-Lab9]
$ ls -lh working/
total 39M
-rw-rw-r-- 1 kali kali 14M Sep 25 10:05 file_working
-rw-rw-r-- 1 kali kali 13M Sep 25 10:36 file_working-dec
-rw-rw-r-- 1 kali kali 13M Sep 25 10:05 file_working.xz

(kali@kali)-[~/SBT-DF203-Lab9]
$ file working/*
working/file_working:      pcap capture file, microsecond ts (little-endian) - version 2.4 (802.11, capture length 65535)
working/file_working-dec:  pcap capture file, microsecond ts (little-endian) - version 2.4 (Ethernet, capture length 65535)
working/file_working.xz:   XZ compressed data, checksum CRC64
```

The original decompressed capture was identified as:

pcap capture file, microsecond ts (little-endian) - version 2.4 (802.11, capture length 65535)

The Airdecap-ng output was identified as:

pcap capture file, microsecond ts (little-endian) - version 2.4 (Ethernet, capture length 65535)

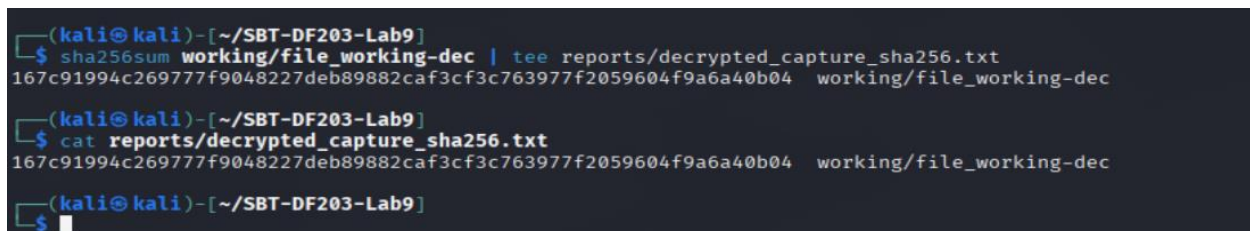
The decrypted derivative is approximately 13 MB, while the original decompressed capture is approximately 14 MB according to the directory listing.

The change from an 802.11 link-layer capture to an Ethernet link-layer capture is significant. Airdcap-ng has removed the WEP protection and converted the decrypted traffic into a form suitable for conventional network-layer analysis. This verification establishes the exact derivative that will be used for subsequent IP, endpoint, conversation, and object analysis. The original 802.11 evidence remains preserved as file_working, while file_working-dec is treated as a derived examination copy.

The distinction between the source and derivative is important for forensic integrity. The source capture is not overwritten or modified during the analysis, while the decrypted derivative can be examined independently and hashed to establish its own integrity baseline.

SHA-256 Hashing of the Decrypted Capture Derivative

After confirming the creation and file format of the Airdcap-ng decrypted capture, a SHA-256 cryptographic hash was calculated and recorded for the resulting forensic derivative. The hash was stored in reports/decrypted_capture_sha256.txt to provide a reproducible integrity reference for the decrypted evidence used in subsequent analysis.



```
(kali@kali)~[~/SBT-DF203-Lab9]
$ sha256sum working/file_working-dec | tee reports/decrypted_capture_sha256.txt
167c91994c269777f9048227deb89882caf3cf3c763977f2059604f9a6a40b04  working/file_working-dec

(kali@kali)~[~/SBT-DF203-Lab9]
$ cat reports/decrypted_capture_sha256.txt
167c91994c269777f9048227deb89882caf3cf3c763977f2059604f9a6a40b04  working/file_working-dec

(kali@kali)~[~/SBT-DF203-Lab9]
$
```

The calculated SHA-256 value is:

167c91994c269777f9048227deb89882caf3cf3c763977f2059604f9a6a40b04

The hash corresponds to working/file_working-dec

The recorded hash was subsequently displayed using cat, confirming that the value stored in the report file matched the hash generated by the SHA-256 calculation. The SHA-256 value establishes an integrity baseline for the decrypted derivative. Any

subsequent modification to file_working-dec, whether intentional or accidental, would result in a different SHA-256 value. This allows the derivative used for IP, endpoint, conversation, and object analysis to be independently verified against the recorded forensic baseline.

The original compressed evidence, decompressed working capture, and decrypted derivative have therefore been maintained as distinct evidence states, with cryptographic hashing used to document their integrity throughout the examination process.

The WEP portion is essentially complete:

Evidence stage	Finding
Original evidence	SHA-256: dd54144caef34f228bfb4b87a9101ca7173969376f7ed357bd06 8061d4f4b8d6
Capture	45,169 packets
Protected traffic	15,713 frames
Primary BSSID	00:26:66:55:97:d6
ESSID	cgnetwork
WEP IV reuse	Demonstrated; top IVs repeated up to 7 times
WEP data processed	15,477
Recovered WEP40 key	A4:3D:F6:F3:74
Aircrack validation	100% decrypted correctly
Airdecap validation	15,477 decrypted / 0 corrupted
Decrypted derivative	working/file_working-dec

Evidence stage	Finding
Decrypted derivative SHA-256	167c91994c269777f9048227deb89882caf3cf3c763977f20596 04f9a6a40b04

Protocol Hierarchy Analysis of the Decrypted Traffic

Following successful WEP decryption and integrity hashing, the decrypted capture was examined using TShark's protocol hierarchy statistics. This analysis was performed against `working/file_working-dec` to identify the network and application protocols that became visible after removal of the WEP encryption layer.

```
(kali@kali)-[~/SBT-DF203-Lab9]
$ tshark -r working/file_working-dec -q -z io,phs
Warning: program compiled against libxml 215 using older 214

Protocol Hierarchy Statistics
Filter:

frame                                frames:15477 bytes:12536845
eth                                  frames:15477 bytes:12536845
  arp                                frames:60 bytes:2610
  ip                                  frames:15361 bytes:12525585
    udp                              frames:159 bytes:31074
      dhcp                           frames:7 bytes:2890
      dns                            frames:81 bytes:10351
      mdns                           frames:40 bytes:6556
      sssdp                          frames:31 bytes:11277
      igmp                           frames:7 bytes:364
      tcp                            frames:15195 bytes:12494147
        tls                          frames:139 bytes:83267
          tls                         frames:14 bytes:12413
          http                        frames:278 bytes:183421
            data-text-lines           frames:29 bytes:22925
            image-gif                 frames:7 bytes:3461
            json                      frames:7 bytes:6424
            data-text-lines           frames:1 bytes:813
            media                     frames:28 bytes:21854
            png                       frames:29 bytes:21950
            image-jfif                frames:1 bytes:435
            xml                       frames:1 bytes:313
  ipv6                               frames:56 bytes:8650
    icmpv6                           frames:20 bytes:1628
    udp                              frames:36 bytes:7022
      mdns                           frames:36 bytes:7022
```

The decrypted capture contains 15,477 Ethernet frames, representing approximately 12.54 MB of captured data according to the protocol hierarchy statistics.

The protocol hierarchy shows the following major traffic categories:

- ARP: 60 frames.
- IPv4: 15,361 frames.
- IPv6: 56 frames.

- TCP: 15,195 frames.
- UDP: 159 frames.
- ICMPv6: 20 frames.
- DHCP: 7 frames.
- DNS: 81 frames.
- mDNS: 40 IPv4/UDP frames and an additional 36 IPv6/UDP frames.
- SSDP: 31 frames.
- TLS: 139 frames.
- HTTP: 278 frames.

The HTTP traffic is particularly significant because TShark identified several higher-level content types within the captured HTTP communications. These include:

- Data-text-lines: 29 frames.
- GIF images: 7 frames.
- JSON: 7 frames, including one JSON instance containing data-text-lines.
- Media: 28 frames.
- PNG images: 29 frames.
- JFIF/JPEG image: 1 frame.
- XML: 1 frame.

The TCP traffic dominates the decrypted capture, with 15,195 TCP frames, compared with 159 UDP frames. The presence of HTTP, TLS, DNS, DHCP, service-discovery protocols, and identifiable HTTP content demonstrates that the successful WEP decryption exposed higher-layer communications that were not directly available from the original encrypted 802.11 traffic. This protocol hierarchy establishes that the decrypted derivative contains meaningful network and application-layer evidence suitable for further forensic investigation. In

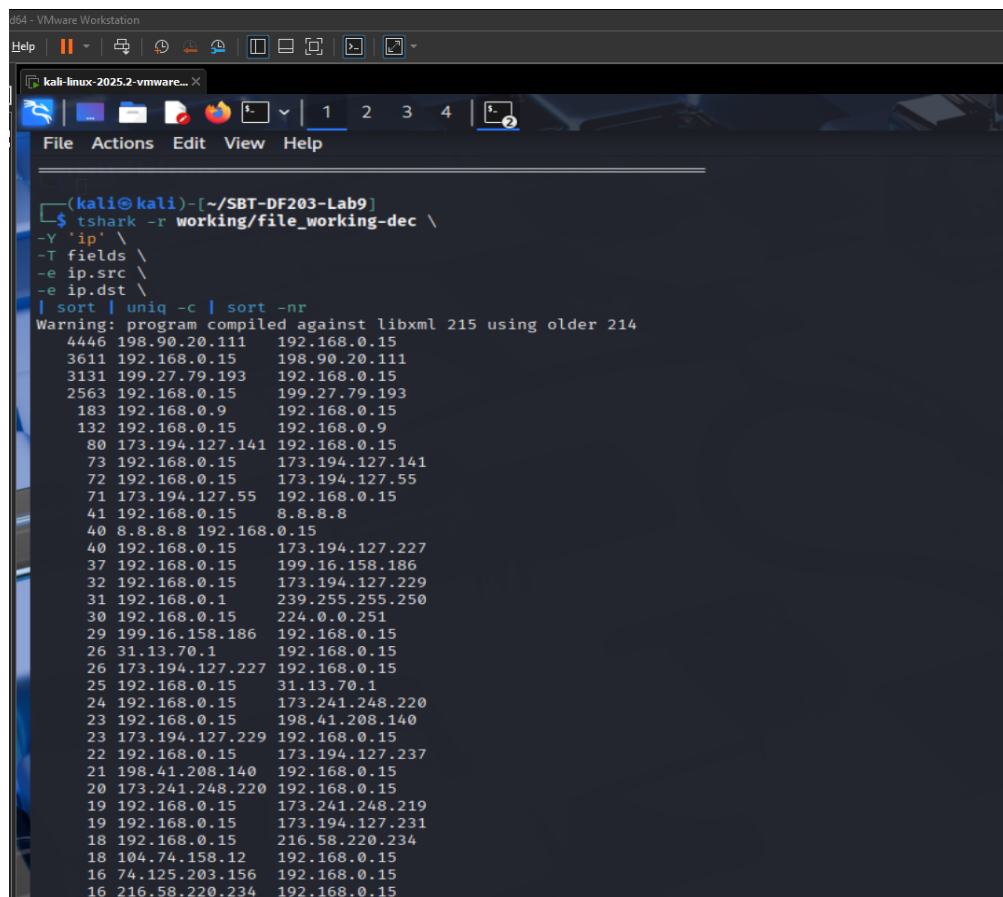
particular, the presence of HTTP and identifiable image, JSON, XML, and media content indicates that the capture contains potentially recoverable network objects.

The result also confirms that the Airdecap-ng derivative is suitable for the next stage of the investigation. Rather than continuing to analyse only wireless-layer information, the examination can now identify communicating IP addresses, MAC addresses, TCP/UDP conversations, HTTP transactions, and transferred objects.

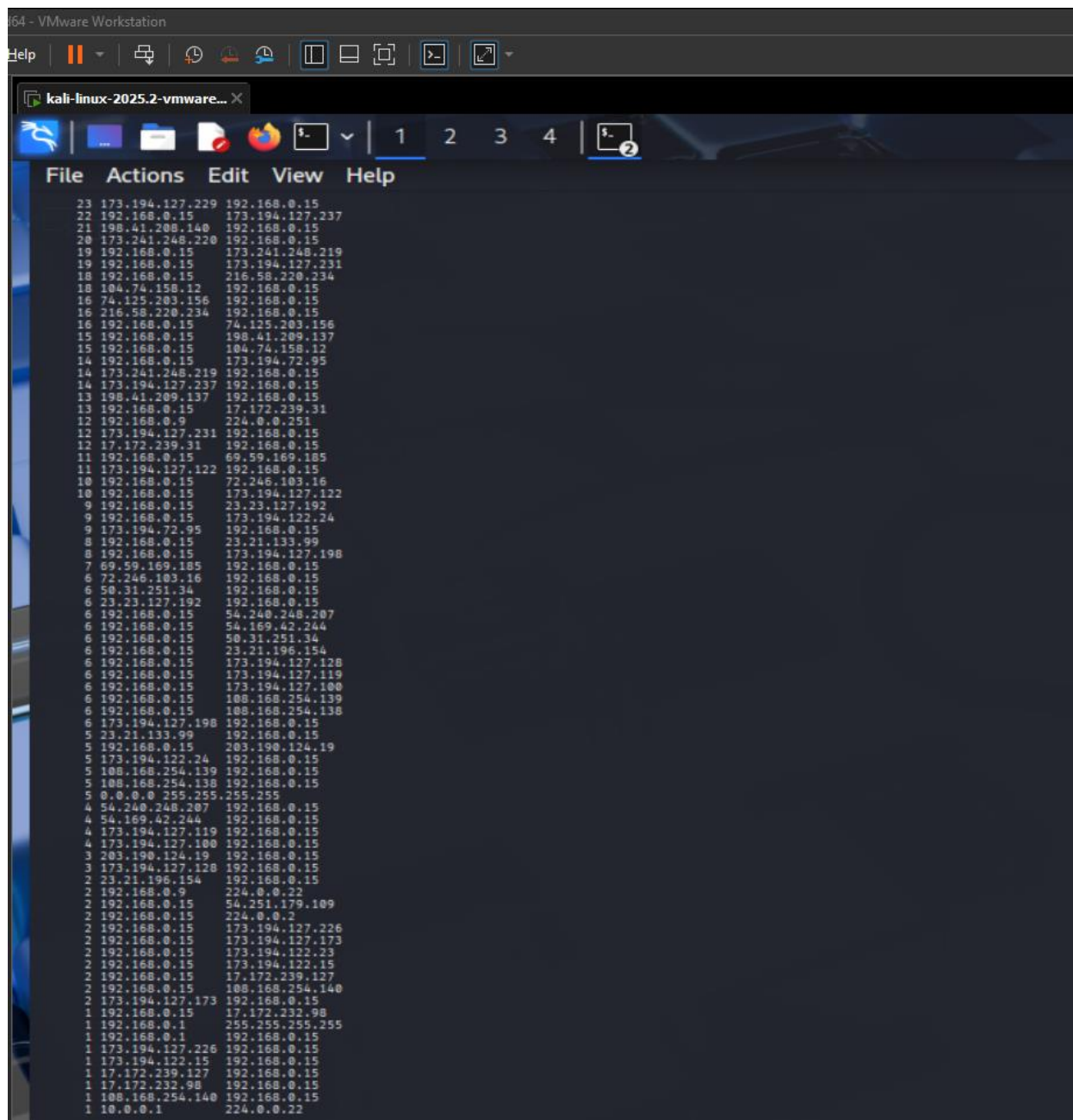
The libxml warning is a local TShark software-library compatibility warning and is not treated as a finding from the captured evidence.

IPv4 Endpoint and Communication Inventory

The decrypted capture was examined to identify the IPv4 endpoints communicating within and beyond the local network. TShark was used to extract the source and destination IPv4 addresses, after which identical communication pairs were grouped and counted.



```
(kali@kali)-[~/SBT-DF203-Lab9]
$ tshark -r working/file_working-dec \
-Y 'ip' \
-T fields \
-e ip.src \
-e ip.dst \
| sort | uniq -c | sort -nr
Warning: program compiled against libxml 215 using older 214
4446 198.90.20.111 192.168.0.15
3611 192.168.0.15 198.90.20.111
3131 199.27.79.193 192.168.0.15
2563 192.168.0.15 199.27.79.193
183 192.168.0.9 192.168.0.15
132 192.168.0.15 192.168.0.9
80 173.194.127.141 192.168.0.15
73 192.168.0.15 173.194.127.141
72 192.168.0.15 173.194.127.55
71 173.194.127.55 192.168.0.15
41 192.168.0.15 8.8.8.8
40 8.8.8.8 192.168.0.15
40 192.168.0.15 173.194.127.227
37 192.168.0.15 199.16.158.186
32 192.168.0.15 173.194.127.229
31 192.168.0.1 239.255.255.250
30 192.168.0.15 224.0.0.251
29 199.16.158.186 192.168.0.15
26 31.13.70.1 192.168.0.15
26 173.194.127.227 192.168.0.15
25 192.168.0.15 31.13.70.1
24 192.168.0.15 173.241.248.220
23 192.168.0.15 198.41.208.140
23 173.194.127.229 192.168.0.15
22 192.168.0.15 173.194.127.237
21 198.41.208.140 192.168.0.15
20 173.241.248.220 192.168.0.15
19 192.168.0.15 173.241.248.219
19 192.168.0.15 173.194.127.231
18 192.168.0.15 216.58.220.234
18 104.74.158.12 192.168.0.15
16 74.125.203.156 192.168.0.15
16 216.58.220.234 192.168.0.15
```



The analysis identifies 192.168.0.15 as the principal internal IPv4 endpoint in the capture. The largest observed communication pairs involving this host were:

Source	Destination	Frames
198.90.20.111	192.168.0.15	4,446

192.168.0.15	198.90.20.111	3,611
199.27.79.193	192.168.0.15	3,131
192.168.0.15	199.27.79.193	2,563
192.168.0.9	192.168.0.15	183
192.168.0.15	192.168.0.9	132

Additional communications were observed between 192.168.0.15 and several external addresses, including 173.194.127.141, 173.194.127.55, 8.8.8.8, 173.194.127.227, 199.16.158.186, and 31.13.70.1.

The capture also contains local and multicast traffic, including communications involving 192.168.0.1, 224.0.0.251, 239.255.255.250, and the broadcast address 255.255.255.255.

The presence of 192.168.0.15 across a large number of the highest-frequency conversations indicates that it is the principal IPv4 host represented in the decrypted traffic. However, the packet counts alone do not establish the identity of the device or the owner of any external IP address. The endpoint inventory establishes the principal IPv4 host and its communication relationships within the historical capture. The repeated bidirectional exchanges between 192.168.0.15 and external addresses provide the basis for subsequent investigation of application protocols, services, and potentially transferred objects.

The results also demonstrate the value of decrypting the original WEP traffic. The wireless capture initially exposed primarily 802.11 frame information, while the decrypted derivative now permits examination of IP-level communications and higher-layer protocols.

Ethernet MAC Address and Endpoint Inventory

The decrypted Ethernet capture was also examined at the link layer to identify the MAC addresses participating in the reconstructed network communications. Source and destination Ethernet addresses were extracted and grouped by frequency.

```
(kali@kali)~[~/SBT-DF203-Lab9]
$ tshark -r working/file_working-dec \
-T fields \
-e eth.src \
-e eth.dst \
| sort | uniq -c | sort -nr | head -30
Warning: program compiled against libxml 215 using older 214
 8145 00:26:66:55:97:d4      f0:f6:1c:68:96:7c
 6842 f0:f6:1c:68:96:7c      00:26:66:55:97:d4
  186 48:5b:39:2a:c2:7a      f0:f6:1c:68:96:7c
  135 f0:f6:1c:68:96:7c      48:5b:39:2a:c2:7a
   31 00:26:66:55:97:d4      01:00:5e:7f:ff:fa
   30 f0:f6:1c:68:96:7c      01:00:5e:00:00:fb
   27 f0:f6:1c:68:96:7c      33:33:00:00:00:fb
   26 f0:f6:1c:68:96:7c      ff:ff:ff:ff:ff:ff
   12 48:5b:39:2a:c2:7a      01:00:5e:00:00:fb
   9 48:5b:39:2a:c2:7a      33:33:00:00:00:fb
   6 f0:f6:1c:68:96:7c      33:33:00:00:00:02
   6 48:5b:39:2a:c2:7a      33:33:00:00:00:16
   4 f0:f6:1c:68:96:7c      33:33:00:00:00:16
   3 48:5b:39:2a:c2:7a      ff:ff:ff:ff:ff:ff
   3 48:5b:39:2a:c2:7a      33:33:00:00:00:02
   3 48:5b:39:2a:c2:7a      01:00:5e:00:00:16
   2 f0:f6:1c:68:96:7c      01:00:5e:00:00:02
   2 00:26:66:55:97:d4      ff:ff:ff:ff:ff:ff
   1 f0:f6:1c:68:96:7c      c8:3a:35:58:c4:ba
   1 f0:f6:1c:68:96:7c      08:10:77:92:7c:2f
   1 f0:f6:1c:68:96:7c      04:8d:38:48:e5:b4
   1 f0:f6:1c:68:96:7c      04:8d:38:48:a8:b5
   1 48:5b:39:2a:c2:7a      33:33:ff:2a:c2:7a

(kali@kali)~[~/SBT-DF203-Lab9]
$
```

The dominant communication relationship is between:

- 00:26:66:55:97:d4
- f0:f6:1c:68:96:7c

The most frequent observed Ethernet exchanges were:

Source MAC	Destination MAC	Frames
00:26:66:55:97:d4	f0:f6:1c:68:96:7c	8,145
f0:f6:1c:68:96:7c	00:26:66:55:97:d4	6,842
48:5b:39:2a:c2:7a	f0:f6:1c:68:96:7c	186
f0:f6:1c:68:96:7c	48:5b:39:2a:c2:7a	135

Additional traffic includes multicast, broadcast, and IPv6-related destination MAC addresses. Examples include ff:ff:ff:ff:ff:ff, 33:33:00:00:00:fb, 33:33:00:00:00:02, and 01:00:5e:00:00:fb.

The dominant bidirectional relationship accounts for 14,987 frames when the two directions are combined. This represents the principal Ethernet-level communication relationship observed in the decrypted capture. The MAC inventory provides a link-layer view that complements the IPv4 endpoint analysis. It establishes which physical/network interface addresses were involved in the highest-volume reconstructed communications and provides identifiers that can be correlated with the wireless-layer evidence.

The MAC address 00:26:66:55:97:d4 was already observed in the original protected 802.11 traffic, while f0:f6:1c:68:96:7c was also repeatedly observed as a wireless station. Their prominence in the decrypted Ethernet traffic therefore provides continuity between the original wireless evidence and the decrypted network-layer evidence.

ARP-Based Correlation of IP and MAC Addresses

To correlate the network-layer endpoints identified in the decrypted traffic with their corresponding Ethernet addresses, the ARP traffic within the decrypted capture was examined. ARP provides direct evidence linking an IPv4 address to a MAC address through request and reply exchanges.

```

d64 - VMware Workstation
Help
kali-linux-2025.2-vmware... X
File Actions Edit View Help
(kali@kali)-[~/SBT-DF203-Lab9]
$ tshark -r working/file_working-dec \
-Y 'arp' \
-T fields \
-e frame.number \
-e arp.opcode \
-e arp.src.proto_ipv4 \
-e arp.src.hw_mac \
-e arp.dst.proto_ipv4 \
-e arp.dst.hw_mac
Warning: program compiled against libxml 215 using older 214
1      1      192.168.0.1      00:26:66:55:97:d4      192.168.0.15      00:00:00:00:00:00
2      1      192.168.1.41      f0:f6:1c:68:96:7c      192.168.1.1      00:00:00:00:00:00
3      1      192.168.0.15      f0:f6:1c:68:96:7c      192.168.0.1      00:00:00:00:00:00
4      1      192.168.1.12      f0:f6:1c:68:96:7c      192.168.1.1      00:00:00:00:00:00
5      1      192.168.0.15      f0:f6:1c:68:96:7c      192.168.0.1      00:00:00:00:00:00
6      1      192.168.10.113      f0:f6:1c:68:96:7c      192.168.10.1      00:00:00:00:00:00
8      2      192.168.0.1      00:26:66:55:97:d4      192.168.0.15      f0:f6:1c:68:96:7c
12     1      192.168.0.15      f0:f6:1c:68:96:7c      192.168.0.15      00:00:00:00:00:00
14     1      192.168.0.15      f0:f6:1c:68:96:7c      192.168.0.15      00:00:00:00:00:00
15     1      192.168.0.15      f0:f6:1c:68:96:7c      169.254.255.255      00:00:00:00:00:00
16     1      192.168.0.15      f0:f6:1c:68:96:7c      169.254.255.255      00:00:00:00:00:00
17     1      192.168.0.15      f0:f6:1c:68:96:7c      192.168.0.1      00:00:00:00:00:00
18     1      192.168.0.15      f0:f6:1c:68:96:7c      192.168.0.1      00:00:00:00:00:00
19     2      192.168.0.1      00:26:66:55:97:d4      192.168.0.15      f0:f6:1c:68:96:7c
28     1      192.168.0.15      f0:f6:1c:68:96:7c      169.254.255.255      00:00:00:00:00:00
29     1      192.168.0.15      f0:f6:1c:68:96:7c      169.254.255.255      00:00:00:00:00:00
32     1      192.168.0.15      f0:f6:1c:68:96:7c      169.254.255.255      00:00:00:00:00:00
34     1      192.168.0.15      f0:f6:1c:68:96:7c      169.254.255.255      00:00:00:00:00:00
39     1      192.168.0.15      f0:f6:1c:68:96:7c      169.254.255.255      00:00:00:00:00:00
40     1      192.168.0.15      f0:f6:1c:68:96:7c      169.254.255.255      00:00:00:00:00:00
53     1      192.168.0.15      f0:f6:1c:68:96:7c      169.254.255.255      00:00:00:00:00:00
54     1      192.168.0.15      f0:f6:1c:68:96:7c      169.254.255.255      00:00:00:00:00:00
62     1      192.168.0.15      f0:f6:1c:68:96:7c      192.168.0.1      00:00:00:00:00:00
63     1      192.168.0.15      f0:f6:1c:68:96:7c      192.168.0.1      00:00:00:00:00:00
64     2      192.168.0.1      00:26:66:55:97:d4      192.168.0.15      f0:f6:1c:68:96:7c
67     1      192.168.0.15      f0:f6:1c:68:96:7c      192.168.0.15      00:00:00:00:00:00
68     1      192.168.0.15      f0:f6:1c:68:96:7c      192.168.0.15      00:00:00:00:00:00
71     1      192.168.0.15      f0:f6:1c:68:96:7c      192.168.0.1      00:00:00:00:00:00
72     1      192.168.0.15      f0:f6:1c:68:96:7c      192.168.0.1      00:00:00:00:00:00
73     2      192.168.0.1      00:26:66:55:97:d4      192.168.0.15      f0:f6:1c:68:96:7c
89     1      192.168.0.1      00:26:66:55:97:d4      192.168.0.15      00:00:00:00:00:00
90     2      192.168.0.15      f0:f6:1c:68:96:7c      192.168.0.1      00:26:66:55:97:d4
2948   1      192.168.0.1      00:26:66:55:97:d4      192.168.0.15      00:00:00:00:00:00
2950   2      192.168.0.15      f0:f6:1c:68:96:7c      192.168.0.1      00:26:66:55:97:d4
6472   1      192.168.0.15      f0:f6:1c:68:96:7c      192.168.0.1      00:00:00:00:00:00
6473   2      192.168.0.1      00:26:66:55:97:d4      192.168.0.15      f0:f6:1c:68:96:7c
6481   1      192.168.0.1      00:26:66:55:97:d4      192.168.0.15      00:00:00:00:00:00
6482   2      192.168.0.15      f0:f6:1c:68:96:7c      192.168.0.1      00:26:66:55:97:d4
12874   1      192.168.0.1      00:26:66:55:97:d4      192.168.0.15      00:00:00:00:00:00
12875   2      192.168.0.15      f0:f6:1c:68:96:7c      192.168.0.1      00:26:66:55:97:d4
15007   1      192.168.0.15      f0:f6:1c:68:96:7c      192.168.0.1      00:00:00:00:00:00
15008   2      192.168.0.1      00:26:66:55:97:d4      192.168.0.15      f0:f6:1c:68:96:7c

```

```

15007   1      192.168.0.15      f0:f6:1c:68:96:7c      192.168.0.1      00:00:00:00:00:00
15008   2      192.168.0.1      00:26:66:55:97:d4      192.168.0.15      f0:f6:1c:68:96:7c
15009   1      192.168.0.15      f0:f6:1c:68:96:7c      192.168.0.9      00:00:00:00:00:00
15010   1      192.168.0.15      f0:f6:1c:68:96:7c      192.168.0.9      00:00:00:00:00:00
15011   2      192.168.0.9      48:5b:39:2a:c2:7a      192.168.0.15      f0:f6:1c:68:96:7c
15016   1      192.168.0.9      48:5b:39:2a:c2:7a      192.168.0.15      00:00:00:00:00:00
15017   2      192.168.0.15      f0:f6:1c:68:96:7c      192.168.0.9      48:5b:39:2a:c2:7a
15018   1      192.168.0.1      00:26:66:55:97:d4      192.168.0.15      00:00:00:00:00:00
15019   2      192.168.0.15      f0:f6:1c:68:96:7c      192.168.0.1      00:26:66:55:97:d4
15024   1      192.168.0.9      48:5b:39:2a:c2:7a      192.168.0.1      00:00:00:00:00:00
15056   1      192.168.0.15      f0:f6:1c:68:96:7c      192.168.0.1      00:26:66:55:97:d4
15057   2      192.168.0.1      00:26:66:55:97:d4      192.168.0.15      f0:f6:1c:68:96:7c
15095   1      192.168.0.9      48:5b:39:2a:c2:7a      192.168.0.15      00:00:00:00:00:00
15096   2      192.168.0.15      f0:f6:1c:68:96:7c      192.168.0.9      48:5b:39:2a:c2:7a
15106   1      192.168.0.15      f0:f6:1c:68:96:7c      192.168.0.1      00:26:66:55:97:d4
15107   2      192.168.0.1      00:26:66:55:97:d4      192.168.0.15      f0:f6:1c:68:96:7c
15160   1      192.168.0.1      00:26:66:55:97:d4      192.168.0.15      00:00:00:00:00:00
15161   2      192.168.0.15      f0:f6:1c:68:96:7c      192.168.0.1      00:26:66:55:97:d4
15165   1      192.168.0.15      f0:f6:1c:68:96:7c      192.168.0.9      48:5b:39:2a:c2:7a
15166   2      192.168.0.9      48:5b:39:2a:c2:7a      192.168.0.15      f0:f6:1c:68:96:7c

```

The ARP output establishes several important address relationships.

The primary internal host 192.168.0.15 is consistently associated with MAC address f0:f6:1c:68:96:7c. For example, frames 3, 5, 17, 18 and subsequent ARP exchanges show 192.168.0.15 originating from f0:f6:1c:68:96:7c while resolving the gateway 192.168.0.1.

More importantly, frames 8, 19, 64, 73 and later exchanges contain ARP replies from:

192.168.0.1 → 00:26:66:55:97:d4

confirming the MAC address associated with the local gateway.

The capture therefore provides the following primary correlation:

IPv4 Address	MAC Address	Observed Role
192.168.0.15	f0:f6:1c:68:96:7c	Primary internal host
192.168.0.1	00:26:66:55:97:d4	Local gateway

The ARP traffic also shows additional local address relationships. For example, 192.168.0.9 is associated with 48:5b:39:2a:c2:7a. There are also ARP requests involving 192.168.1.41, 192.168.1.12, and 192.168.10.113, all originating from the MAC address f0:f6:1c:68:96:7c. These entries should be retained as observed capture evidence rather than interpreted as proof that those addresses belonged simultaneously to the same active network configuration.

The ARP evidence strengthens the endpoint inventory because it provides an explicit protocol-level relationship between IP and MAC addresses. In particular, it confirms that the high-volume IPv4 endpoint 192.168.0.15 corresponds to wireless station MAC f0:f6:1c:68:96:7c, while the local gateway 192.168.0.1 corresponds to 00:26:66:55:97:d4.

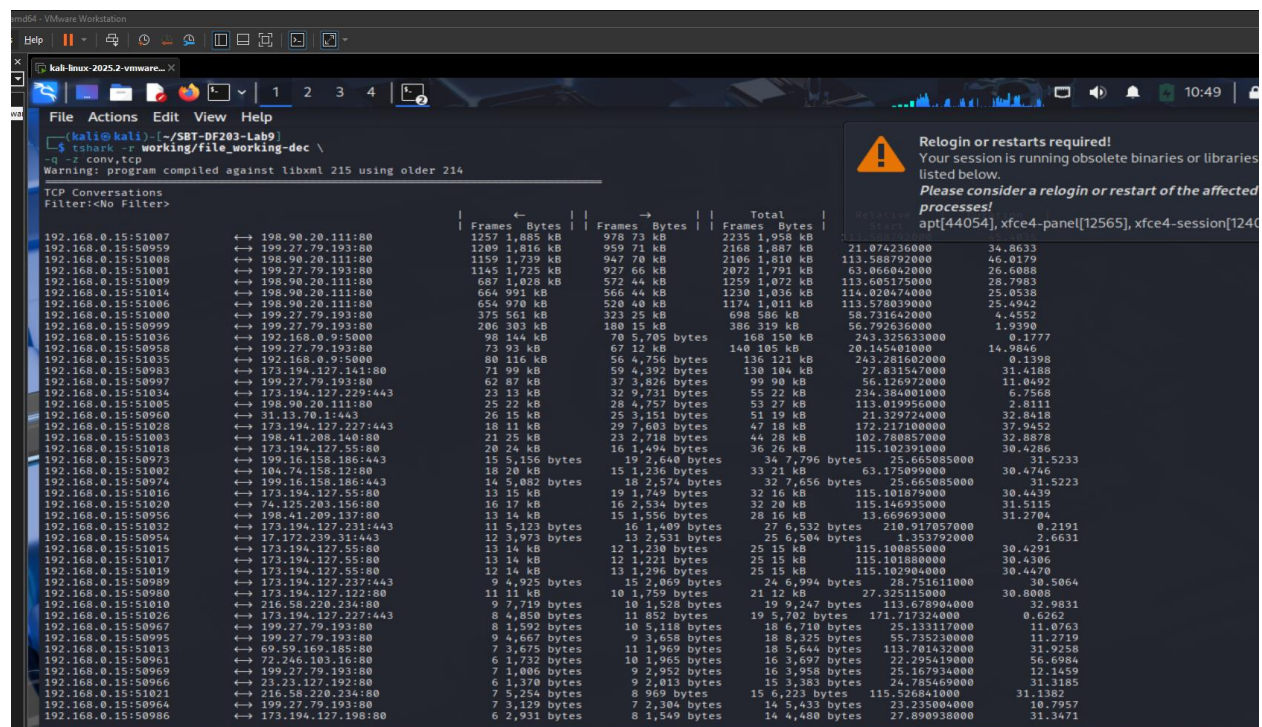
This correlation connects the decrypted Ethernet evidence back to the original 802.11 investigation, where 00:26:66:55:97:d4 was observed communicating through the WEP network identified by BSSID 00:26:66:55:97:d6.

The repeated ARP request/reply exchanges also demonstrate normal local address-resolution activity within the historical capture. No conclusion about malicious ARP activity is made from these exchanges alone.

TCP Conversation Inventory

The decrypted capture was analysed using TShark's TCP conversation statistics to identify the principal communicating endpoints, destination ports, traffic volumes, and directionality. The analysis shows that 192.168.0.15 was the dominant internal endpoint and was responsible for the majority of observed TCP communications.

The highest-volume conversations were HTTP sessions over TCP port 80. The largest recorded conversation was between 192.168.0.15:51007 and 198.90.20.111:80, comprising 2,235 frames and approximately 1.96 MB of traffic. Other significant sessions included communications between 192.168.0.15 and 199.27.79.193:80, with individual conversations exceeding 1.7 MB. These high-volume sessions are particularly relevant to the subsequent HTTP object-recovery stage because the decrypted protocol hierarchy identified HTTP traffic carrying images, JSON, XML, and other content.



```
File Actions Edit View Help
(kali@kali) (~/.SBT-DF202-Lab9)
tshark -r working/file_working-dec \
-q -z conv,tcp
Warning: program compiled against libxml 215 using older 214

TCP Conversations
Filter: <No Filter>

192.168.0.15:51007 <--> 198.90.20.111:80 1257 1,885 kB 978 73 kB 2235 1,958 kB
192.168.0.15:50959 <--> 199.27.79.193:80 1209 1,816 kB 950 71 kB 2168 1,887 kB
192.168.0.15:51008 <--> 198.90.20.111:80 1159 1,739 kB 947 70 kB 2106 1,810 kB
192.168.0.15:51001 <--> 199.27.79.193:80 1145 1,725 kB 927 66 kB 2072 1,791 kB
192.168.0.15:51009 <--> 198.90.20.111:80 687 1,028 kB 572 44 kB 1259 1,072 kB
192.168.0.15:51014 <--> 198.90.20.111:80 664 991 kB 566 44 kB 1230 1,036 kB
192.168.0.15:51006 <--> 198.90.20.111:80 654 970 kB 520 40 kB 1174 1,011 kB
192.168.0.15:51000 <--> 199.27.79.193:80 375 561 kB 323 25 kB 698 586 kB
192.168.0.15:50999 <--> 199.27.79.193:80 286 383 kB 180 15 kB 386 319 kB
192.168.0.15:51036 <--> 192.168.0.9:5000 98 144 kB 70 5,705 bytes 168 150 kB 243.325633000
192.168.0.15:50958 <--> 199.27.79.193:80 73 93 kB 67 12 kB 140 105 kB 20.145401000
192.168.0.15:51035 <--> 192.168.0.9:5000 80 116 kB 56 4,756 bytes 136 121 kB 243.251602000
192.168.0.15:50983 <--> 173.194.127.141:80 71 99 kB 59 4,392 bytes 130 104 kB 27.831547000
192.168.0.15:50997 <--> 199.27.79.193:80 62 87 kB 37 3,826 bytes 99 90 kB 56.126972000
192.168.0.15:51034 <--> 173.194.127.229:443 23 13 kB 32 9,731 bytes 55 22 kB 234.384001000
192.168.0.15:51005 <--> 198.90.20.111:80 25 22 kB 28 4,757 bytes 53 27 kB 113.019956000
192.168.0.15:50960 <--> 31.13.70.1:443 26 15 kB 25 3,151 bytes 51 19 kB 21.329724000
192.168.0.15:51028 <--> 173.194.127.227:443 18 11 kB 20 7,603 bytes 47 18 kB 172.217100000
192.168.0.15:51003 <--> 198.41.208.140:80 21 25 kB 23 7,718 bytes 44 28 kB 102.780857000
192.168.0.15:51018 <--> 173.194.127.55:80 20 24 kB 16 1,494 bytes 36 26 kB 115.102391000
192.168.0.15:50973 <--> 199.16.158.186:443 15 5,156 bytes 19 2,640 bytes 34 7,796 bytes 25.665085000
192.168.0.15:51002 <--> 104.74.158.12:80 18 20 kB 15 1,236 bytes 33 21 kB 63.175099000
192.168.0.15:50974 <--> 199.16.158.186:443 14 5,082 bytes 18 2,574 bytes 32 7,656 bytes 25.665085000
192.168.0.15:51016 <--> 173.194.127.55:80 13 15 kB 19 1,749 bytes 32 16 kB 115.101879000
192.168.0.15:51020 <--> 74.125.203.156:80 16 17 kB 16 7,534 bytes 32 20 kB 115.146935000
192.168.0.15:50956 <--> 198.41.209.137:80 13 14 kB 15 1,556 bytes 28 16 kB 13.669693000
192.168.0.15:51032 <--> 173.194.127.231:443 11 5,123 bytes 16 1,409 bytes 27 6,532 bytes 210.917057000
192.168.0.15:50954 <--> 17.172.239.31:443 12 2,973 bytes 12 2,531 bytes 25 6,504 bytes 1.353792000
192.168.0.15:51015 <--> 173.194.127.55:80 13 14 kB 12 1,230 bytes 25 15 kB 115.100855000
192.168.0.15:51017 <--> 173.194.127.55:80 13 14 kB 12 1,221 bytes 25 15 kB 115.101880000
192.168.0.15:51019 <--> 173.194.127.55:80 12 14 kB 13 1,296 bytes 25 15 kB 115.102904000
192.168.0.15:50989 <--> 173.194.127.237:443 9 4,925 bytes 15 2,069 bytes 24 6,994 bytes 38.751611000
192.168.0.15:50980 <--> 173.194.127.122:80 11 11 kB 10 1,759 bytes 21 12 kB 27.325115000
192.168.0.15:51010 <--> 216.58.220.234:80 9 7,719 bytes 10 1,528 bytes 19 9,247 bytes 113.678904000
192.168.0.15:51026 <--> 173.194.127.227:443 8 4,858 bytes 11 852 bytes 19 5,702 bytes 171.717224000
192.168.0.15:50967 <--> 199.27.79.193:80 8 1,592 bytes 10 5,118 bytes 18 6,710 bytes 25.133117000
192.168.0.15:50995 <--> 199.27.79.193:80 9 4,667 bytes 9 3,658 bytes 18 8,325 bytes 55.735230000
192.168.0.15:51013 <--> 69.59.169.183:80 7 2,675 bytes 11 1,969 bytes 18 5,644 bytes 113.701432000
192.168.0.15:50961 <--> 72.246.103.16:80 6 1,732 bytes 10 1,965 bytes 16 3,697 bytes 22.295419000
192.168.0.15:50969 <--> 199.27.79.193:80 7 1,806 bytes 9 2,952 bytes 16 3,958 bytes 25.167934000
192.168.0.15:50966 <--> 23.23.127.192:80 6 1,378 bytes 9 2,013 bytes 15 3,383 bytes 24.785469000
192.168.0.15:51021 <--> 216.58.220.234:80 7 5,254 bytes 8 969 bytes 15 6,223 bytes 115.526841000
192.168.0.15:50964 <--> 199.27.79.193:80 7 3,129 bytes 7 2,304 bytes 14 5,433 bytes 23.235004000
192.168.0.15:50986 <--> 173.194.127.198:80 6 2,931 bytes 8 1,549 bytes 14 4,480 bytes 27.890938000
```

Relogin or restarts required!
Your session is running obsolete binaries or libraries listed below.
Please consider a relogin or restart of the affected processes!
apt[44054], xfce4-panel[12565], xfce4-session[1240]

nd64 - VMware Workstation

Help | [Icons] | 10:49

kali-linux-2025.2-vmware... X

File Actions Edit View Help

192.168.0.15:50995	↔ 199.27.79.193:80	9 4,667 bytes	9 3,658 bytes	18 8,325 by
192.168.0.15:51013	↔ 69.59.169.185:80	7 3,675 bytes	11 1,969 bytes	18 5,644 by
192.168.0.15:50961	↔ 72.246.103.16:80	6 1,732 bytes	10 1,965 bytes	16 3,697 by
192.168.0.15:50969	↔ 199.27.79.193:80	7 1,006 bytes	9 2,952 bytes	16 3,958 by
192.168.0.15:50966	↔ 23.23.127.192:80	6 1,370 bytes	9 2,013 bytes	15 3,383 by
192.168.0.15:51021	↔ 216.58.220.234:80	7 5,254 bytes	8 969 bytes	15 6,223 by
192.168.0.15:50964	↔ 199.27.79.193:80	7 3,129 bytes	7 2,304 bytes	14 5,433 by
192.168.0.15:50986	↔ 173.194.127.198:80	6 2,931 bytes	8 1,549 bytes	14 4,480 by
192.168.0.15:50990	↔ 173.194.122.24:80	5 688 bytes	9 1,036 bytes	14 1,724 by
192.168.0.15:50996	↔ 199.27.79.193:80	7 2,500 bytes	7 1,989 bytes	14 4,489 by
192.168.0.15:50963	↔ 199.27.79.193:80	5 983 bytes	7 2,349 bytes	13 3,332 bytes
192.168.0.15:50970	↔ 199.27.79.193:80	6 932 bytes	7 2,670 bytes	13 3,602 bytes
192.168.0.15:50984	↔ 173.194.127.141:80	5 1,460 bytes	8 2,054 bytes	13 3,514 bytes
192.168.0.15:50988	↔ 199.27.79.193:80	6 939 bytes	7 2,014 bytes	13 2,953 bytes
192.168.0.15:51004	↔ 23.21.133.99:80	5 880 bytes	8 1,434 bytes	13 2,314 bytes
192.168.0.15:50968	↔ 199.27.79.193:80	5 866 bytes	7 2,813 bytes	12 3,679 bytes
192.168.0.15:50985	↔ 173.194.127.237:80	5 1,510 bytes	7 1,022 bytes	12 2,332 bytes
192.168.0.15:51011	↔ 173.194.72.95:80	5 1,373 bytes	7 977 bytes	12 2,350 bytes
192.168.0.15:51022	↔ 50.31.251.34:80	6 2,777 bytes	6 706 bytes	12 3,483 bytes
192.168.0.15:50972	↔ 173.241.248.219:80	5 856 bytes	6 928 bytes	11 1,784 bytes
192.168.0.15:50975	↔ 173.241.248.219:80	5 964 bytes	6 933 bytes	11 1,697 bytes
192.168.0.15:50976	↔ 173.241.248.220:80	5 983 bytes	6 1,041 bytes	11 2,024 bytes
192.168.0.15:50977	↔ 173.241.248.220:80	5 948 bytes	6 1,046 bytes	11 1,994 bytes
192.168.0.15:50978	↔ 173.241.248.219:80	4 987 bytes	7 1,065 bytes	11 2,052 bytes
192.168.0.15:50979	↔ 173.241.248.220:80	5 699 bytes	6 817 bytes	11 1,516 bytes
192.168.0.15:50981	↔ 173.241.248.220:80	5 492 bytes	6 822 bytes	11 1,314 bytes
192.168.0.15:51012	↔ 173.194.127.198:80	4 859 bytes	7 868 bytes	11 1,727 bytes
192.168.0.15:51023	↔ 108.168.254.139:80	5 1,690 bytes	6 965 bytes	11 2,655 bytes
192.168.0.15:51025	↔ 108.168.254.138:80	5 1,124 bytes	6 1,209 bytes	11 2,333 bytes
192.168.0.15:50965	↔ 54.240.248.207:80	4 441 bytes	6 776 bytes	10 1,217 bytes
192.168.0.15:50971	↔ 173.194.127.100:80	4 715 bytes	6 1,206 bytes	10 1,921 bytes
192.168.0.15:50982	↔ 173.194.127.141:80	4 1,056 bytes	6 918 bytes	10 1,974 bytes
192.168.0.15:50987	↔ 54.169.42.244:80	4 1,054 bytes	6 1,000 bytes	10 2,054 bytes
192.168.0.15:50994	↔ 173.194.127.119:80	4 670 bytes	6 864 bytes	10 1,534 bytes
192.168.0.15:50993	↔ 173.194.127.128:80	3 604 bytes	6 864 bytes	9 1,468 bytes
192.168.0.15:50957	↔ 23.21.196.154:80	2 140 bytes	6 835 bytes	8 975 bytes
192.168.0.15:50962	↔ 203.190.124.19:80	3 640 bytes	5 1,094 bytes	8 1,742 bytes
192.168.0.15:51037	↔ 192.168.0.9:5000	3 1,162 bytes	4 631 bytes	7 1,793 bytes
192.168.0.15:50955	↔ 17.172.239.127:443	1 74 bytes	2 132 bytes	3 206 bytes
192.168.0.15:50991	↔ 173.194.127.15:80	1 74 bytes	2 132 bytes	3 206 bytes
192.168.0.15:51024	↔ 108.168.254.140:80	1 66 bytes	2 132 bytes	3 198 bytes
192.168.0.15:51027	↔ 173.194.127.226:443	1 74 bytes	2 132 bytes	3 206 bytes
192.168.0.15:51031	↔ 173.194.127.221:443	1 74 bytes	2 132 bytes	3 206 bytes
17.172.232.98:443	↔ 192.168.0.15:50948	1 54 bytes	1 103 bytes	2 157 bytes
192.168.0.15:50988	↔ 54.251.179.109:80	0 0 bytes	2 132 bytes	2 132 bytes
192.168.0.15:50992	↔ 173.194.122.23:80	0 0 bytes	2 132 bytes	2 132 bytes
173.194.127.173:80	↔ 192.168.0.15:50887	1 54 bytes	1 66 bytes	2 120 bytes
173.194.127.173:80	↔ 192.168.0.15:50889	1 54 bytes	1 66 bytes	2 120 bytes
192.168.0.15:51029	↔ 192.168.0.9:5000	1 60 bytes	1 78 bytes	2 138 bytes
192.168.0.15:51033	↔ 192.168.0.9:5000	1 60 bytes	1 78 bytes	2 138 bytes
192.168.0.15:51030	↔ 173.194.127.231:443	0 0 bytes	1 78 bytes	2 138 bytes

Relogin or restarts required!
Your session is running obsolete binaries or libraries as listed below.
Please consider a relogin or restart of the affected processes!
apt[44054], xfce4-panel[12565], xfce4-session[12406]

HTTPS traffic was also observed over TCP port 443, including connections to several external addresses. Because these sessions are encrypted at the application layer, their payloads cannot be interpreted in the same manner as the observed HTTP traffic without the appropriate session keys.

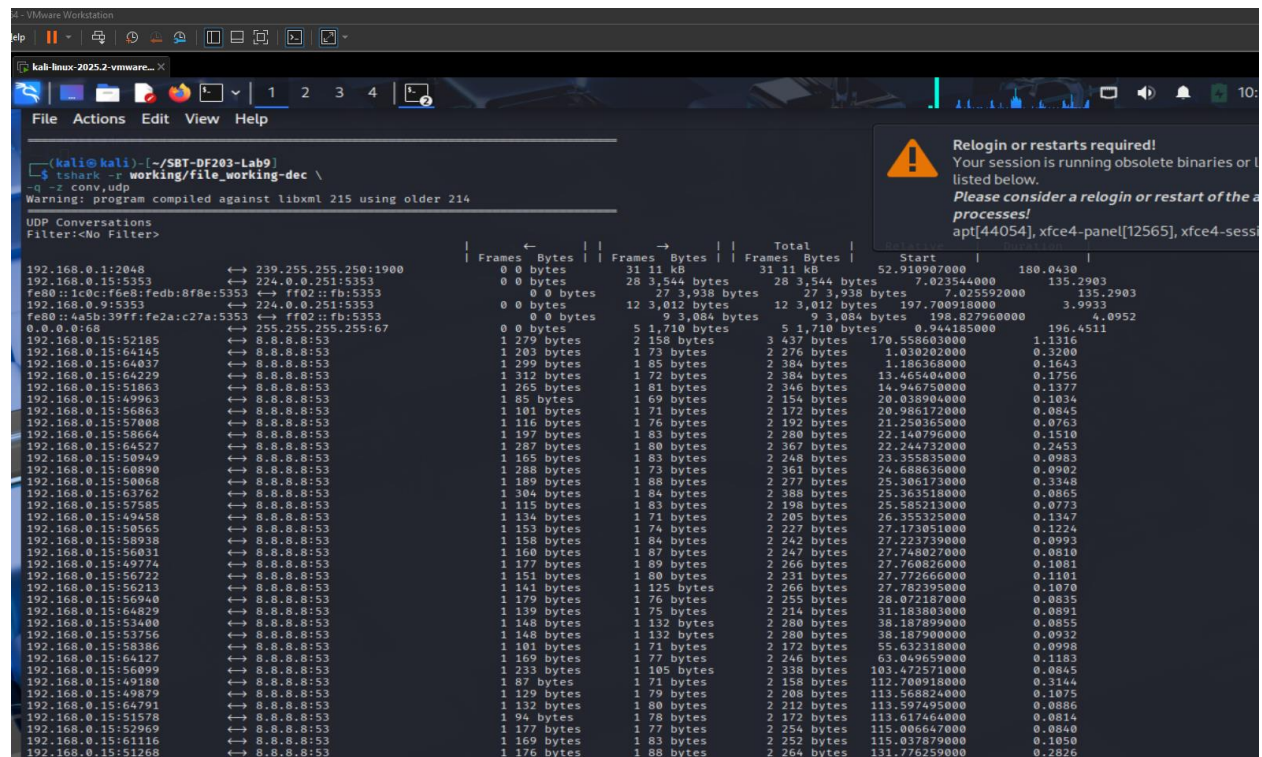
A local TCP service was also observed between 192.168.0.15 and 192.168.0.9 on destination port 5000. This provides evidence of communication between two internal endpoints within the decrypted capture.

The conversation inventory establishes the principal network relationships and identifies the high-volume HTTP sessions that warrant deeper examination for transferred objects and application-layer evidence.

192.168.0.15 is the principal internal endpoint in the decrypted capture, with substantial HTTP communication to 198.90.20.111 and 199.27.79.193, alongside smaller HTTPS and internal TCP sessions.

UDP Conversation Inventory

UDP conversation analysis identified several supporting network services within the decrypted capture. The traffic includes multicast service discovery, DNS resolution, DHCP activity, and IPv6 multicast DNS.



	Frames	Bytes	Frames	Bytes	Total	Start	
	←	→	←	→	Frames Bytes		
192.168.0.1:2048 ↔ 239.255.255.250:1900	0	0 bytes	31	11 kB	31 11 kB	52.910907000	180.0430
192.168.0.15:5353 ↔ 224.0.0.251:5353	0	0 bytes	28	3,544 bytes	28 3,544 bytes	7.023544000	135.2903
fe80::1c0c:f6e9:fedb:8f8e::5353 ↔ ff02::fb:5353	0	0 bytes	27	3,938 bytes	27 3,938 bytes	7.025592000	135.2903
192.168.0.9:5353 ↔ 224.0.0.251:5353	0	0 bytes	12	3,012 bytes	12 3,012 bytes	197.700910000	3.9933
fe80::a45b:39ff:fe2a:c27a::5353 ↔ ff02::fb:5353	0	0 bytes	9	3,084 bytes	9 3,084 bytes	198.827960000	4.0952
0.0.0.0:68 ↔ 255.255.255.255:67	0	0 bytes	5	1,710 bytes	5 1,710 bytes	0.941185000	136.4511
192.168.0.15:52185 ↔ 8.8.8.8:53	1	279 bytes	3	437 bytes	3 437 bytes	178.558603000	1.1316
192.168.0.15:64145 ↔ 8.8.8.8:53	1	203 bytes	1	73 bytes	2 276 bytes	1.030202000	0.3200
192.168.0.15:64037 ↔ 8.8.8.8:53	1	299 bytes	1	85 bytes	2 384 bytes	1.186368000	0.1643
192.168.0.15:64229 ↔ 8.8.8.8:53	1	312 bytes	1	72 bytes	2 384 bytes	13.465404000	0.1756
192.168.0.15:51863 ↔ 8.8.8.8:53	1	265 bytes	1	81 bytes	2 346 bytes	14.946750000	0.1377
192.168.0.15:49963 ↔ 8.8.8.8:53	1	185 bytes	1	69 bytes	2 254 bytes	20.038904000	0.1034
192.168.0.15:56863 ↔ 8.8.8.8:53	1	101 bytes	1	71 bytes	2 172 bytes	20.986172000	0.0845
192.168.0.15:57008 ↔ 8.8.8.8:53	1	116 bytes	1	76 bytes	2 192 bytes	21.250365000	0.0763
192.168.0.15:58664 ↔ 8.8.8.8:53	1	197 bytes	1	83 bytes	2 280 bytes	22.140796000	0.1510
192.168.0.15:64527 ↔ 8.8.8.8:53	1	287 bytes	1	80 bytes	2 367 bytes	22.244732000	0.2453
192.168.0.15:50949 ↔ 8.8.8.8:53	1	165 bytes	1	83 bytes	2 248 bytes	23.355835000	0.0983
192.168.0.15:60890 ↔ 8.8.8.8:53	1	288 bytes	1	73 bytes	2 361 bytes	24.688636000	0.0902
192.168.0.15:50068 ↔ 8.8.8.8:53	1	189 bytes	1	88 bytes	2 277 bytes	25.306173000	0.3348
192.168.0.15:63762 ↔ 8.8.8.8:53	1	304 bytes	1	84 bytes	2 388 bytes	25.363518000	0.0865
192.168.0.15:57585 ↔ 8.8.8.8:53	1	115 bytes	1	83 bytes	2 198 bytes	25.585213000	0.0773
192.168.0.15:49458 ↔ 8.8.8.8:53	1	134 bytes	1	71 bytes	2 205 bytes	26.355255000	0.1347
192.168.0.15:50565 ↔ 8.8.8.8:53	1	153 bytes	1	74 bytes	2 227 bytes	27.173051000	0.1224
192.168.0.15:58938 ↔ 8.8.8.8:53	1	158 bytes	1	84 bytes	2 242 bytes	27.223739000	0.0993
192.168.0.15:56031 ↔ 8.8.8.8:53	1	160 bytes	1	87 bytes	2 247 bytes	27.748027000	0.0810
192.168.0.15:49774 ↔ 8.8.8.8:53	1	177 bytes	1	89 bytes	2 266 bytes	27.760826000	0.1081
192.168.0.15:56722 ↔ 8.8.8.8:53	1	151 bytes	1	80 bytes	2 231 bytes	27.772666000	0.1101
192.168.0.15:56213 ↔ 8.8.8.8:53	1	141 bytes	1	125 bytes	2 266 bytes	27.782395000	0.1070
192.168.0.15:56940 ↔ 8.8.8.8:53	1	179 bytes	1	76 bytes	2 255 bytes	28.072187000	0.0835
192.168.0.15:64829 ↔ 8.8.8.8:53	1	139 bytes	1	75 bytes	2 214 bytes	31.183803000	0.0891
192.168.0.15:53400 ↔ 8.8.8.8:53	1	148 bytes	1	132 bytes	2 280 bytes	38.187899000	0.0855
192.168.0.15:53756 ↔ 8.8.8.8:53	1	148 bytes	1	132 bytes	2 280 bytes	38.187900000	0.0932
192.168.0.15:58385 ↔ 8.8.8.8:53	1	101 bytes	1	71 bytes	2 172 bytes	55.632318000	0.0996
192.168.0.15:64127 ↔ 8.8.8.8:53	1	169 bytes	1	77 bytes	2 246 bytes	63.049659000	0.1183
192.168.0.15:56099 ↔ 8.8.8.8:53	1	233 bytes	1	105 bytes	2 338 bytes	103.472571000	0.0845
192.168.0.15:49180 ↔ 8.8.8.8:53	1	87 bytes	1	71 bytes	2 158 bytes	112.700918000	0.3144
192.168.0.15:49879 ↔ 8.8.8.8:53	1	129 bytes	1	79 bytes	2 208 bytes	113.568824000	0.1075
192.168.0.15:64791 ↔ 8.8.8.8:53	1	132 bytes	1	80 bytes	2 212 bytes	113.597495000	0.0886
192.168.0.15:51578 ↔ 8.8.8.8:53	1	94 bytes	1	78 bytes	2 172 bytes	113.617464000	0.0814
192.168.0.15:52969 ↔ 8.8.8.8:53	1	177 bytes	1	77 bytes	2 254 bytes	115.006647000	0.0840
192.168.0.15:61116 ↔ 8.8.8.8:53	1	169 bytes	1	83 bytes	2 252 bytes	115.037879000	0.1050
192.168.0.15:51268 ↔ 8.8.8.8:53	1	176 bytes	1	88 bytes	2 264 bytes	131.776259000	0.2826
192.168.0.15:55815 ↔ 8.8.8.8:53	1	132 bytes	1	74 bytes	2 206 bytes	153.535556000	0.1423
192.168.0.15:61162 ↔ 8.8.8.8:53	1	137 bytes	1	73 bytes	2 210 bytes	153.841795000	0.0906
192.168.0.15:58835 ↔ 8.8.8.8:53	1	181 bytes	1	76 bytes	2 257 bytes	154.931397000	0.3748
192.168.0.15:52454 ↔ 8.8.8.8:53	1	150 bytes	1	75 bytes	2 225 bytes	181.314382000	0.3082
192.168.0.1:67 ↔ 255.255.255.255:68	0	0 bytes	1	590 bytes	1 590 bytes	0.954435000	0.0000
192.168.0.1:67 ↔ 192.168.0.15:68	0	0 bytes	1	590 bytes	1 590 bytes	2.095232000	0.0000

The largest UDP conversation was between 192.168.0.1:2048 and multicast address 239.255.255.250:1900, consisting of 31 frames and approximately 11 KB of traffic. This is consistent with SSDP/UPnP service-discovery traffic identified earlier in the protocol hierarchy.

Multicast DNS activity was also observed from both 192.168.0.15 and 192.168.0.9 using UDP port 5353. IPv6 mDNS traffic was additionally present through the corresponding link-local addresses.

The capture also contains DHCP traffic between 0.0.0.0:68 and 255.255.255.255:67, indicating DHCP activity during the captured network session. Numerous individual DNS conversations were observed between 192.168.0.15 and 8.8.8.8 over UDP port 53, showing repeated external DNS resolution activity.

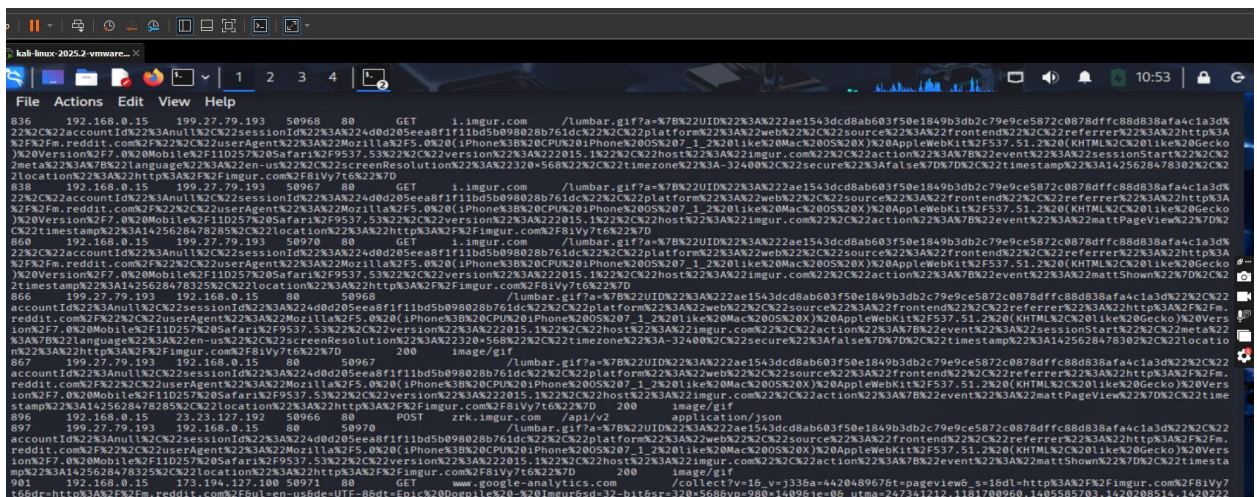
The UDP inventory provides additional context for the network environment and confirms the presence of DNS, DHCP, mDNS, and service-discovery traffic surrounding the primary endpoint.

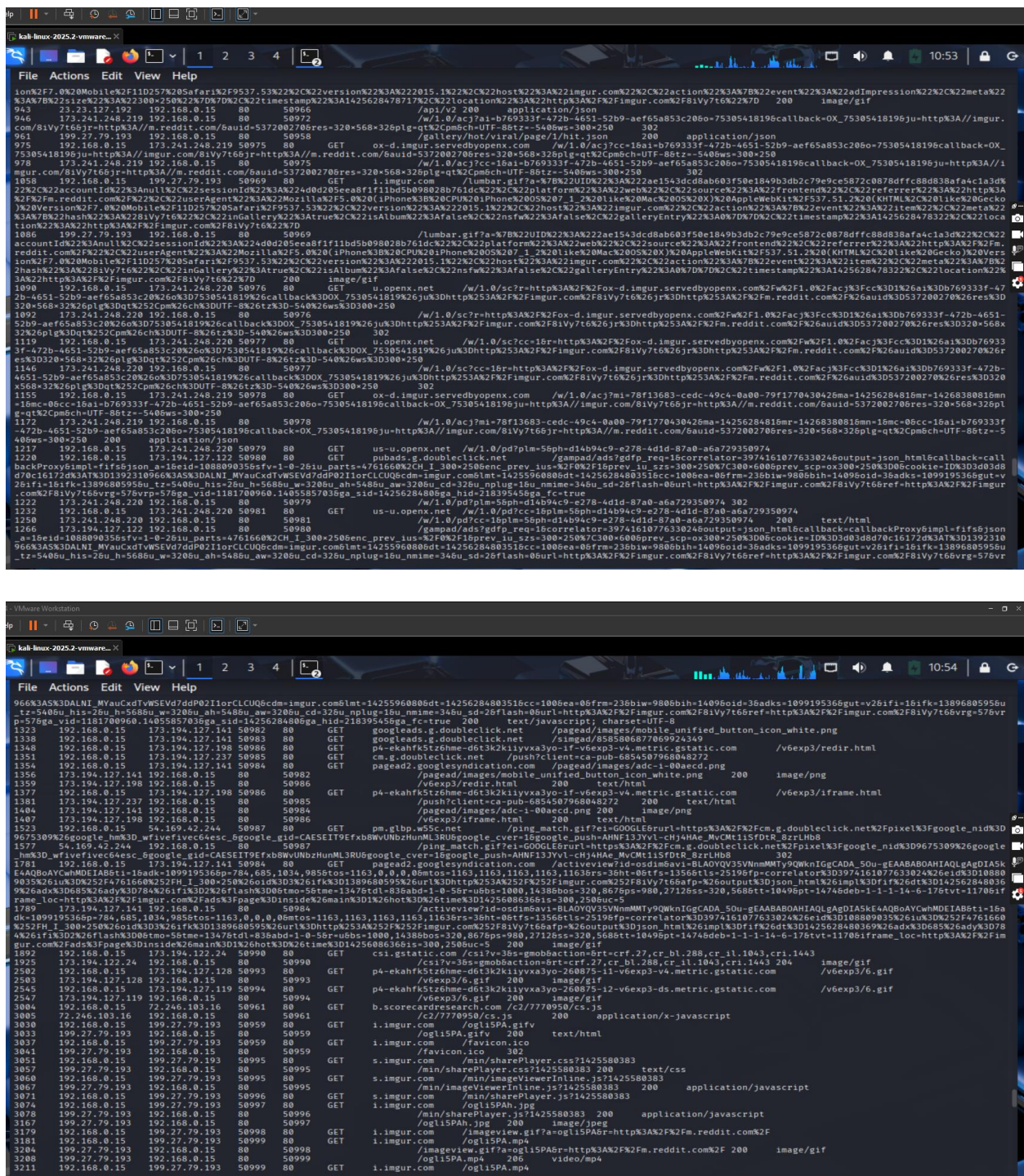
The decrypted evidence contains normal network-service traffic supporting address configuration, name resolution, and local service discovery, with 192.168.0.15 acting as the principal client generating DNS and mDNS activity.

HTTP Request and Response Inventory

Following successful WEP decryption, the Ethernet capture was examined at the HTTP application layer to identify web requests, responses, hosts, requested resources, response codes, and content types.

The traffic shows 192.168.0.15 communicating with multiple external HTTP services. Early activity includes requests to m.reddit.com and pixel.redditmedia.com, followed by extensive communication with imgur.com and i.imgur.com. The capture contains requests for the resource /8iVy7t6.jpg, JSON resources under the Imgur gallery paths, GIF resources, JavaScript files, CSS files, and additional image content.





[illegible]

Time	IP	Port	Status	Method	Host	Path	Content Type
12818	192.168.0.15	192.168.0.15	80	51009	GET	/wp-content/uploads/dynamik-gen/theme/images/header-contact.png	image/png
12824	192.168.0.15	192.168.0.15	80	51009	GET	/wp-content/uploads/dynamik-gen/theme/images/share-print.png	image/png
12835	192.168.0.15	192.168.0.15	80	51009	GET	/wp-content/uploads/dynamik-gen/theme/images/share-print.png	image/png
12841	192.168.0.15	192.168.0.15	80	51009	GET	/wp-content/uploads/dynamik-gen/theme/images/twitter.png	image/png
12907	192.168.0.15	192.168.0.15	80	51009	GET	/wp-content/uploads/dynamik-gen/theme/images/twitter.png	image/png
14915	192.168.0.15	192.168.0.15	80	51008	GET	/wp-content/uploads/2015/03/Figure-17.png	image/png
14145	192.168.0.15	192.168.0.15	80	51007	GET	/wp-content/uploads/2015/03/Figure-16.png	image/png
14164	192.168.0.15	192.168.0.15	80	51007	GET	/wp-content/uploads/2015/03/Figure-16.png	image/png
15177	192.168.0.15	192.168.0.15	80	51007	GET	/wp-content/uploads/2015/03/Figure-21.png	image/png
14412	192.168.0.15	192.168.0.15	80	51007	GET	/wp-content/uploads/2015/03/Figure-21.png	image/png
14414	192.168.0.15	192.168.0.15	80	51007	GET	/wp-content/uploads/2015/03/Figure-26.png	image/png
15194	192.168.0.15	192.168.0.15	80	51007	GET	/wp-content/uploads/2015/03/Figure-26.png	image/png
14709	192.168.0.15	192.168.0.15	80	51007	GET	/wp-content/uploads/dynamik-gen/theme/images/share-linkedin.png	image/png
14727	192.168.0.15	192.168.0.15	80	51007	GET	/wp-content/uploads/dynamik-gen/theme/images/share-linkedin.png	image/png
14730	192.168.0.15	192.168.0.15	80	51007	GET	/wp-content/uploads/dynamik-gen/theme/images/phone.png	image/png
14747	192.168.0.15	192.168.0.15	80	51007	GET	/wp-content/uploads/dynamik-gen/theme/images/phone.png	image/png
14749	192.168.0.15	192.168.0.15	80	51007	GET	/wp-content/uploads/dynamik-gen/theme/images/logo.png	image/png
14763	192.168.0.15	192.168.0.15	80	51007	GET	/wp-content/uploads/dynamik-gen/theme/images/logo.png	image/png
14814	192.168.0.15	192.168.0.15	80	51008	GET	/wp-content/uploads/2015/03/Figure-22.png	image/png
14863	192.168.0.15	192.168.0.15	80	51008	GET	/wp-content/uploads/dynamik-gen/theme/images/header-rss.png	image/png
14864	192.168.0.15	192.168.0.15	80	51008	GET	/wp-content/uploads/dynamik-gen/theme/images/share-facebook.png	image/png
14869	192.168.0.15	192.168.0.15	80	51008	GET	/wp-content/uploads/dynamik-gen/theme/images/share-facebook.png	image/png
14873	192.168.0.15	192.168.0.15	80	51008	GET	/wp-content/uploads/dynamik-gen/theme/images/heart.png	image/png
14875	192.168.0.15	192.168.0.15	80	51008	GET	/wp-content/uploads/dynamik-gen/theme/images/heart.png	image/png
14877	192.168.0.15	192.168.0.15	80	51007	GET	/wp-content/plugins/collapsible-widget-area/css/collapsible-widgets.css	text/css
14878	192.168.0.15	192.168.0.15	80	51007	GET	/wp-content/plugins/collapsible-widget-area/css/collapsible-widgets.css	text/css
14885	192.168.0.15	192.168.0.15	80	51022	GET	cdn.pardot.com /pd.js	application/javascript
14888	192.168.0.15	192.168.0.15	80	51022	GET	cdn.pardot.com /pd.js	application/javascript
14896	192.168.0.15	192.168.0.15	80	51023	GET	/analytics?only=true&visitor_id=736851146pi_opt_in=6campaign_id=15646account_id=473826title=Decoding%20Zeus%20disguised%20as%20a%20PhishMeBurl-http%3A%2F%2Fphishme.com%2Fdecoding-zeus-disguised-as-an-rtf-file%2Freferrer=http%3A%2F%2Freddit.com%2Ffrk2fnetsec	text/html; charset=utf-8
14900	192.168.0.15	192.168.0.15	80	51023	GET	/analytics?only=true&visitor_id=736851146pi_opt_in=6campaign_id=15646account_id=473826title=Decoding%20Zeus%20disguised%20as%20a%20PhishMeBurl-http%3A%2F%2Fphishme.com%2Fdecoding-zeus-disguised-as-an-rtf-file%2Freferrer=http%3A%2F%2Freddit.com%2Ffrk2fnetsec	text/html; charset=utf-8
14907	192.168.0.15	192.168.0.15	80	51025	GET	/analytics?only=true&visitor_id=736851146pi_opt_in=6campaign_id=15646account_id=473826title=Decoding%20Zeus%20disguised%20as%20a%20PhishMeBurl-http%3A%2F%2Fphishme.com%2Fdecoding-zeus-disguised-as-an-rtf-file%2Freferrer=http%3A%2F%2Freddit.com%2Ffrk2fnetsec	text/html; charset=utf-8
14909	192.168.0.15	192.168.0.15	80	51025	GET	/analytics?only=true&visitor_id=736851146pi_opt_in=6campaign_id=15646account_id=473826title=Decoding%20Zeus%20disguised%20as%20a%20PhishMeBurl-http%3A%2F%2Fphishme.com%2Fdecoding-zeus-disguised-as-an-rtf-file%2Freferrer=http%3A%2F%2Freddit.com%2Ffrk2fnetsec	text/html; charset=utf-8
15170	192.168.0.9	192.168.0.9	51035	5000	GET	/static/css/bootstrap-responsive.min.css	text/css; charset=utf-8
15172	192.168.0.9	192.168.0.9	51035	5000	GET	/static/css/bootstrap-responsive.min.css	text/css; charset=utf-8
15174	192.168.0.9	192.168.0.9	51035	5000	GET	/static/css/bootstrap-responsive.min.css	text/css; charset=utf-8
15181	192.168.0.9	192.168.0.9	51037	5000	GET	/static/css/bootstrap-responsive.min.css	text/css; charset=utf-8
15183	192.168.0.9	192.168.0.9	51037	5000	GET	/static/css/bootstrap-responsive.min.css	text/css; charset=utf-8
15206	192.168.0.9	192.168.0.9	51037	5000	GET	/static/css/bootstrap-responsive.min.css	text/css; charset=utf-8
15237	192.168.0.9	192.168.0.9	51037	5000	GET	/static/css/bootstrap-responsive.min.css	text/css; charset=utf-8
15313	192.168.0.9	192.168.0.9	51035	5000	GET	/static/css/bootstrap-responsive.min.css	text/css; charset=utf-8
15329	192.168.0.9	192.168.0.9	51035	5000	GET	/static/css/bootstrap-responsive.min.css	text/css; charset=utf-8
15330	192.168.0.9	192.168.0.9	51035	5000	GET	/static/css/bootstrap-responsive.min.css	text/css; charset=utf-8
15333	192.168.0.9	192.168.0.9	51035	5000	GET	/static/css/bootstrap-responsive.min.css	text/css; charset=utf-8
15367	192.168.0.9	192.168.0.9	51036	5000	GET	/static/js/shai.js	application/javascript
15448	192.168.0.9	192.168.0.9	51036	5000	GET	/static/js/shai.js	application/javascript

Several HTTP responses were successfully returned with status code 200, while other transactions returned 204 and 302 responses. The presence of HTTP content types such as image/gif, image/png, image/jpeg, application/json, text/html, and JavaScript confirms that the decrypted traffic contains recoverable application-layer objects rather than only connection metadata.

The HTTP inventory also identifies a request to phishme.com near the end of the displayed output. This request requires additional targeted examination because its associated traffic may contain a significant artefact within the historical training scenario.

Forensic significance: The HTTP inventory establishes that WEP decryption exposed application-layer web activity and identifies specific hosts, URIs, response codes, and content types that can be correlated with recovered objects.

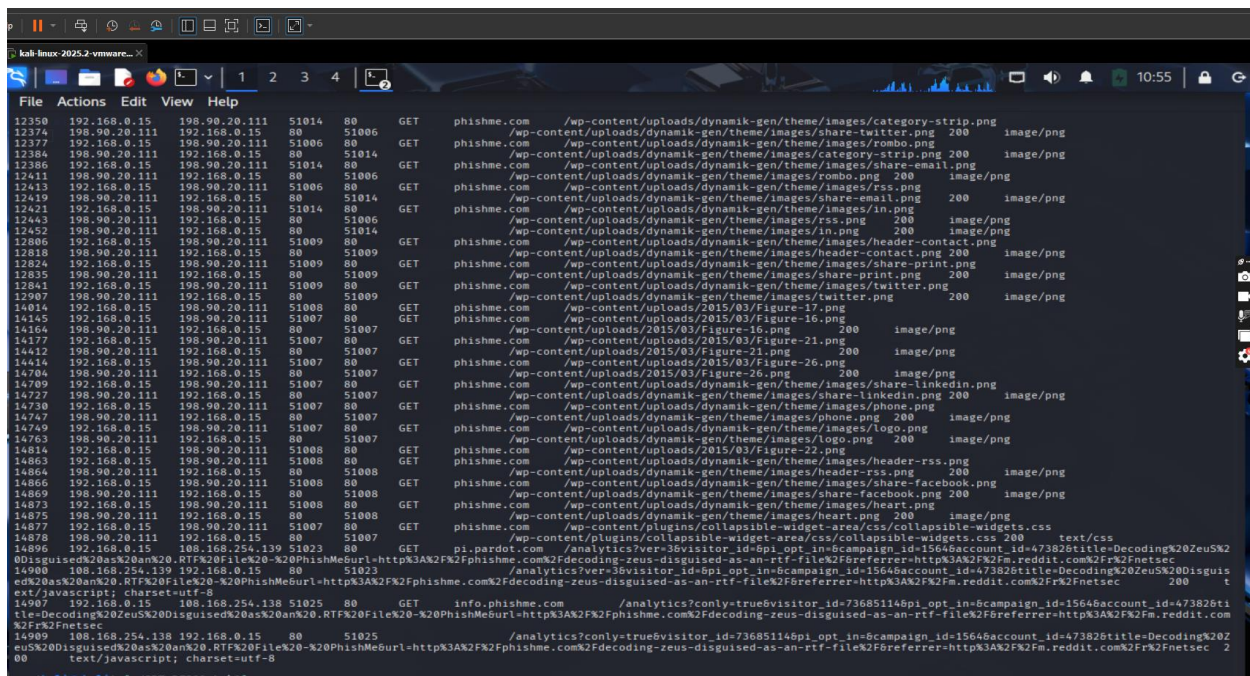
Finding: The decrypted capture contains extensive HTTP activity generated by 192.168.0.15, including image, JSON, JavaScript, CSS, and HTML resources. The phishme.com traffic is a specific item requiring further investigation before conclusions are drawn about its significance.

Targeted HTTP Analysis of PhishMe Web Activity

The targeted HTTP analysis identified web activity between the internal endpoint 192.168.0.15 and the remote server 198.90.20.111 over TCP port 80. The first identified request accessed the PhishMe page `/decoding-zeus-disguised-as-an-rtf-file/`. Subsequent requests retrieved supporting resources associated with the page, including CSS files, JavaScript files and numerous PNG images stored under the site's WordPress content directories.

```
kali@kali:~/SBT-DF203-Lab9$ tshark -r working/file_working-dec \
-Y 'http.host contains "phishme.com" || http.request.full_uri contains "phishme.com" \
-I fields \
-e frame.number \
-e ip.src \
-e ip.dst \
-e tcp.srcport \
-e tcp.dstport \
-e http.request.method \
-e http.host \
-e http.request.uri \
-e http.response.code \
-e http.content_type \
Warning: program compiled against libxml 215 using older 214
6549 192.168.0.15 198.90.20.111 51005 80 GET phishme.com /decoding-zeus-disguised-as-an-rtf-file/
6567 192.168.0.15 198.90.20.111 51005 80 GET phishme.com /wp-content/plugins/nivo-slider/scripts/nivo-slider.css 200 text/css
6568 198.90.20.111 192.168.0.15 80 51005
6593 192.168.0.15 198.90.20.111 51007 80 GET phishme.com /wp-content/uploads/dynamik-gen/theme/dynamik-min.css
6598 192.168.0.15 198.90.20.111 51006 80 GET phishme.com /wp-content/plugins/portfolio-slideshow-pro/css/portfolio-slideshow.min.css
6600 192.168.0.15 198.90.20.111 51008 80 GET phishme.com /wp-content/plugins/sitemap/css/page-list.css
6602 192.168.0.15 198.90.20.111 51009 80 GET phishme.com /wp-content/plugins/ultimate-posts-widget/css/upw-theme-standard.min.css
6631 198.90.20.111 192.168.0.15 80 51007 /wp-content/uploads/dynamik-gen/theme/dynamik-min.css 200 text/css
6642 198.90.20.111 192.168.0.15 80 51006 /wp-content/plugins/portfolio-slideshow-pro/css/portfolio-slideshow.min.css 200 text/css
6645 198.90.20.111 192.168.0.15 80 51009 /wp-content/plugins/ultimate-posts-widget/css/upw-theme-standard.min.css 200 text/css
6648 198.90.20.111 192.168.0.15 80 51008 /wp-content/plugins/sitemap/css/page-list.css 200 text/css
6650 192.168.0.15 198.90.20.111 51008 80 GET phishme.com /wp-includes/js/jquery/jquery.js
6651 192.168.0.15 198.90.20.111 51005 80 GET phishme.com /wp-includes/js/jquery/jquery-migrate.min.js
6652 192.168.0.15 198.90.20.111 51006 80 GET phishme.com /wp-content/plugins/portfolio-slideshow-pro/js/portfolio-slideshow.min.js
6653 192.168.0.15 198.90.20.111 51007 80 GET phishme.com /wp-includes/js/comment-reply.min.js
6654 192.168.0.15 198.90.20.111 51009 80 GET phishme.com /wp-content/themes/genesis/lib/js/menu/superfish.min.js
6655 192.168.0.15 198.90.20.111 51005 80 GET phishme.com /wp-includes/js/jquery/ui/core.min.js
6656 192.168.0.15 198.90.20.111 51005 80 GET phishme.com /wp-includes/js/jquery/ui/widget.min.js
6657 192.168.0.15 198.90.20.111 51005 80 GET phishme.com /wp-content/plugins/collapsible-widget-area/scripts/collapsible-widgets.js
6659 198.90.20.111 192.168.0.15 80 51007 /wp-includes/js/comment-reply.min.js 200 application/javascript
6661 192.168.0.15 198.90.20.111 51007 80 GET phishme.com /wp-includes/js/jquery/ui/tabs.min.js
6663 198.90.20.111 192.168.0.15 80 51009 /wp-content/themes/genesis/lib/js/menu/superfish.min.js 200 application/javascript
6665 192.168.0.15 198.90.20.111 51009 80 GET phishme.com /wp-content/plugins/collapsible-widget-area/scripts/jquery.cookie.js
6674 198.90.20.111 192.168.0.15 80 51005 /wp-content/plugins/collapsible-widget-area/scripts/collapsible-widgets.js 200 application/javas
cript
6679 198.90.20.111 192.168.0.15 80 51006 /wp-content/plugins/portfolio-slideshow-pro/js/portfolio-slideshow.min.js 200 application/javas
cript
6695 192.168.0.15 198.90.20.111 51014 80 GET phishme.com /wp-content/themes/genesis/lib/js/menu/superfish.args.min.js
6702 192.168.0.15 198.90.20.111 51006 80 GET phishme.com /wp-includes/js/jquery/ui/accordion.min.js
6712 198.90.20.111 192.168.0.15 80 51007 /wp-includes/js/jquery/ui/tabs.min.js 200 application/javascript
6715 198.90.20.111 192.168.0.15 80 51009 /wp-content/plugins/collapsible-widget-area/scripts/jquery.cookie.js 200 application/javascript
6719 198.90.20.111 192.168.0.15 80 51014 /wp-content/themes/genesis/lib/js/menu/superfish.args.min.js 200 application/javascript
6729 198.90.20.111 192.168.0.15 80 51006 /wp-includes/js/jquery/ui/accordion.min.js 200 application/javascript
6752 192.168.0.15 198.90.20.111 51008 80 GET phishme.com /wp-content/themes/dynamik-gen/lib/js/dynamik-responsive.js
6753 192.168.0.15 69.59.169.185 51013 80 GET t4.trackalyzer.com /trackalyze.asp?r=http%3A//m.reddit.com/r/netsec&p=http%3A//phishme.com/decoding-zeus-dis
guised-as-an-rtf-file/G1=19612611nocookies=undefined
6754 192.168.0.15 198.90.20.111 51005 80 GET phishme.com /wp-admin/admin-ajax.php?action=wordfence_logHuman&hid=305E48E6D496430879857CB91BBAF7146r=0.19585
```

The HTTP responses provide evidence that many of these resources were successfully transferred, with repeated HTTP 200 responses and content types including text/css, application/javascript and image/png. The capture also records supporting analytics activity involving external services, including Trackalyzer, Google Analytics/DoubleClick and PhishMe-related analytics endpoints. A Wordfence AJAX request returned HTTP 500, representing a server-side response to that individual request.



From a forensic perspective, this traffic demonstrates that the decrypted capture contains a complete or substantially complete historical HTTP browsing session involving the PhishMe website. The requested page and its associated resources provide identifiable application-layer objects that can be examined and exported from the decrypted capture.

The presence of the phrase "Decoding Zeus Disguised as an .RTF File" in the requested URL identifies the subject of the historical web page, but the HTTP request inventory alone does not establish that a Zeus executable or other malware sample was downloaded to the internal endpoint. Any such conclusion would require direct evidence of a transferred executable or other corresponding object in the capture.

What we can establish is that the main PhishMe page and its resources are spread across several TCP streams:

TCP stream	Local port	Remote	Observed activity
54	51005	198.90.20.111:80	Main page and several JavaScript/AJAX requests

TCP stream	Local port	Remote	Observed activity
55	51006	198.90.20.111:80	CSS, JavaScript and multiple PNG objects
56	51007	198.90.20.111:80	JavaScript and multiple PNG objects
57	51008	198.90.20.111:80	JavaScript and multiple PNG objects
58	51009	198.90.20.111:80	JavaScript, CSS and PNG objects
63	51014	198.90.20.111:80	Additional page resources and PNG objects
62	51013	69.59.169.185:80	Trackalyzer referral/analytics
69	51020	74.125.203.156:80	Google Analytics
72	51023	108.168.254.139:80	Pardot analytics
74	51025	108.168.254.138:80	PhishMe-related analytics

The most significant streams for actual page content/object recovery are therefore 54, 55, 56, 57, 58 and 63. Streams 62, 69, 72 and 74 are primarily analytics-related.

```
d64 - VMware Workstation
Help
kali-linux-2025.2-vmware...
File Actions Edit View Help
(kali@kali)-[~/SBT-DF203-Lab9]
$ tshark -r working/file_working-dec \
-Y 'http.host contains "phishme.com" || http.request.full_uri contains "phishme.com"' \
-T fields \
-e frame.number \
-e tcp.stream \
-e ip.src \
-e ip.dst \
-e tcp.srcport \
-e tcp.dstport
Warning: program compiled against libxml 215 using older 214
6549 54 192.168.0.15 198.90.20.111 51005 80
6567 54 192.168.0.15 198.90.20.111 51005 80
6588 54 198.90.20.111 192.168.0.15 80 51005
6593 56 192.168.0.15 198.90.20.111 51007 80
6598 55 192.168.0.15 198.90.20.111 51006 80
6600 57 192.168.0.15 198.90.20.111 51008 80
6602 58 192.168.0.15 198.90.20.111 51009 80
6631 56 198.90.20.111 192.168.0.15 80 51007
6642 55 198.90.20.111 192.168.0.15 80 51006
6645 58 198.90.20.111 192.168.0.15 80 51009
6648 57 198.90.20.111 192.168.0.15 80 51008
6650 57 192.168.0.15 198.90.20.111 51008 80
6651 54 192.168.0.15 198.90.20.111 51005 80
6652 55 192.168.0.15 198.90.20.111 51006 80
6653 56 192.168.0.15 198.90.20.111 51007 80
6654 58 192.168.0.15 198.90.20.111 51009 80
6655 54 192.168.0.15 198.90.20.111 51005 80
6656 54 192.168.0.15 198.90.20.111 51005 80
6657 54 192.168.0.15 198.90.20.111 51005 80
6659 56 198.90.20.111 192.168.0.15 80 51007
6661 56 192.168.0.15 198.90.20.111 51007 80
6663 58 198.90.20.111 192.168.0.15 80 51009
6665 58 192.168.0.15 198.90.20.111 51009 80
6674 54 198.90.20.111 192.168.0.15 80 51005
6679 55 198.90.20.111 192.168.0.15 80 51006
6695 63 192.168.0.15 198.90.20.111 51014 80
6702 55 192.168.0.15 198.90.20.111 51006 80
6712 56 198.90.20.111 192.168.0.15 80 51007
6715 58 198.90.20.111 192.168.0.15 80 51009
6719 63 198.90.20.111 192.168.0.15 80 51014
6729 55 198.90.20.111 192.168.0.15 80 51006
6752 57 192.168.0.15 198.90.20.111 51008 80
6753 62 192.168.0.15 69.59.169.185 51013 80
6754 54 192.168.0.15 198.90.20.111 51005 80
6755 57 198.90.20.111 192.168.0.15 80 51008
6757 62 69.59.169.185 192.168.0.15 80 51013
6927 54 198.90.20.111 192.168.0.15 80 51005
6932 55 192.168.0.15 198.90.20.111 51006 80
6933 56 192.168.0.15 198.90.20.111 51007 80
6934 57 192.168.0.15 198.90.20.111 51008 80
6935 58 192.168.0.15 198.90.20.111 51009 80
6936 63 192.168.0.15 198.90.20.111 51014 80
```

VMware Workstation

kali-linux-2025.2-vmware...

File	Actions	Edit	View	Help
9801	55	192.168.0.15	198.90.20.111	51006 80
9915	55	198.90.20.111	192.168.0.15	80 51006
9924	55	192.168.0.15	198.90.20.111	51006 80
10938	58	192.168.0.15	198.90.20.111	51009 80
11127	56	192.168.0.15	198.90.20.111	51007 80
11323	55	192.168.0.15	198.90.20.111	51006 80
11578	63	192.168.0.15	198.90.20.111	51014 80
12022	58	192.168.0.15	198.90.20.111	51009 80
12319	58	192.168.0.15	198.90.20.111	51009 80
12329	55	192.168.0.15	198.90.20.111	51006 80
12350	63	192.168.0.15	198.90.20.111	51014 80
12374	55	198.90.20.111	192.168.0.15	80 51006
12377	55	192.168.0.15	198.90.20.111	51006 80
12384	63	198.90.20.111	192.168.0.15	80 51014
12386	63	192.168.0.15	198.90.20.111	51014 80
12411	55	198.90.20.111	192.168.0.15	80 51006
12413	55	192.168.0.15	198.90.20.111	51006 80
12419	63	198.90.20.111	192.168.0.15	80 51014
12421	63	192.168.0.15	198.90.20.111	51014 80
12443	55	198.90.20.111	192.168.0.15	80 51006
12452	63	198.90.20.111	192.168.0.15	80 51014
12806	58	192.168.0.15	198.90.20.111	51009 80
12818	58	198.90.20.111	192.168.0.15	80 51009
12824	58	192.168.0.15	198.90.20.111	51009 80
12835	58	198.90.20.111	192.168.0.15	80 51009
12841	58	192.168.0.15	198.90.20.111	51009 80
12907	58	198.90.20.111	192.168.0.15	80 51009
14014	57	192.168.0.15	198.90.20.111	51008 80
14145	56	192.168.0.15	198.90.20.111	51007 80
14164	56	198.90.20.111	192.168.0.15	80 51007
14177	56	192.168.0.15	198.90.20.111	51007 80
14412	56	198.90.20.111	192.168.0.15	80 51007
14414	56	192.168.0.15	198.90.20.111	51007 80
14704	56	198.90.20.111	192.168.0.15	80 51007
14709	56	192.168.0.15	198.90.20.111	51007 80
14727	56	198.90.20.111	192.168.0.15	80 51007
14730	56	192.168.0.15	198.90.20.111	51007 80
14747	56	198.90.20.111	192.168.0.15	80 51007
14749	56	192.168.0.15	198.90.20.111	51007 80
14763	56	198.90.20.111	192.168.0.15	80 51007
14814	57	192.168.0.15	198.90.20.111	51008 80
14863	57	192.168.0.15	198.90.20.111	51008 80
14864	57	198.90.20.111	192.168.0.15	80 51008
14866	57	192.168.0.15	198.90.20.111	51008 80
14869	57	198.90.20.111	192.168.0.15	80 51008
14873	57	192.168.0.15	198.90.20.111	51008 80
14875	57	198.90.20.111	192.168.0.15	80 51008
14877	56	192.168.0.15	198.90.20.111	51007 80
14878	56	198.90.20.111	192.168.0.15	80 51007
14896	72	192.168.0.15	108.168.254.139	51023 80
14900	72	108.168.254.139	192.168.0.15	80 51023
14907	74	192.168.0.15	108.168.254.138	51025 80
14909	74	108.168.254.138	192.168.0.15	80 51025

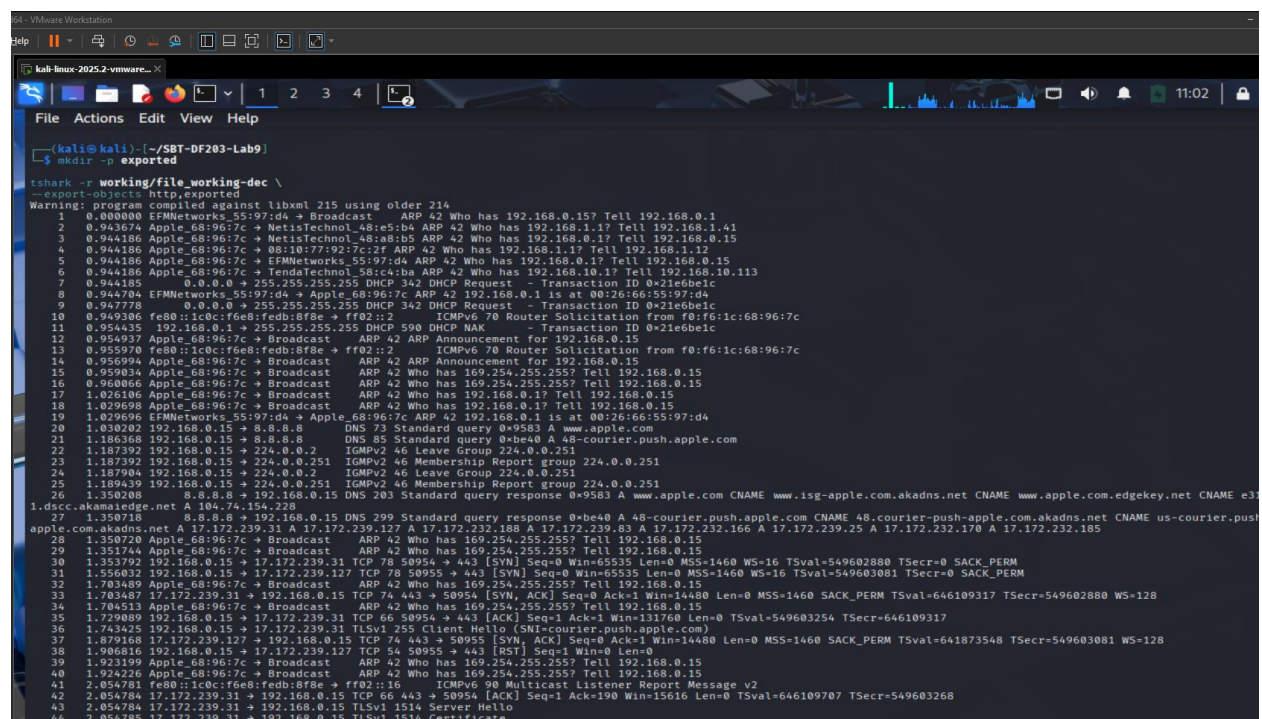
HTTP Object Recovery: Initial Extraction Attempt

Following successful WEP decryption and validation of the capture, the decrypted Ethernet capture was used as the source for HTTP object recovery. An initial TShark

extraction command was executed against working/file_working-dec with the intention of exporting HTTP objects to the exported directory.

The resulting terminal output displayed packet-level information rather than a list of recovered objects. The output included ARP, DHCP, DNS, TCP, TLS, mDNS and ICMPv6 traffic, confirming that the decrypted capture remained readable and was being dissected successfully. However, the displayed packet records do not constitute evidence that HTTP objects had been successfully exported.

Therefore, this output is treated as an intermediate command result rather than a completed object-recovery finding. Object recovery will be validated separately by inspecting the contents of the designated exported directory and subsequently identifying and hashing the recovered files.



```
kali-linux-2025.2-vmware-
(kali@kali) (~/.SBT-DF203-Lab9)
$ mkdir -p exported
tshark -r working/file_working-dec \
--export-objects http,exported
Warning: program compiled against libxml 215 using older 214
1 0.000000 EFMNetworks:55:97:d4 → Broadcast ARP 42 Who has 192.168.0.15? Tell 192.168.0.1
2 0.943674 Apple:68:96:7c → NetisTechnol_48:e5:b4 ARP 42 Who has 192.168.1.1? Tell 192.168.1.41
3 0.944186 Apple:68:96:7c → NetisTechnol_48:a8:b5 ARP 42 Who has 192.168.0.1? Tell 192.168.0.15
4 0.944186 Apple:68:96:7c → 08:10:77:92:7c:2f ARP 42 Who has 192.168.1.1? Tell 192.168.1.12
5 0.944186 Apple:68:96:7c → EFMNetworks:55:97:d4 ARP 42 Who has 192.168.0.1? Tell 192.168.0.15
6 0.944186 Apple:68:96:7c → Tendatechnol_58:c4:ba ARP 42 Who has 192.168.10.1? Tell 192.168.10.113
7 0.944185 0.0.0.0 → 255.255.255.255 DHCP 342 DHCP Request - Transaction ID 0x21e6b1c
8 0.944704 EFMNetworks:55:97:d4 → Apple:68:96:7c ARP 42 192.168.0.1 is at 00:26:66:55:97:d4
9 0.947778 0.0.0.0 → 255.255.255.255 DHCP 342 DHCP Request - Transaction ID 0x21e6b1c
10 0.949206 re80:1c0c:f6e8:fedb:8f8e → ff02::2 ICMPv6 70 Router Solicitation from f0:f6:1c:68:96:7c
11 0.954435 192.168.0.1 → 255.255.255.255 DHCP 590 DHCP NAK - Transaction ID 0x21e6b1c
12 0.954937 Apple:68:96:7c → Broadcast ARP 42 ARP Announcement for 192.168.0.15
13 0.955970 re80:1c0c:f6e8:fedb:8f8e → ff02::2 ICMPv6 70 Router Solicitation from f0:f6:1c:68:96:7c
14 0.955994 Apple:68:96:7c → Broadcast ARP 42 ARP Announcement for 192.168.0.15
15 0.959834 Apple:68:96:7c → Broadcast ARP 42 Who has 169.254.255.255? Tell 192.168.0.15
16 0.960866 Apple:68:96:7c → Broadcast ARP 42 Who has 169.254.255.255? Tell 192.168.0.15
17 1.026106 Apple:68:96:7c → Broadcast ARP 42 Who has 192.168.0.1? Tell 192.168.0.15
18 1.029698 Apple:68:96:7c → Broadcast ARP 42 Who has 192.168.0.1? Tell 192.168.0.15
19 1.029698 EFMNetworks:55:97:d4 → Apple:68:96:7c ARP 42 192.168.0.1 is at 00:26:66:55:97:d4
20 1.030202 192.168.0.15 → 8.8.8.8 DNS 73 Standard query 0x9583 A www.apple.com
21 1.186368 192.168.0.15 → 8.8.8.8 DNS 85 Standard query 0xbe40 A 48-courier.push.apple.com
22 1.187392 192.168.0.15 → 224.0.0.2 IGMPv2 46 Leave Group 224.0.0.251
23 1.187392 192.168.0.15 → 224.0.0.251 IGMPv2 46 Membership Report group 224.0.0.251
24 1.187904 192.168.0.15 → 224.0.0.2 IGMPv2 46 Leave Group 224.0.0.251
25 1.189439 192.168.0.15 → 224.0.0.251 IGMPv2 46 Membership Report group 224.0.0.251
26 1.350208 8.8.8.8 → 192.168.0.15 DNS 203 Standard query response 0x9583 A www.apple.com CNAME www.isg-apple.com.akadns.net CNAME www.apple.com.edgekey.net CNAME e31
1.dscc.akadns.net 8.104.74.154.228
27 1.350718 8.8.8.8 → 192.168.0.15 DNS 299 Standard query response 0xbe40 A 48-courier.push.apple.com CNAME 48.courier-push.apple.com.akadns.net CNAME us-courier-push
apple.com.akadns.net A 17.172.239.31 A 17.172.239.127 A 17.172.232.188 A 17.172.239.83 A 17.172.232.166 A 17.172.239.25 A 17.172.232.170 A 17.172.232.185
28 1.350720 Apple:68:96:7c → Broadcast ARP 42 Who has 169.254.255.255? Tell 192.168.0.15
29 1.351744 Apple:68:96:7c → Broadcast ARP 42 Who has 169.254.255.255? Tell 192.168.0.15
30 1.353792 192.168.0.15 → 17.172.239.31 TCP 78 50954 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=16 TSval=549602880 TSecr=0 SACK_PERM
31 1.556032 192.168.0.15 → 17.172.239.127 TCP 78 50955 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=16 TSval=549603081 TSecr=0 SACK_PERM
32 1.703489 Apple:68:96:7c → Broadcast ARP 42 Who has 169.254.255.255? Tell 192.168.0.15
33 1.703487 17.172.239.31 → 192.168.0.15 TCP 74 443 → 50954 [SYN, ACK] Seq=0 Ack=1 Win=14480 Len=0 MSS=1460 SACK_PERM TSval=646109317 TSecr=549602880 WS=128
34 1.704511 Apple:68:96:7c → Broadcast ARP 42 Who has 169.254.255.255? Tell 192.168.0.15
35 1.729800 192.168.0.15 → 17.172.239.127 TCP 66 50954 → 443 [ACK] Seq=1 Ack=1 Win=11760 Len=0 TSval=549603254 TSecr=646109317
36 1.743425 192.168.0.15 → 17.172.239.31 TLSv1 255 Client Hello (SN1-courier.push.apple.com)
37 1.879168 17.172.239.127 → 192.168.0.15 TCP 74 443 → 50955 [SYN, ACK] Seq=0 Ack=1 Win=14480 Len=0 MSS=1460 SACK_PERM TSval=641873548 TSecr=549603081 WS=128
38 1.906016 192.168.0.15 → 17.172.239.127 TCP 54 50955 → 443 [RST] Seq=1 Win=0 Len=0
39 1.923199 Apple:68:96:7c → Broadcast ARP 42 Who has 169.254.255.255? Tell 192.168.0.15
40 1.924226 Apple:68:96:7c → Broadcast ARP 42 Who has 169.254.255.255? Tell 192.168.0.15
41 2.054781 re80:1c0c:f6e8:fedb:8f8e → ff02::16 ICMPv6 90 Multicast Listener Report Message v2
42 2.054784 17.172.239.31 → 192.168.0.15 TCP 66 443 → 50954 [ACK] Seq=1 Ack=190 Win=15616 Len=0 TSval=646109707 TSecr=549603268
43 2.054784 17.172.239.31 → 192.168.0.15 TLSv1 1514 Server Hello
44 2.054785 17.172.239.31 → 192.168.0.15 TLSv1 1514 Certificate
```

```
kali-linux-2025.2-vmware-
File Actions Edit View Help
43 2.054784 17.172.239.31 → 192.168.0.15 TLSv1 1514 Server Hello
44 2.054785 17.172.239.31 → 192.168.0.15 TLSv1 1514 Certificate
45 2.055297 17.172.239.31 → 192.168.0.15 TLSv1 194 Server Key Exchange, Certificate Request, Server Hello Done
46 2.056834 fe80::1c0c:f6e8:fedb:8f8e → ff02::16 ICMPv6 90 Multicast Listener Report Message v2
47 2.088572 0.0.0.0 → 255.255.255.255 DHCP 342 DHCP Request - Transaction ID 0x21e6beid
48 2.092161 0.0.0.0 → 255.255.255.255 DHCP 342 DHCP Request - Transaction ID 0x21e6beid
49 2.094716 192.168.0.15 → 17.172.239.31 TCP 66 50954 → 443 [ACK] Seq=190 Ack=2897 Win=128864 Len=0 TSval=549603611 TSecr=646109715
50 2.094716 192.168.0.15 → 17.172.239.31 TCP 66 50954 → 443 [ACK] Seq=190 Ack=3025 Win=138736 Len=0 TSval=549603611 TSecr=646109715
51 2.095232 192.168.0.15 → 192.168.0.15 DHCP 590 DHCP ACK - Transaction ID 0x21e6beid
52 2.229373 192.168.0.15 → 17.172.239.31 TLSv1 840 Certificate
53 2.255997 Apple 68:96:7c → Broadcast ARP 42 Who has 169.254.255.255? Tell 192.168.0.15
54 2.257826 Apple 68:96:7c → Broadcast ARP 42 Who has 169.254.255.255? Tell 192.168.0.15
55 2.580606 17.172.239.31 → 192.168.0.15 TCP 66 443 → 50954 [ACK] Seq=3025 Ack=964 Win=17152 Len=0 TSval=646110233 TSecr=549603649
56 2.608253 192.168.0.15 → 17.172.239.31 TLSv1 339 Client Key Exchange, Certificate Verify, Change Cipher Spec, Encrypted Handshake Message
57 2.930878 17.172.239.31 → 192.168.0.15 TCP 66 443 → 50954 [ACK] Seq=3025 Ack=1217 Win=18688 Len=0 TSval=646110571 TSecr=549604128
58 2.930878 17.172.239.31 → 192.168.0.15 TLSv1 125 Change Cipher Spec, Encrypted Handshake Message
59 2.960864 192.168.0.15 → 17.172.239.31 TCP 66 50954 → 443 [ACK] Seq=1237 Ack=3084 Win=131008 Len=0 TSval=549604479 TSecr=646110573
60 2.961008 192.168.0.15 → 17.172.239.31 TLSv1 199 Application Data
61 2.283645 17.172.239.31 → 192.168.0.15 TLSv1 119 Application Data
62 3.311294 Apple 68:96:7c → Broadcast ARP 42 Who has 192.168.0.17? Tell 192.168.0.15
63 3.312328 Apple 68:96:7c → Broadcast ARP 42 Who has 192.168.0.17? Tell 192.168.0.15
64 3.312318 EFMNetworks 55:97:d4 → Apple 68:96:7c ARP 42 192.168.0.1 is at 00:24:66:55:97:d4
65 3.313343 192.168.0.15 → 17.172.239.31 TCP 66 50954 → 443 [ACK] Seq=1370 Ack=3137 Win=131008 Len=0 TSval=549604829 TSecr=646110924
66 3.314878 192.168.0.15 → 17.172.239.31 TLSv1 103 Application Data
67 3.459264 Apple 68:96:7c → Broadcast ARP 42 ARP Announcement for 192.168.0.15
68 3.460287 Apple 68:96:7c → Broadcast ARP 42 ARP Announcement for 192.168.0.15
69 3.635352 17.172.239.31 → 192.168.0.15 TCP 66 443 → 50954 [ACK] Seq=3137 Ack=1407 Win=20224 Len=0 TSval=646111318 TSecr=549604833
70 3.635352 192.168.0.15 → 17.172.239.31 TLSv1 321 Application Data, Application Data, Application Data
71 3.810560 Apple 68:96:7c → Broadcast ARP 42 Who has 192.168.0.17? Tell 192.168.0.15
72 3.811581 Apple 68:96:7c → Broadcast ARP 42 Who has 192.168.0.17? Tell 192.168.0.15
73 3.811579 EFMNetworks 55:97:d4 → Apple 68:96:7c ARP 42 192.168.0.1 is at 00:24:66:55:97:d4
74 3.987707 17.172.239.31 → 192.168.0.15 TCP 66 443 → 50954 [ACK] Seq=3137 Ack=1662 Win=21760 Len=0 TSval=646111627 TSecr=549605181
75 3.987707 17.172.239.31 → 192.168.0.15 TLSv1 103 Application Data
76 4.016892 fe80::1c0c:f6e8:fedb:8f8e → ff02::16 ICMPv6 90 Multicast Listener Report Message v2
77 4.016892 fe80::1c0c:f6e8:fedb:8f8e → ff02::16 ICMPv6 90 Multicast Listener Report Message v2
78 4.016895 192.168.0.15 → 17.172.239.31 TCP 66 50954 → 443 [ACK] Seq=1662 Ack=3174 Win=131024 Len=0 TSval=549605533 TSecr=646111627
79 5.743932 fe80::1c0c:f6e8:fedb:8f8e → ff02::2 ICMPv6 70 Router Solicitation from f0:f6:1c:68:96:7c
80 5.744955 fe80::1c0c:f6e8:fedb:8f8e → ff02::2 ICMPv6 70 Router Solicitation from f0:f6:1c:68:96:7c
81 7.023544 192.168.0.15 → 224.0.0.251 MDNS 146 Standard query response 0x0000 TXT OPT
82 7.025592 fe80::1c0c:f6e8:fedb:8f8e → ff02::fb MDNS 166 Standard query response 0x0000 TXT OPT
83 7.027133 192.168.0.15 → 224.0.0.251 MDNS 72 Standard query response 0x0000 TXT OPT
84 7.029180 fe80::1c0c:f6e8:fedb:8f8e → ff02::fb MDNS 166 Standard query response 0x0000 TXT OPT
85 8.023553 192.168.0.15 → 224.0.0.251 MDNS 146 Standard query response 0x0000 TXT OPT
86 8.023553 fe80::1c0c:f6e8:fedb:8f8e → ff02::fb MDNS 166 Standard query response 0x0000 TXT OPT
87 8.025086 192.168.0.15 → 224.0.0.251 MDNS 146 Standard query response 0x0000 TXT OPT
88 8.027134 fe80::1c0c:f6e8:fedb:8f8e → ff02::fb MDNS 166 Standard query response 0x0000 TXT OPT
89 8.869950 EFMNetworks 55:97:d4 → Apple 68:96:7c ARP 42 Who has 192.168.0.15? Tell 192.168.0.1
90 8.870083 Apple 68:96:7c → EFMNetworks 55:97:d4 ARP 42 192.168.0.15 is at f0:f6:1c:68:96:7c
91 9.997440 192.168.0.15 → 224.0.0.251 MDNS 146 Standard query response 0x0000 TXT OPT
92 9.997440 fe80::1c0c:f6e8:fedb:8f8e → ff02::fb MDNS 166 Standard query response 0x0000 TXT OPT
93 10.005121 192.168.0.15 → 224.0.0.251 MDNS 146 Standard query response 0x0000 TXT OPT
94 10.007680 fe80::1c0c:f6e8:fedb:8f8e → ff02::fb MDNS 166 Standard query response 0x0000 TXT OPT
95 10.303614 fe80::1c0c:f6e8:fedb:8f8e → ff02::2 ICMPv6 70 Router Solicitation from f0:f6:1c:68:96:7c
```

```
kali-linux-2025.2-vmware-
File Actions Edit View Help
94 10.007680 fe80::1c0c:f6e8:fedb:8f8e → ff02::fb MDNS 166 Standard query response 0x0000 TXT OPT
95 10.303614 fe80::1c0c:f6e8:fedb:8f8e → ff02::2 ICMPv6 70 Router Solicitation from f0:f6:1c:68:96:7c
96 10.304640 fe80::1c0c:f6e8:fedb:8f8e → ff02::2 ICMPv6 70 Router Solicitation from f0:f6:1c:68:96:7c
97 13.465484 192.168.0.15 → 8.8.8.8 DNS 72 Standard query response 0x0000 A m.reddit.com
98 13.451024 8.8.8.8 → 192.168.0.15 DNS 112 Standard query response 0x0000 A m.reddit.com A 198.41.209.137 A 198.41.208.140 A 198.41.208.141 A 198.41.208.142
09 136 A 198.41.209.142 A 198.41.208.143 A 198.41.208.139 A 198.41.209.140 A 198.41.209.139 A 198.41.208.137 A 198.41.209.141 A 198.41.209.143 A 198.41.208.138 A 198.41.208.142
99 13.669693 192.168.0.15 → 198.41.209.137 TCP 78 50956 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=16 TSval=549615188 TSecr=0 SACK_PERM
100 13.817728 198.41.209.137 → 192.168.0.15 TCP 66 80 → 50956 [SYN, ACK] Seq=0 Ack=1 Win=29200 Len=0 MSS=1460 SACK_PERM WS=1024
101 13.844351 192.168.0.15 → 198.41.209.137 TCP 54 50956 → 80 [ACK] Seq=1 Ack=1 Win=262144 Len=0
102 13.847934 192.168.0.15 → 198.41.209.137 HTTP 740 GET / HTTP/1.1
103 13.938592 198.41.209.137 → 192.168.0.15 TCP 54 80 → 50956 [ACK] Seq=1 Ack=687 Win=30720 Len=0
104 13.997438 192.168.0.15 → 224.0.0.251 MDNS 146 Standard query response 0x0000 TXT OPT
105 13.997439 fe80::1c0c:f6e8:fedb:8f8e → ff02::fb MDNS 166 Standard query response 0x0000 TXT OPT
106 13.998978 192.168.0.15 → 224.0.0.251 MDNS 146 Standard query response 0x0000 TXT OPT
107 14.817728 198.41.209.137 → 192.168.0.15 HTTP 1514 [TCP Previous segment not captured] Continuation
108 14.817728 198.41.209.137 → 192.168.0.15 TCP 1514 [TCP Retransmission] 80 → 50956 [ACK] Seq=1 Ack=687 Win=30720 Len=1460
109 14.818240 198.41.209.137 → 192.168.0.15 HTTP 1514 [TCP Previous segment not captured] Continuation
110 14.818240 198.41.209.137 → 192.168.0.15 TCP 1514 [TCP Retransmission] 80 → 50956 [ACK] Seq=2921 Ack=687 Win=30720 Len=1460
111 14.818752 198.41.209.137 → 192.168.0.15 HTTP 1514 Continuation
112 14.819264 198.41.209.137 → 192.168.0.15 HTTP 1514 Continuation
113 14.819265 198.41.209.137 → 192.168.0.15 HTTP 1514 Continuation
114 14.819777 198.41.209.137 → 192.168.0.15 HTTP 1514 Continuation
115 14.819777 198.41.209.137 → 192.168.0.15 HTTP 873 [TCP Previous segment not captured] Continuation
116 14.820289 198.41.209.137 → 192.168.0.15 TCP 1514 [TCP Retransmission] 80 → 50956 [ACK] Seq=11681 Ack=687 Win=30720 Len=1460
117 14.843326 192.168.0.15 → 198.41.209.137 TCP 66 [TCP Dup ACK 10181] 50956 → 80 [ACK] Seq=687 Ack=1 Win=262144 Len=0 SLE=1461 SRE=2921
118 14.843326 192.168.0.15 → 198.41.209.137 TCP 54 50956 → 80 [ACK] Seq=687 Ack=2921 Win=259216 Len=0
119 14.843326 192.168.0.15 → 198.41.209.137 TCP 66 [TCP Dup ACK 11881] 50956 → 80 [ACK] Seq=687 Ack=2921 Win=259216 Len=0 SLE=4381 SRE=5841
120 14.843326 192.168.0.15 → 198.41.209.137 TCP 54 50956 → 80 [ACK] Seq=687 Ack=5841 Win=256304 Len=0
121 14.843326 192.168.0.15 → 198.41.209.137 TCP 54 50956 → 80 [ACK] Seq=687 Ack=8761 Win=253376 Len=0
122 14.843838 192.168.0.15 → 198.41.209.137 TCP 54 50956 → 80 [ACK] Seq=687 Ack=11681 Win=250464 Len=0 SLE=13141 SRE=13960
123 14.843838 192.168.0.15 → 198.41.209.137 TCP 66 [TCP Dup ACK 12281] 50956 → 80 [ACK] Seq=687 Ack=11681 Win=250464 Len=0 SLE=13141 SRE=13960
124 14.843838 192.168.0.15 → 198.41.209.137 TCP 54 50956 → 80 [ACK] Seq=687 Ack=13968 Win=248176 Len=0
125 14.844350 192.168.0.15 → 198.41.209.137 TCP 54 [TCP Window Update] 50956 → 80 [ACK] Seq=687 Ack=13968 Win=252272 Len=0
126 14.844350 192.168.0.15 → 198.41.209.137 TCP 54 [TCP Window Update] 50956 → 80 [ACK] Seq=687 Ack=13968 Win=262128 Len=0
127 14.946750 192.168.0.15 → 8.8.8.8 DNS 81 Standard query response 0x22e7 A pixel.redditmedia.com
128 15.804480 8.8.8.8 → 192.168.0.15 DNS 265 Standard query response 0x22e7 A pixel.redditmedia.com CNAME pixelb-1456612799.us-east-1.elb.amazonaws.com A 23.21.196.154 A
23 21.88.241 A 23.23.110.30 A 23.23.143.250 A 107.20.196.53 A 184.73.157.198 A 107.20.184.26 A 54.243.36.132
129 15.110590 192.168.0.15 → 23.21.196.154 TCP 78 50957 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=16 TSval=549616620 TSecr=0 SACK_PERM
130 15.434680 23.21.196.154 → 192.168.0.15 TCP 74 80 → 50957 [SYN, ACK] Seq=0 Ack=1 Win=14480 Len=0 MSS=1460 SACK_PERM TSval=543956293 TSecr=549616620 WS=256
131 15.450749 192.168.0.15 → 23.21.196.154 TCP 66 50957 → 80 [ACK] Seq=1 Ack=1 Win=131760 Len=0 TSval=549616967 TSecr=543956293
132 15.461822 192.168.0.15 → 23.21.196.154 HTTP 493 GET /pixel/of_destiny.png?v=D09724Zz0RSDwp18HP6SgbKcwddtuhtk7f82bV9AXtFHYN2FXH1dpdfjUjC18RAWL5s0Ho8NnqzJg1K3D HTTP/1.1
133 15.763902 192.168.0.15 → 23.21.196.154 TCP 66 [TCP ACKed unseen segment] 50957 → 80 [ACK] Seq=428 Ack=334 Win=131424 Len=0 TSval=549617270 TSecr=543956361
134 16.865339 17.172.232.98 → 192.168.0.15 TLSv1 103 Encrypted Alert
135 16.889919 192.168.0.15 → 17.172.232.98 TCP 54 50948 → 443 [RST] Seq=1 Win=0 Len=0
136 20.638984 192.168.0.15 → 8.8.8.8 DNS 69 Standard query response 0x6b95 A imgur.com
137 20.152335 8.8.8.8 → 192.168.0.15 DNS 85 Standard query response 0x6b95 A imgur.com A 199.27.79.193
138 20.145401 192.168.0.15 → 199.27.79.193 TCP 78 50958 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=16 TSval=549621648 TSecr=0 SACK_PERM
139 20.372731 192.168.0.15 → 199.27.79.193 TCP 66 50958 → 80 [ACK] Seq=1 Ack=2 Win=131760 Len=0 TSval=549621875 TSecr=2558576616
140 20.370980 192.168.0.15 → 199.27.79.193 TCP 1514 GET /8iVv7t6 HTTP/1.1
141 20.379899 192.168.0.15 → 199.27.79.193 HTTP 365 GET /8iVv7t6 HTTP/1.1
142 20.553984 199.27.79.193 → 192.168.0.15 TCP 66 80 → 50958 [ACK] Seq=1 Ack=1449 Win=34 Len=0 TSval=2558576675 TSecr=549621881
143 20.553984 199.27.79.193 → 192.168.0.15 TCP 66 80 → 50958 [ACK] Seq=1 Ack=1746 Win=40 Len=0 TSval=2558576675 TSecr=549621881
144 20.859200 199.27.79.193 → 192.168.0.15 TCP 1514 HTTP/1.1 200 OK
```

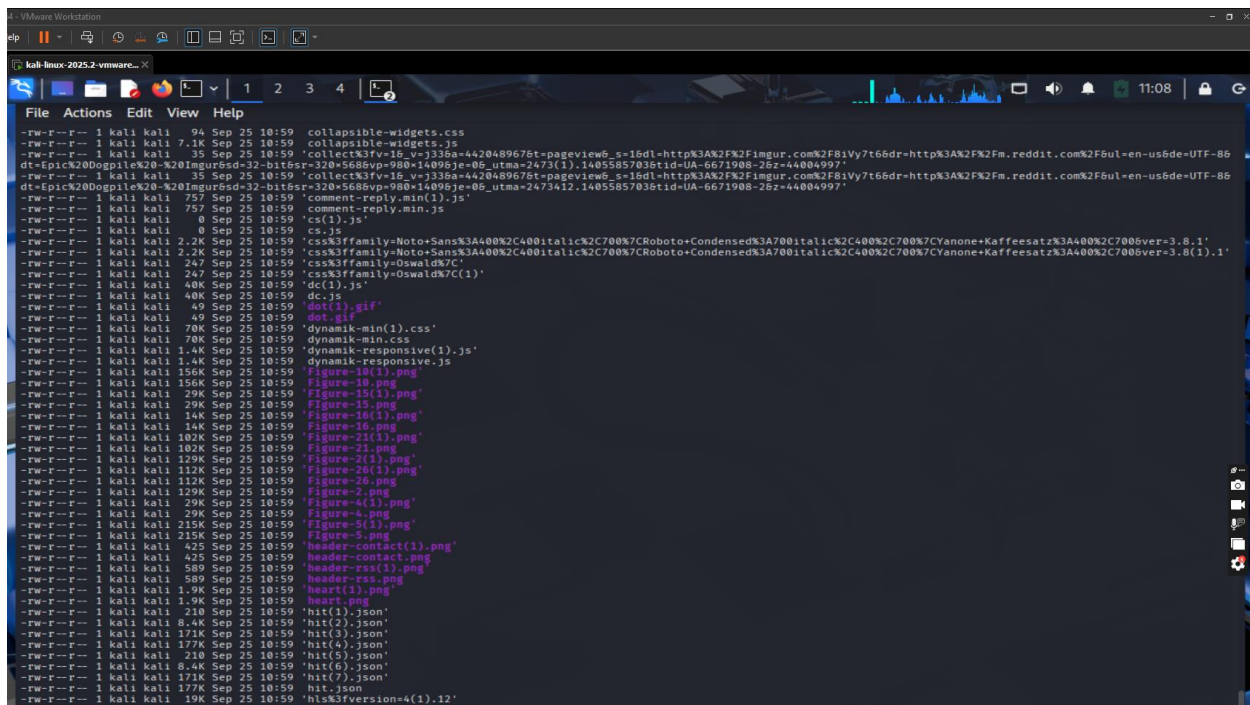
```
File Actions Edit View Help
146 20.859711 199.27.79.193 → 192.168.0.15 TCP 1514 80 → 50958 [ACK] Seq=2897 Ack=1748 Win=40 Len=1448 TSval=2558576742 TSecr=549621881
147 20.860224 199.27.79.193 → 192.168.0.15 TCP 1514 [TCP Previous segment not captured] 80 → 50958 [ACK] Seq=5793 Ack=1748 Win=40 Len=1448 TSval=2558576742 TSecr=549621881
148 20.860735 199.27.79.193 → 192.168.0.15 TCP 1514 80 → 50958 [ACK] Seq=7241 Ack=1748 Win=40 Len=1448 TSval=2558576742 TSecr=549621881
149 20.860735 199.27.79.193 → 192.168.0.15 TCP 1514 80 → 50958 [ACK] Seq=8689 Ack=1748 Win=40 Len=1448 TSval=2558576742 TSecr=549621881
150 20.861247 199.27.79.193 → 192.168.0.15 TCP 1514 80 → 50958 [ACK] Seq=10117 Ack=1748 Win=40 Len=1448 TSval=2558576742 TSecr=549621881
151 20.862271 199.27.79.193 → 192.168.0.15 TCP 1514 80 → 50958 [ACK] Seq=11585 Ack=1748 Win=40 Len=1448 TSval=2558576742 TSecr=549621881
152 20.862271 199.27.79.193 → 192.168.0.15 TCP 598 80 → 50958 [PSH, ACK] Seq=13033 Ack=1748 Win=40 Len=532 TSval=2558576742 TSecr=549621881
153 20.885820 192.168.0.15 → 199.27.79.193 TCP 66 50958 → 80 [ACK] Seq=1748 Ack=2897 Win=128864 Len=0 TSval=549622385 TSecr=2558576742
154 20.885820 192.168.0.15 → 199.27.79.193 TCP 66 [TCP ACKed unseen segment] 50958 → 80 [ACK] Seq=1748 Ack=5793 Win=125968 Len=0 TSval=549622385 TSecr=2558576742
155 20.886332 192.168.0.15 → 199.27.79.193 TCP 66 50958 → 80 [ACK] Seq=1748 Ack=8689 Win=125072 Len=0 TSval=549622385 TSecr=2558576742
156 20.886332 192.168.0.15 → 199.27.79.193 TCP 66 50958 → 80 [ACK] Seq=1748 Ack=11585 Win=120176 Len=0 TSval=549622385 TSecr=2558576742
157 20.886332 192.168.0.15 → 199.27.79.193 TCP 66 50958 → 80 [ACK] Seq=1748 Ack=13565 Win=118192 Len=0 TSval=549622385 TSecr=2558576742
158 20.887356 192.168.0.15 → 199.27.79.193 TCP 66 [TCP Window Update] 50958 → 80 [ACK] Seq=1748 Ack=13565 Win=121600 Len=0 TSval=549622387 TSecr=2558576742
159 20.889916 192.168.0.15 → 199.27.79.193 TCP 66 [TCP Window Update] 50958 → 80 [ACK] Seq=1748 Ack=13565 Win=131072 Len=0 TSval=549622387 TSecr=2558576742
160 20.886172 192.168.0.15 → 8.8.8.8 DNS 71 Standard query 0=3de0 A 1.imgur.com
161 21.070655 8.8.8.8 → 192.168.0.15 DNS 101 Standard query response 0=3de0 A 1.imgur.com CNAME imgur.com A 199.27.79.193
162 21.074236 192.168.0.15 → 199.27.79.193 TCP 78 50959 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=16 TSval=549622772 TSecr=0 SACK_PERM
163 21.223743 199.27.79.193 → 192.168.0.15 TCP 74 80 → 50959 [SYN, ACK] Seq=0 Ack=1 Win=14480 Len=0 MSS=1460 SACK_PERM TSval=2417916410 TSecr=549622572 WS=512
164 21.249853 192.168.0.15 → 199.27.79.193 TCP 66 50959 → 80 [ACK] Seq=1 Ack=1 Win=131760 Len=0 TSval=549622747 TSecr=2417916410
165 21.250365 192.168.0.15 → 8.8.8.8 DNS 76 Standard query 0=b55c A www.facebook.com
166 21.257021 192.168.0.15 → 199.27.79.193 TCP 1514 GET /81vY7t6.jpg HTTP/1.1
167 21.258556 192.168.0.15 → 199.27.79.193 HTTP 108 GET /81vY7t6.jpg HTTP/1.1
168 21.316556 8.8.8.8 → 192.168.0.15 DNS 116 Standard query response 0=b55c A www.facebook.com CNAME star.c10r.facebook.com A 31.13.70.1
169 21.329724 192.168.0.15 → 31.13.70.1 TCP 78 50960 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=16 TSval=549622824 TSecr=0 SACK_PERM
170 21.422400 199.27.79.193 → 192.168.0.15 TCP 66 80 → 50959 [ACK] Seq=1 Ack=1449 Win=17408 Len=0 TSval=2417916456 TSecr=549622751
171 21.422400 199.27.79.193 → 192.168.0.15 TCP 66 80 → 50959 [ACK] Seq=1 Ack=1491 Win=17408 Len=0 TSval=2417916456 TSecr=549622751
172 21.423423 199.27.79.193 → 192.168.0.15 TCP 1514 HTTP/1.1 200 OK
173 21.423423 199.27.79.193 → 192.168.0.15 TCP 1514 80 → 50959 [ACK] Seq=1449 Ack=1491 Win=17408 Len=1448 TSval=2417916456 TSecr=549622751
174 21.423935 199.27.79.193 → 192.168.0.15 TCP 1514 80 → 50959 [ACK] Seq=2097 Ack=1491 Win=17408 Len=1448 TSval=2417916456 TSecr=549622751
175 21.423935 199.27.79.193 → 192.168.0.15 TCP 1514 80 → 50959 [ACK] Seq=4345 Ack=1491 Win=17408 Len=1448 TSval=2417916456 TSecr=549622751
176 21.424448 199.27.79.193 → 192.168.0.15 TCP 1514 80 → 50959 [ACK] Seq=5793 Ack=1491 Win=17408 Len=1448 TSval=2417916456 TSecr=549622751
177 21.424448 199.27.79.193 → 192.168.0.15 TCP 1514 80 → 50959 [ACK] Seq=7241 Ack=1491 Win=17408 Len=1448 TSval=2417916456 TSecr=549622751
178 21.424960 199.27.79.193 → 192.168.0.15 TCP 1514 80 → 50959 [ACK] Seq=8689 Ack=1491 Win=17408 Len=1448 TSval=2417916456 TSecr=549622751
179 21.425472 199.27.79.193 → 192.168.0.15 TCP 1514 [TCP Previous segment not captured] 80 → 50959 [ACK] Seq=11585 Ack=1491 Win=17408 Len=1448 TSval=2417916456 TSecr=549622751
1
180 21.425980 192.168.0.15 → 199.27.79.193 TCP 66 50959 → 80 [ACK] Seq=1491 Ack=2897 Win=128864 Len=0 TSval=549622918 TSecr=2417916456
181 21.425980 192.168.0.15 → 199.27.79.193 TCP 66 50959 → 80 [ACK] Seq=1491 Ack=5793 Win=128176 Len=0 TSval=549622918 TSecr=2417916456
182 21.425980 192.168.0.15 → 199.27.79.193 TCP 66 50959 → 80 [ACK] Seq=1491 Ack=8689 Win=125208 Len=0 TSval=549622918 TSecr=2417916456
183 21.429053 192.168.0.15 → 199.27.79.193 TCP 78 50959 → 80 [ACK] Seq=1491 Ack=10137 Win=123824 Len=0 TSval=549622919 TSecr=2417916456 SLE=11585 SRE=13033
184 21.429053 192.168.0.15 → 199.27.79.193 TCP 78 [TCP Dup ACK 378#1] 50959 → 80 [ACK] Seq=1491 Ack=10137 Win=131072 Len=0 TSval=549622920 TSecr=2417916456 SLE=11585 SRE=13033
3
185 21.506880 31.13.70.1 → 192.168.0.15 TCP 74 443 → 50960 [SYN, ACK] Seq=0 Ack=1 Win=13980 Len=0 MSS=1410 SACK_PERM TSval=4264302262 TSecr=549622824 WS=256
186 21.507900 192.168.0.15 → 31.13.70.1 TCP 66 50960 → 443 [ACK] Seq=1 Ack=1 Win=131408 Len=0 TSval=549623000 TSecr=4264302262
187 21.514556 192.168.0.15 → 31.13.70.1 TLSv1 281 Client Hello (SNW=www.facebook.com)
188 21.585216 199.27.79.193 → 192.168.0.15 TCP 1514 [TCP Previous segment not captured] 80 → 50959 [ACK] Seq=14481 Ack=1491 Win=17408 Len=1448 TSval=2417916498 TSecr=54962291
8
189 21.585216 199.27.79.193 → 192.168.0.15 TCP 1514 80 → 50959 [ACK] Seq=15929 Ack=1491 Win=17408 Len=1448 TSval=2417916498 TSecr=549622918
190 21.585216 199.27.79.193 → 192.168.0.15 TCP 1514 80 → 50959 [ACK] Seq=17377 Ack=1491 Win=17408 Len=1448 TSval=2417916498 TSecr=549622918
191 21.585216 199.27.79.193 → 192.168.0.15 TCP 1514 80 → 50959 [ACK] Seq=18825 Ack=1491 Win=17408 Len=1448 TSval=2417916498 TSecr=549622918
192 21.586248 199.27.79.193 → 192.168.0.15 TCP 1514 80 → 50959 [ACK] Seq=20273 Ack=1491 Win=17408 Len=1448 TSval=2417916498 TSecr=549622918
193 21.586248 199.27.79.193 → 192.168.0.15 TCP 1514 80 → 50959 [ACK] Seq=21721 Ack=1491 Win=17408 Len=1448 TSval=2417916498 TSecr=549622918
194 21.586752 199.27.79.193 → 192.168.0.15 TCP 1514 80 → 50959 [ACK] Seq=23169 Ack=1491 Win=17408 Len=1448 TSval=2417916498 TSecr=549622918
195 21.586752 199.27.79.193 → 192.168.0.15 TCP 1514 80 → 50959 [ACK] Seq=24617 Ack=1491 Win=17408 Len=1448 TSval=2417916498 TSecr=549622918
```

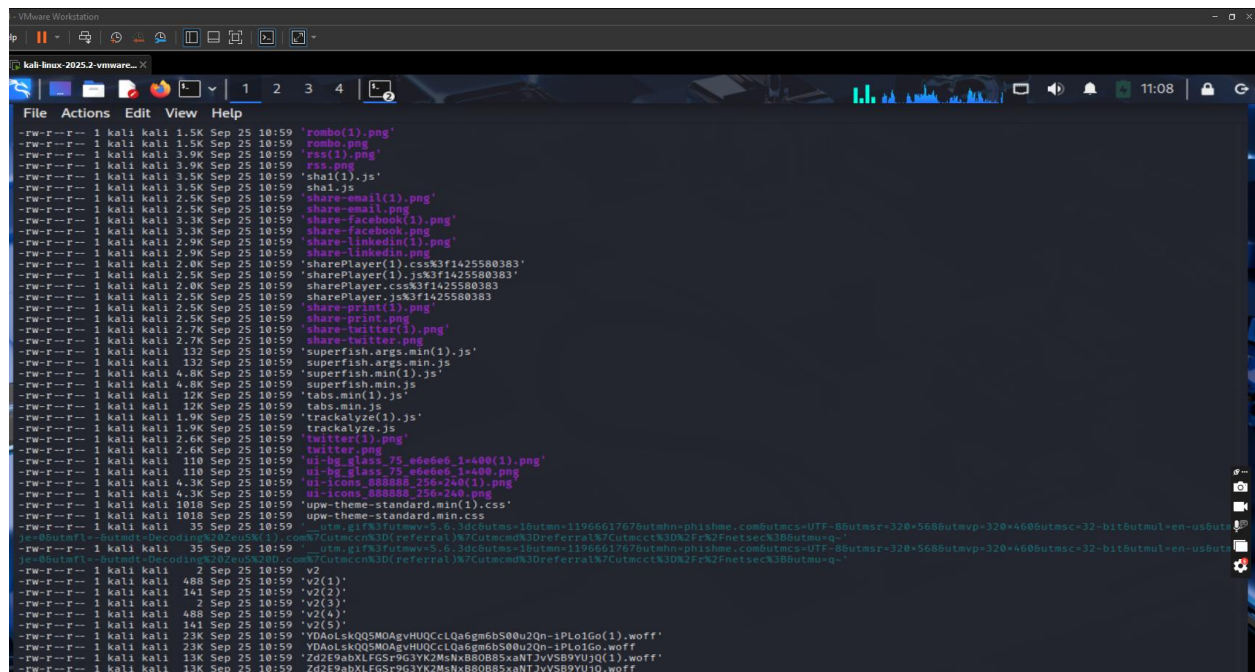
```
File Actions Edit View Help
83
376 22.657920 199.27.79.193 → 192.168.0.15 TCP 1514 80 → 50959 [PSH, ACK] Seq=127425 Ack=1491 Win=17408 Len=1448 TSval=2417916769 TSecr=549623983
377 22.658431 199.27.79.193 → 192.168.0.15 TCP 1514 80 → 50959 [ACK] Seq=128873 Ack=1491 Win=17408 Len=1448 TSval=2417916769 TSecr=549623983
378 22.658940 192.168.0.15 → 199.27.79.193 TCP 78 50959 → 80 [ACK] Seq=1491 Ack=124529 Win=131072 Len=0 TSval=549624125 TSecr=2417916733 SLE=125977 SRE=127425
379 22.659451 192.168.0.15 → 199.27.79.193 TCP 78 [TCP Dup ACK 378#1] 50959 → 80 [ACK] Seq=1491 Ack=124529 Win=131072 Len=0 TSval=549624125 TSecr=2417916733 SLE=125977 SRE=12
8873
380 22.659455 199.27.79.193 → 192.168.0.15 TCP 1514 80 → 50959 [ACK] Seq=138321 Ack=1491 Win=17408 Len=1448 TSval=2417916769 TSecr=549623983
381 22.659455 199.27.79.193 → 192.168.0.15 TCP 1514 80 → 50959 [ACK] Seq=131769 Ack=1491 Win=17408 Len=1448 TSval=2417916769 TSecr=549623983
382 22.659968 199.27.79.193 → 192.168.0.15 TCP 1514 80 → 50959 [ACK] Seq=133217 Ack=1491 Win=17408 Len=1448 TSval=2417916769 TSecr=549623984
383 22.660479 199.27.79.193 → 192.168.0.15 TCP 1514 80 → 50959 [ACK] Seq=134665 Ack=1491 Win=17408 Len=1448 TSval=2417916769 TSecr=549623984
384 22.660480 199.27.79.193 → 192.168.0.15 TCP 1514 80 → 50959 [ACK] Seq=136113 Ack=1491 Win=17408 Len=1448 TSval=2417916769 TSecr=549623985
385 22.660992 199.27.79.193 → 192.168.0.15 TCP 1514 80 → 50959 [ACK] Seq=137561 Ack=1491 Win=17408 Len=1448 TSval=2417916769 TSecr=549623985
386 22.660993 199.27.79.193 → 192.168.0.15 TCP 1514 80 → 50959 [ACK] Seq=139009 Ack=1491 Win=17408 Len=1448 TSval=2417916769 TSecr=549623985
387 22.661505 199.27.79.193 → 192.168.0.15 TCP 1514 80 → 50959 [ACK] Seq=140457 Ack=1491 Win=17408 Len=1448 TSval=2417916769 TSecr=549623985
388 22.662011 192.168.0.15 → 199.27.79.193 TCP 78 [TCP Dup ACK 378#2] 50959 → 80 [ACK] Seq=1491 Ack=124529 Win=131072 Len=0 TSval=549624125 TSecr=2417916733 SLE=125977 SRE=13
0321
389 22.663848 199.27.79.193 → 192.168.0.15 TCP 1514 80 → 50959 [ACK] Seq=141905 Ack=1491 Win=17408 Len=1448 TSval=2417916770 TSecr=549623986
390 22.663035 192.168.0.15 → 199.27.79.193 TCP 78 [TCP Dup ACK 378#3] 50959 → 80 [ACK] Seq=1491 Ack=124529 Win=131072 Len=0 TSval=549624126 TSecr=2417916733 SLE=125977 SRE=13
1769
391 22.663035 192.168.0.15 → 199.27.79.193 TCP 78 [TCP Dup ACK 378#4] 50959 → 80 [ACK] Seq=1491 Ack=124529 Win=131072 Len=0 TSval=549624126 TSecr=2417916733 SLE=125977 SRE=13
3217
392 22.663035 192.168.0.15 → 199.27.79.193 TCP 78 [TCP Dup ACK 378#5] 50959 → 80 [ACK] Seq=1491 Ack=124529 Win=131072 Len=0 TSval=549624126 TSecr=2417916733 SLE=125977 SRE=13
4665
393 22.663035 192.168.0.15 → 199.27.79.193 TCP 78 [TCP Dup ACK 378#6] 50959 → 80 [ACK] Seq=1491 Ack=124529 Win=131072 Len=0 TSval=549624126 TSecr=2417916733 SLE=125977 SRE=13
6113
394 22.663547 192.168.0.15 → 199.27.79.193 TCP 78 [TCP Dup ACK 378#7] 50959 → 80 [ACK] Seq=1491 Ack=124529 Win=131072 Len=0 TSval=549624127 TSecr=2417916733 SLE=125977 SRE=13
7561
395 22.663547 192.168.0.15 → 199.27.79.193 TCP 78 [TCP Dup ACK 378#8] 50959 → 80 [ACK] Seq=1491 Ack=124529 Win=131072 Len=0 TSval=549624127 TSecr=2417916733 SLE=125977 SRE=13
9009
396 22.663547 192.168.0.15 → 199.27.79.193 TCP 78 [TCP Dup ACK 378#9] 50959 → 80 [ACK] Seq=1491 Ack=124529 Win=131072 Len=0 TSval=549624127 TSecr=2417916733 SLE=125977 SRE=14
0457
397 22.663547 192.168.0.15 → 199.27.79.193 TCP 78 [TCP Dup ACK 378#10] 50959 → 80 [ACK] Seq=1491 Ack=124529 Win=131072 Len=0 TSval=549624127 TSecr=2417916733 SLE=125977 SRE=1
41905
398 22.664059 192.168.0.15 → 199.27.79.193 TCP 78 [TCP Dup ACK 378#11] 50959 → 80 [ACK] Seq=1491 Ack=124529 Win=131072 Len=0 TSval=549624128 TSecr=2417916733 SLE=125977 SRE=1
43353
399 22.739835 192.168.0.15 → 199.27.79.193 TCP 1514 GET /gallery/shot/viral/page/0/hit.json HTTP/1.1
400 22.739836 192.168.0.15 → 199.27.79.193 HTTP 487 GET /gallery/shot/viral/page/0/hit.json HTTP/1.1
401 22.807484 192.168.0.15 → 199.27.79.193 TCP 78 [TCP Dup ACK 378#12] 50959 → 80 [ACK] Seq=1491 Ack=124529 Win=131072 Len=0 TSval=549624270 TSecr=2417916733 SLE=125977 SRE=1
44801
402 22.807484 192.168.0.15 → 199.27.79.193 TCP 78 [TCP Dup ACK 378#13] 50959 → 80 [ACK] Seq=1491 Ack=124529 Win=131072 Len=0 TSval=549624270 TSecr=2417916733 SLE=125977 SRE=1
46249
403 22.807996 192.168.0.15 → 199.27.79.193 TCP 78 [TCP Dup ACK 378#14] 50959 → 80 [ACK] Seq=1491 Ack=124529 Win=131072 Len=0 TSval=549624270 TSecr=2417916733 SLE=125977 SRE=1
47697
404 22.809024 199.27.79.193 → 192.168.0.15 TCP 1514 [TCP Fast Retransmission] 80 → 50959 [ACK] Seq=124529 Ack=1491 Win=17408 Len=1448 TSval=2417916807 TSecr=549624125
405 22.809024 199.27.79.193 → 192.168.0.15 TCP 1514 [TCP Previous segment not captured] 80 → 50959 [ACK] Seq=147697 Ack=1491 Win=17408 Len=1448 TSval=2417916807 TSecr=5496241
26
406 22.812096 199.27.79.193 → 192.168.0.15 TCP 1514 80 → 50959 [ACK] Seq=149145 Ack=1491 Win=17408 Len=1448 TSval=2417916807 TSecr=549624126
407 22.812096 199.27.79.193 → 192.168.0.15 TCP 1514 80 → 50959 [ACK] Seq=150593 Ack=1491 Win=17408 Len=1448 TSval=2417916807 TSecr=549624126
408 22.812608 199.27.79.193 → 192.168.0.15 TCP 1514 80 → 50959 [ACK] Seq=152041 Ack=1491 Win=17408 Len=1448 TSval=2417916807 TSecr=549624127
409 22.812608 199.27.79.193 → 192.168.0.15 TCP 1514 80 → 50959 [ACK] Seq=151489 Ack=1491 Win=17408 Len=1448 TSval=2417916807 TSecr=549624127
410 22.813628 192.168.0.15 → 199.27.79.193 TCP 66 [TCP ACKed unseen segment] 50959 → 80 [ACK] Seq=1491 Ack=147697 Win=107904 Len=0 TSval=549624272 TSecr=2417916807
411 22.813627 192.168.0.15 → 199.27.79.193 TCP 66 50959 → 80 [ACK] Seq=1491 Ack=149145 Win=131072 Len=0 TSval=549624272 TSecr=2417916807
412 22.813628 192.168.0.15 → 199.27.79.193 TCP 66 50959 → 80 [ACK] Seq=1491 Ack=152041 Win=120176 Len=0 TSval=549624274 TSecr=2417916807
```

```
md4 - VMware Workstation
Help
File Actions Edit View Help
kali-linux-2025.2-vmware-...
15429 243.473585 192.168.0.9 -> 192.168.0.15 TCP 1514 5000 -> 51036 [ACK] Seq=121408 Ack=1074 Win=32256 Len=1448 TSval=60880358 TSecr=549842619
15430 243.473585 192.168.0.9 -> 192.168.0.15 TCP 1514 5000 -> 51036 [ACK] Seq=122856 Ack=1074 Win=32256 Len=1448 TSval=60880358 TSecr=549842619
15431 243.473585 192.168.0.9 -> 192.168.0.15 TCP 1514 5000 -> 51036 [ACK] Seq=124304 Ack=1074 Win=32256 Len=1448 TSval=60880358 TSecr=549842620
15432 243.475633 192.168.0.9 -> 192.168.0.15 TCP 1514 5000 -> 51036 [ACK] Seq=125752 Ack=1074 Win=32256 Len=1448 TSval=60880358 TSecr=549842620
15433 243.476145 192.168.0.9 -> 192.168.0.15 TCP 1514 5000 -> 51036 [ACK] Seq=127200 Ack=1074 Win=32256 Len=1448 TSval=60880358 TSecr=549842620
15434 243.480257 192.168.0.15 -> 192.168.0.9 TCP 66 51036 -> 5000 [ACK] Seq=1074 Ack=67832 Win=129616 Len=0 TSval=549842623 TSecr=60880352
15435 243.480258 192.168.0.15 -> 192.168.0.9 TCP 66 51036 -> 5000 [ACK] Seq=1074 Ack=69280 Win=131072 Len=0 TSval=549842624 TSecr=60880352
15436 243.480258 192.168.0.15 -> 192.168.0.9 TCP 66 51036 -> 5000 [ACK] Seq=1074 Ack=72176 Win=129616 Len=0 TSval=549842625 TSecr=60880352
15437 243.480257 192.168.0.15 -> 192.168.0.9 TCP 66 51036 -> 5000 [ACK] Seq=1074 Ack=73624 Win=131072 Len=0 TSval=549842626 TSecr=60880352
15438 243.480258 192.168.0.15 -> 192.168.0.9 TCP 66 51036 -> 5000 [ACK] Seq=1074 Ack=76520 Win=129616 Len=0 TSval=549842627 TSecr=60880352
15439 243.480770 192.168.0.15 -> 192.168.0.9 TCP 66 51036 -> 5000 [ACK] Seq=1074 Ack=77968 Win=131072 Len=0 TSval=549842628 TSecr=60880352
15440 243.480770 192.168.0.15 -> 192.168.0.9 TCP 66 51036 -> 5000 [ACK] Seq=1074 Ack=80864 Win=129616 Len=0 TSval=549842630 TSecr=60880352
15441 243.482289 192.168.0.9 -> 192.168.0.15 TCP 1514 5000 -> 51036 [ACK] Seq=128648 Ack=1074 Win=32256 Len=1448 TSval=60880358 TSecr=549842620
15442 243.482800 192.168.0.9 -> 192.168.0.15 TCP 1514 5000 -> 51036 [ACK] Seq=130096 Ack=1074 Win=32256 Len=1448 TSval=60880359 TSecr=549842621
15443 243.483313 192.168.0.9 -> 192.168.0.15 TCP 1514 5000 -> 51036 [PSH, ACK] Seq=131544 Ack=1074 Win=32256 Len=1448 TSval=60880359 TSecr=549842621
15444 243.483313 192.168.0.9 -> 192.168.0.15 TCP 1514 5000 -> 51036 [ACK] Seq=132992 Ack=1074 Win=32256 Len=1448 TSval=60880359 TSecr=549842622
15445 243.483825 192.168.0.9 -> 192.168.0.15 TCP 1514 5000 -> 51036 [ACK] Seq=134440 Ack=1074 Win=32256 Len=1448 TSval=60880359 TSecr=549842622
15446 243.483825 192.168.0.9 -> 192.168.0.15 TCP 1514 5000 -> 51036 [ACK] Seq=135888 Ack=1074 Win=32256 Len=1448 TSval=60880359 TSecr=549842622
15447 243.484337 192.168.0.9 -> 192.168.0.15 TCP 1514 5000 -> 51036 [ACK] Seq=137336 Ack=1074 Win=32256 Len=1448 TSval=60880359 TSecr=549842622
15448 243.484849 192.168.0.9 -> 192.168.0.15 HTTP 815 HTTP/1.1 200 OK (application/javascript)
15449 243.485378 192.168.0.15 -> 192.168.0.9 TCP 66 51036 -> 5000 [ACK] Seq=1074 Ack=82312 Win=131072 Len=0 TSval=549842630 TSecr=60880352
15450 243.485377 192.168.0.15 -> 192.168.0.9 TCP 66 51036 -> 5000 [ACK] Seq=1074 Ack=85208 Win=129616 Len=0 TSval=549842631 TSecr=60880352
15451 243.485890 192.168.0.15 -> 192.168.0.9 TCP 66 51036 -> 5000 [ACK] Seq=1074 Ack=86656 Win=131072 Len=0 TSval=549842633 TSecr=60880352
15452 243.485889 192.168.0.15 -> 192.168.0.9 TCP 66 51036 -> 5000 [ACK] Seq=1074 Ack=89552 Win=129616 Len=0 TSval=549842634 TSecr=60880352
15453 243.485889 192.168.0.15 -> 192.168.0.9 TCP 66 51036 -> 5000 [ACK] Seq=1074 Ack=91000 Win=131072 Len=0 TSval=549842635 TSecr=60880352
15454 243.485890 192.168.0.15 -> 192.168.0.9 TCP 66 51036 -> 5000 [ACK] Seq=1074 Ack=93896 Win=129616 Len=0 TSval=549842636 TSecr=60880352
15455 243.486402 192.168.0.15 -> 192.168.0.9 TCP 66 51036 -> 5000 [ACK] Seq=1074 Ack=95344 Win=131072 Len=0 TSval=549842637 TSecr=60880352
15456 243.486400 192.168.0.15 -> 192.168.0.9 TCP 66 51036 -> 5000 [ACK] Seq=1074 Ack=98240 Win=129616 Len=0 TSval=549842639 TSecr=60880352
15457 243.486913 192.168.0.15 -> 192.168.0.9 TCP 66 51036 -> 5000 [ACK] Seq=1074 Ack=99688 Win=131072 Len=0 TSval=549842640 TSecr=60880352
15458 243.487425 192.168.0.15 -> 192.168.0.9 TCP 66 51036 -> 5000 [ACK] Seq=1074 Ack=102584 Win=129616 Len=0 TSval=549842641 TSecr=60880352
15459 243.487426 192.168.0.15 -> 192.168.0.9 TCP 66 51036 -> 5000 [ACK] Seq=1074 Ack=104032 Win=131072 Len=0 TSval=549842642 TSecr=60880352
15460 243.487425 192.168.0.15 -> 192.168.0.9 TCP 66 51036 -> 5000 [ACK] Seq=1074 Ack=106928 Win=129616 Len=0 TSval=549842643 TSecr=60880354
15461 243.487938 192.168.0.15 -> 192.168.0.9 TCP 66 51036 -> 5000 [ACK] Seq=1074 Ack=108376 Win=131072 Len=0 TSval=549842644 TSecr=60880354
15462 243.487937 192.168.0.15 -> 192.168.0.9 TCP 66 51036 -> 5000 [ACK] Seq=1074 Ack=111272 Win=129616 Len=0 TSval=549842645 TSecr=60880354
15463 243.488962 192.168.0.15 -> 192.168.0.9 TCP 66 51036 -> 5000 [ACK] Seq=1074 Ack=112720 Win=129616 Len=0 TSval=549842646 TSecr=60880357
15464 243.488961 192.168.0.15 -> 192.168.0.9 TCP 66 51036 -> 5000 [ACK] Seq=1074 Ack=114168 Win=131072 Len=0 TSval=549842647 TSecr=60880357
15465 243.492545 192.168.0.15 -> 192.168.0.9 TCP 66 51036 -> 5000 [ACK] Seq=1074 Ack=117064 Win=129616 Len=0 TSval=549842648 TSecr=60880357
15466 243.492546 192.168.0.15 -> 192.168.0.9 TCP 66 51036 -> 5000 [ACK] Seq=1074 Ack=118512 Win=131072 Len=0 TSval=549842649 TSecr=60880358
15467 243.493058 192.168.0.15 -> 192.168.0.9 TCP 66 51036 -> 5000 [ACK] Seq=1074 Ack=121408 Win=129616 Len=0 TSval=549842650 TSecr=60880358
15468 243.493618 192.168.0.15 -> 192.168.0.9 TCP 66 51036 -> 5000 [ACK] Seq=1074 Ack=122856 Win=131072 Len=0 TSval=549842651 TSecr=60880358
15469 243.493618 192.168.0.15 -> 192.168.0.9 TCP 66 51036 -> 5000 [ACK] Seq=1074 Ack=125752 Win=129616 Len=0 TSval=549842652 TSecr=60880358
15470 243.499282 192.168.0.15 -> 192.168.0.9 TCP 66 51036 -> 5000 [ACK] Seq=1074 Ack=127200 Win=131072 Len=0 TSval=549842653 TSecr=60880358
15471 243.499282 192.168.0.15 -> 192.168.0.9 TCP 66 51036 -> 5000 [ACK] Seq=1074 Ack=130096 Win=129616 Len=0 TSval=549842654 TSecr=60880358
15472 243.499281 192.168.0.15 -> 192.168.0.9 TCP 66 51036 -> 5000 [ACK] Seq=1074 Ack=131544 Win=131072 Len=0 TSval=549842655 TSecr=60880359
15473 243.502785 192.168.0.15 -> 192.168.0.9 TCP 66 51036 -> 5000 [ACK] Seq=1074 Ack=132992 Win=129616 Len=0 TSval=549842656 TSecr=60880359
15474 243.502785 192.168.0.15 -> 192.168.0.9 TCP 66 51036 -> 5000 [ACK] Seq=1074 Ack=134440 Win=131072 Len=0 TSval=549842657 TSecr=60880359
15475 243.502786 192.168.0.15 -> 192.168.0.9 TCP 66 51036 -> 5000 [ACK] Seq=1074 Ack=137336 Win=129616 Len=0 TSval=549842658 TSecr=60880359
15476 243.503298 192.168.0.15 -> 192.168.0.9 TCP 66 51036 -> 5000 [ACK] Seq=1074 Ack=138784 Win=131072 Len=0 TSval=549842658 TSecr=60880359
15477 243.503297 192.168.0.15 -> 192.168.0.9 TCP 66 51036 -> 5000 [ACK] Seq=1074 Ack=139533 Win=130320 Len=0 TSval=549842659 TSecr=60880361
(kali@kali) ~[-/SBT-DF203-Lab9]
$ mkdir -p exported
```

HTTP Object Recovery: Exported Evidence

Following successful WEP key recovery and offline decryption, the decrypted capture was subjected to HTTP object extraction. The recovered objects were written to the exported directory for separate examination, preserving the original decrypted capture as the primary evidence source.





The directory listing confirms that HTTP object extraction was successful, with approximately 4.3 MB of recovered material. The extracted content includes PNG and GIF images, CSS stylesheets, JavaScript files, web fonts and various small responses associated with web analytics and tracking services. Several objects were exported more

than once and were retained using duplicate filenames with a (1) suffix, allowing the extracted results to be preserved without overwriting an earlier object.

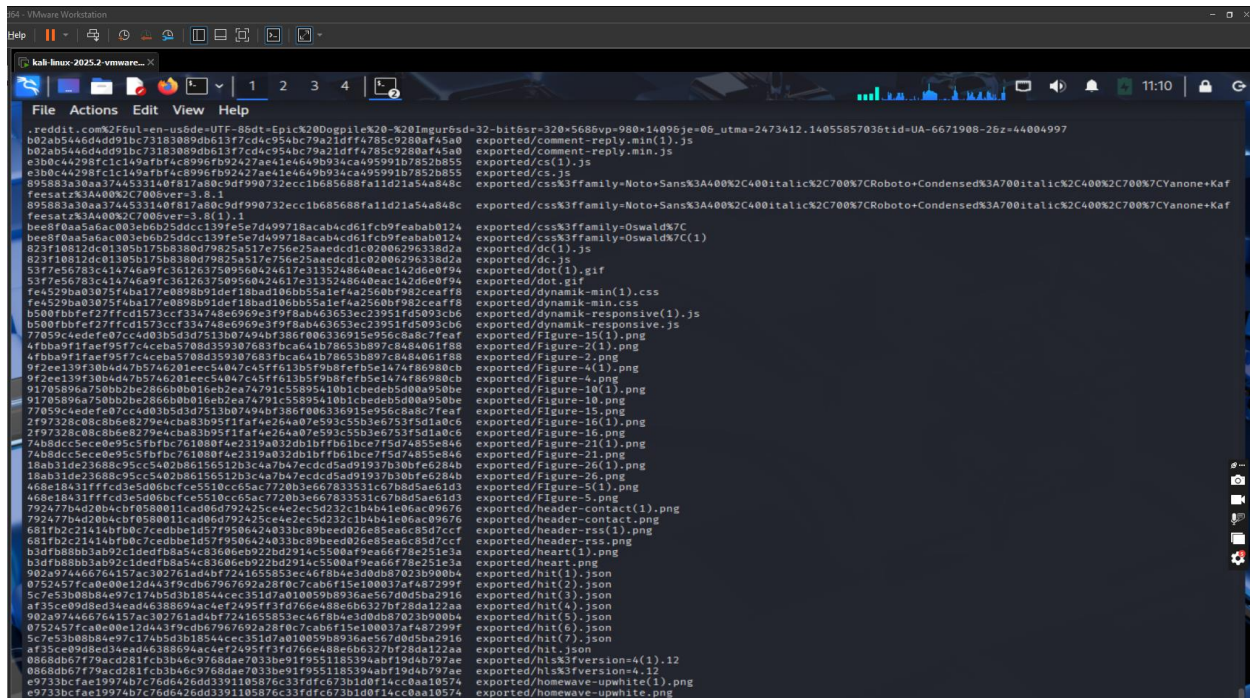
A number of the recovered objects correspond directly with resources identified during the earlier targeted HTTP analysis of the PhishMe web session. For example, multiple Figure-*.png files are present, consistent with the image resources requested from the historical PhishMe page. This provides a direct link between the packet-level HTTP evidence and the recovered application-layer objects.

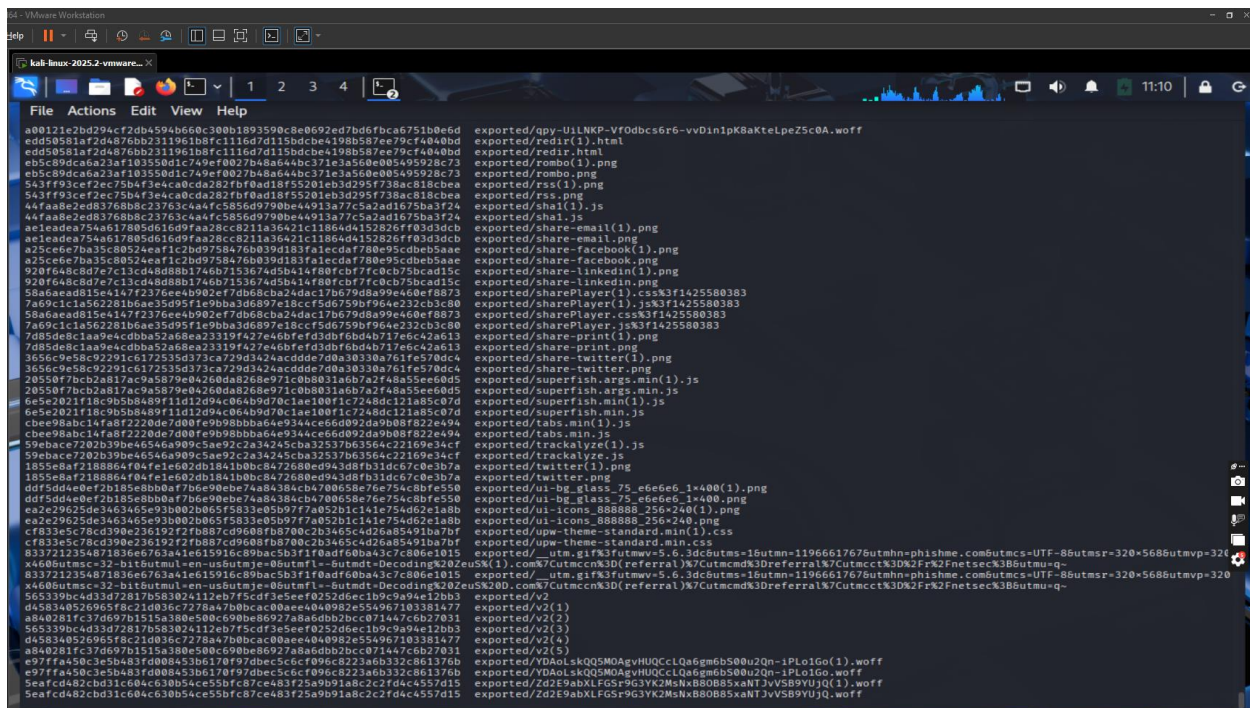
Some extracted objects have zero-byte sizes, indicating that the corresponding HTTP transactions did not contain a recoverable response body. These objects are retained as part of the extraction output but are not treated as substantive recovered files.

At this stage, the directory listing establishes successful HTTP object recovery but does not establish the presence of a malware sample or RTF document. File-type identification and hashing are required before any recovered object can be classified or used as forensic evidence.

SHA-256 Hashing of Recovered HTTP Objects

SHA-256 hashing was performed on the HTTP objects recovered from the decrypted wireless capture. The resulting hashes were recorded in reports/recovered_object_hashes.txt to provide integrity references for the extracted evidence.





The hash inventory shows that several objects exported under duplicate filenames have identical SHA-256 values. For example, Figure-2.png and Figure-2(1).png produced the same SHA-256 hash, as did Figure-10.png and Figure-10(1).png. This indicates that the duplicate files are byte-for-byte identical copies produced during object extraction rather than different versions of the same resource.

The SHA-256 inventory establishes integrity references for the recovered HTTP objects and supports repeatability of the forensic examination. File-type classification is performed separately to determine the nature of the recovered objects and to identify any files requiring further examination.

Following the extraction of HTTP objects from the decrypted wireless capture, the recovered files were subjected to file-type identification using the Linux file utility. This step was performed to establish the actual format and nature of the recovered objects based on their file signatures and content characteristics rather than relying solely on filenames or URL parameters. The results show that the recovered material consists predominantly of ordinary web resources, including HTML documents, JavaScript source files, CSS stylesheets, PNG and GIF images, and a Web Open Font Format (WOFF) font. Several objects were also identified as empty responses, corresponding to HTTP transactions where no substantive response body was recovered.

```

kali-linux-2025.2-vmware-...
File Actions Edit View Help
5eafcd82cb3d1c604c3b0b54ce55bfc7ce483f25a991a8c2c2fd4c4557d15  exported/Zd2E9abXFGS9G3YKZMnXB80B85xaNtJvVS89YUjQ,woff

(kali@kali:~) ~/--(SFT-DF203-Lab9)
└─$ file exported/1 | tee reports/recovered_object_file_types.txt
exported/%2f:
exported/%2f(1): HTML document, ASCII text
exported/6(1).gif: HTML document, ASCII text
exported/6(2).gif: GIF image data, version 89a, 1 x 1
exported/6(3).gif: GIF image data, version 89a, 1 x 1
exported/6.gif: GIF image data, version 89a, 1 x 1
exported/accordion.min(1).js: GIF image data, version 89a, 1 x 1
exported/accordion.min.js: JavaScript source, ASCII text, with very long lines (8280)
exported/activevieww3f1d=osd1m5avi-BLA0VQV35VnNmWTy9QWknIgcCADA_50u-gEABAB0A8IAQIaGdIA5k6AQBoAYVnMDEIA86ti-18a6d-1099195366p=784,685,1034,9856tos-1163,0,0,0,0,0m5tos-1163,116
3,(1).com2Fads3K3pageK3Dinside%26main%3D1%26h0t%3D%26t1me%3D142560863661s-300,2506uc=5: GIF image data, version 89a, 1 x 1
exported/activevieww3f1d=osd1m5avi-BLA0VQV35VnNmWTy9QWknIgcCADA_50u-gEABAB0A8IAQIaGdIA5k6AQBoAYVnMDEIA86ti-18a6d-1099195366p=784,685,1034,9856tos-1163,0,0,0,0,0m5tos-1163,116
3,(1).com2Fads3K3pageK3Dinside%26main%3D1%26h0t%3D%26t1me%3D142560863661s-300,2506uc=5: GIF image data, version 89a, 1 x 1
exported/adc-1-00aeacd(1).png: PNG image data, 12 x 12, 8-bit/color RGBA, non-interlaced
exported/adc-1-00aeacd.png: PNG image data, 12 x 12, 8-bit/color RGBA, non-interlaced
exported/admin-ajax.php%3faction=wordfence_logHumanhshid=305E486D496430879857CB91BBAF7146r-0.1.195857886690644: empty
exported/admin-ajax.php%3faction=wordfence_logHumanhshid=305E486D496430879857CB91BBAF7146r-0.1.195857886690644: empty
exported/ads3K3fgdfp_req=16correlator-39741610776330246output=json.html5callback-callbackProxympl=f1f5f5js0n-a16eid=1088090356fsv-1-0-261u_parts=4761660%2CH_I_300%2506enc_prev
ius=K2F0X2716prev_iu_szs=300(1).140558573036a_sid=142562848086a_hid=218395456ga_fc=true: JavaScript source, Unicode text, UTF-8 text, with very long lines (34622), with no line
terminators
exported/ads3K3fgdfp_req=16correlator-39741610776330246output=json.html5callback-callbackProxympl=f1f5f5js0n-a16eid=1088090356fsv-1-0-261u_parts=4761660%2CH_I_300%2506enc_prev
ius=K2F0X2716prev_iu_szs=300%25.140558573036a_sid=142562848086a_hid=218395456ga_fc=true: JavaScript source, Unicode text, UTF-8 text, with very long lines (34622), with no line
terminators
exported/ads3K3page=inside%26main=1%h0t=6t1me=1425608636: HTML document, ASCII text, with CRLF, LF line terminators
exported/ads3K3page=inside%26main=1%h0t=6t1me=1425608636(1): HTML document, ASCII text, with CRLF, LF line terminators
exported/analyticsh3K3conly=true%26visitor-id=736851146p1_opt_in=6campaign_id=15646account_id=473826title=Decoding%20Zeus%20Disguised%20as%20an%20.RTF%20File%20-%20PhishMeUrl=http
3K3A2F2Fphishh3me.com%2Fdecoding-zeus-disguised-as-an-rtf-file%2F2f9referr=http3K3(1).com%2F%2F2Fnetsec: ASCII text
exported/analyticsh3K3conly=true%26visitor-id=736851146p1_opt_in=6campaign_id=15646account_id=473826title=Decoding%20Zeus%20Disguised%20as%20an%20.RTF%20File%20-%20PhishMeUrl=http
3K3A2F2Fphishh3me.com%2Fdecoding-zeus-disguised-as-an-rtf-file%2F2f9referr=http3K3(1).com%2F%2F2Fnetsec: ASCII text
exported/analyticsh3K3ver=3%26visitor-id=6p1_opt_in=6campaign_id=15646account_id=473826title=Decoding%20Zeus%20Disguised%20as%20an%20.RTF%20File%20-%20PhishMeUrl=http3K3A2F2F2Fphish
h3me.com%2Fdecoding-zeus-disguised-as-an-rtf-file%2F2f9referr=http3K3(1).com%2F%2F2Fnetsec: ASCII text
exported/analyticsh3K3ver=3%26visitor-id=6p1_opt_in=6campaign_id=15646account_id=473826title=Decoding%20Zeus%20Disguised%20as%20an%20.RTF%20File%20-%20PhishMeUrl=http3K3A2F2F2Fphish
h3me.com%2Fdecoding-zeus-disguised-as-an-rtf-file%2F2f9referr=http3K3A2F2F2Fphishh3me.com%2F%2F2Fnetsec: ASCII text
exported/5auid=5372002706res=320x568x326p1g=qtK2Cpm6ch=UTF-8htz=-5406ws=300x250: ASCII text
exported/5auid=5372002706res=320x568x326p1g=qtK2Cpm6ch=UTF-8htz=-5406ws=300x250(1): ASCII text

```

```
File Actions Edit View Help
exported/bidK3fsrc=30796u=httpK3A2FK2Fimgur(1).com%2F8iVy7t66cb=3908366: Web Open Font Format, TrueType, length 13324, version 1.1
exported/bidK3fsrc=30796u=httpK3A2FK2Fimgur.com%2F8iVy7t66cb=3908366: ASCII text, with no line terminators
exported/blah(1).css: ASCII text, with no line terminators
exported/blah.css: ASCII text
exported/bootstrap.min(1).css: ASCII text, with very long lines (65307)
exported/bootstrap.min.css: ASCII text, with very long lines (65307)
exported/bootstrap-responsive.min(1).css: ASCII text, with very long lines (16608)
exported/bootstrap-responsive.min.css: ASCII text, with very long lines (16608)
exported/category-strip(1).png: PNG image data, 16 x 20, 8-bit/color RGBA, non-interlaced
exported/category-strip.png: PNG image data, 16 x 20, 8-bit/color RGBA, non-interlaced
exported/collapsible-widgets(1).css: ASCII text
exported/collapsible-widgets(1).js: JavaScript source, ASCII text, with very long lines (7085), with CRLF line terminators
exported/collapsible-widgets.css: ASCII text
exported/collapsible-widgets.js: JavaScript source, ASCII text, with very long lines (7085), with CRLF line terminators
exported/collectK3fv=16_v=338a=4420489678t=pageview6_s=15dl=httpK3A2FK2Fimgur.com%2F8iVy7t66dr=httpK3A2FK2Fm.reddit.com%2F6ul=en-us&de=UTF-8&dt=Epic%20Dopple%20-%20imgur&sd=32-bit&sr=320x568&vp=980x1409&je=00_utm=2473(1).1405585703&tld=UA-6671908-2&z=44004997: GIF image data, version 89a, 1 x 1
exported/collectK3fv=16_v=338a=4420489678t=pageview6_s=15dl=httpK3A2FK2Fimgur.com%2F8iVy7t66dr=httpK3A2FK2Fm.reddit.com%2F6ul=en-us&de=UTF-8&dt=Epic%20Dopple%20-%20imgur&sd=32-bit&sr=320x568&vp=980x1409&je=00_utm=2473&tld=UA-6671908-2&z=44004997: GIF image data, version 89a, 1 x 1
exported/comment-reply.min(1).js: ASCII text, with very long lines (757), with no line terminators
exported/cs(1).js: ASCII text, with very long lines (757), with no line terminators
exported/cs.js: empty
exported/cssK3ffamily=Noto+SansK3A400%2C400italic%2C700%7CRoboto+CondensedK3A700italic%2C400%2C700%7CYanone+KaffeesatzK3A400%2C700ver=3.8.1: empty
exported/cssK3ffamily=Noto+SansK3A400%2C400italic%2C700%7CRoboto+CondensedK3A700italic%2C400%2C700%7CYanone+KaffeesatzK3A400%2C700ver=3.8(1).1: ASCII text
exported/cssK3ffamily=Oswald%7C: ASCII text
exported/cssK3ffamily=Oswald%7C(1): ASCII text
exported/dc(1).js: JavaScript source, ASCII text, with very long lines (1428)
exported/dc.js: JavaScript source, ASCII text, with very long lines (1428)
```

```
File Actions Edit View Help
exported/dynamik-responsive.js: JavaScript source, ASCII text
exported/Figure-15(1).png: PNG image data, 470 x 145, 8-bit/color RGBA, non-interlaced
exported/Figure-2(1).png: PNG image data, 537 x 274, 8-bit/color RGBA, non-interlaced
exported/Figure-2.png: PNG image data, 537 x 274, 8-bit/color RGBA, non-interlaced
exported/Figure-4(1).png: PNG image data, 440 x 173, 8-bit/color RGBA, non-interlaced
exported/Figure-4.png: PNG image data, 440 x 173, 8-bit/color RGBA, non-interlaced
exported/Figure-10(1).png: PNG image data, 677 x 272, 8-bit/color RGBA, non-interlaced
exported/Figure-10.png: PNG image data, 677 x 272, 8-bit/color RGBA, non-interlaced
exported/Figure-15.png: PNG image data, 470 x 145, 8-bit/color RGBA, non-interlaced
exported/Figure-16(1).png: PNG image data, 230 x 144, 8-bit/color RGBA, non-interlaced
exported/Figure-16.png: PNG image data, 230 x 144, 8-bit/color RGBA, non-interlaced
exported/Figure-21(1).png: PNG image data, 603 x 411, 8-bit/color RGBA, non-interlaced
exported/Figure-21.png: PNG image data, 603 x 411, 8-bit/color RGBA, non-interlaced
exported/Figure-26(1).png: PNG image data, 634 x 209, 8-bit/color RGBA, non-interlaced
exported/Figure-26.png: PNG image data, 634 x 209, 8-bit/color RGBA, non-interlaced
exported/Figure-5(1).png: PNG image data, 673 x 340, 8-bit/color RGBA, non-interlaced
exported/Figure-5.png: PNG image data, 673 x 340, 8-bit/color RGBA, non-interlaced
exported/header-contact(1).png: PNG image data, 19 x 19, 8-bit/color RGBA, non-interlaced
exported/header-contact.png: PNG image data, 19 x 19, 8-bit/color RGBA, non-interlaced
exported/header-rss(1).png: PNG image data, 19 x 19, 8-bit/color RGBA, non-interlaced
exported/header-rss.png: PNG image data, 19 x 19, 8-bit/color RGBA, non-interlaced
exported/heart(1).png: PNG image data, 46 x 48, 8-bit/color RGBA, non-interlaced
exported/heart.png: PNG image data, 46 x 48, 8-bit/color RGBA, non-interlaced
exported/hit(1).json: JSON text data
exported/hit(2).json: JSON text data
exported/hit(3).json: JSON text data
exported/hit(4).json: JSON text data
```

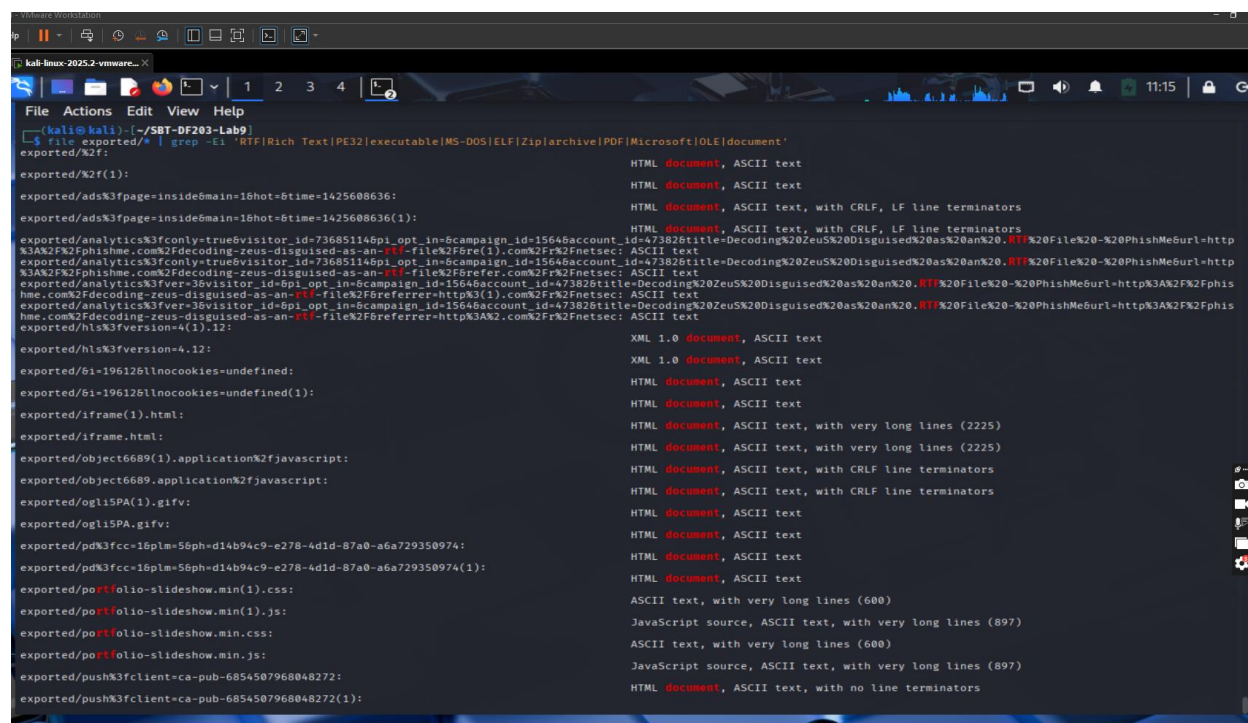
The results also contain objects whose filenames reference the historical PhishMe page title, including strings such as Decoding%20Zeus%20Disguised%20as%20an%20.RTF%20File. However, these objects were identified as ASCII text or analytics-related responses rather than RTF documents or executable files. This distinction is important because a URL, page title, or analytics parameter describing an RTF file does not by itself establish that an RTF

document or malware sample was transferred to the internal endpoint. The file-type analysis therefore provides an evidence-based basis for distinguishing the subject matter of the visited webpage from the actual objects recovered from the network capture.

The presence of duplicate filenames with (1), (2), or (3) suffixes is consistent with multiple instances of the same HTTP resource being exported without overwriting an earlier copy. The identified file types, together with the previously observed HTTP request and response activity, demonstrate that the recovered objects predominantly represent components of the historical web browsing session, such as page content, scripts, stylesheets, images, fonts and tracking resources. These classifications are subsequently supported by SHA-256 hashing to establish the integrity and identity of the recovered objects.

Targeted File-Type Examination for Potential Malware or Document Objects

A targeted file-type examination was performed against the recovered HTTP objects to identify files that could represent an RTF document, executable, archive, PDF, Microsoft Office/OLE document, or other potentially significant file type. The search included classifications such as Rich Text Format, PE32 and MS-DOS executables, ELF binaries, ZIP/archive formats, PDF, Microsoft documents and OLE objects.



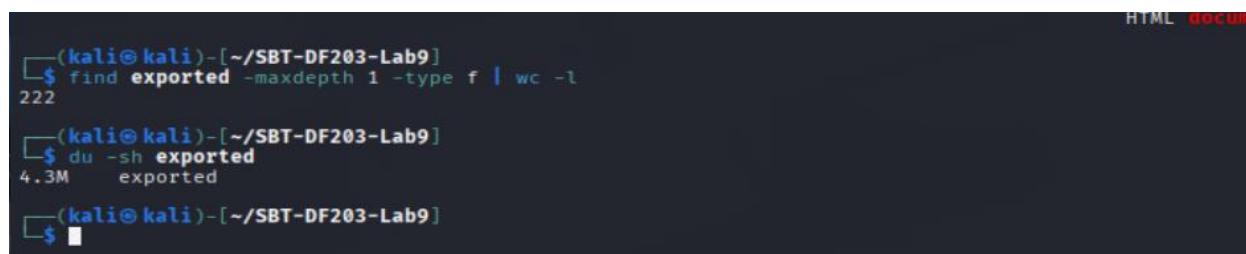
```
kali@kali: ~/SMT-DF203-Lab9
$ file exported/* | grep -Ei 'RTF|Rich Text|PE32|executable|MS-DOS|ELF|Zip|archive|PDF|Microsoft|OLE|document'
exported/%2f:
exported/%2f(1):
exported/ads%3fpage=inside&main=1&hot=6time=1425608636:
exported/ads%3fpage=inside&main=1&hot=6time=1425608636(1):
exported/analytiks%3fonly=true&visitor_id=736851146pi_opt_in=6&campaign_id=1564&account_id=473826title=Decoding%20Zeus%20Disguised%20as%20an%20.RTF%20File%20-%20PhishMe&url=http%3A%2F%2Fphishme.com%2Fdecoding-zeus-disguised-as-an-rtf-file%2Freferrer=http%3A%2F%2Fphishme.com%2F%2Ffnetsec: ASCII text
exported/analytiks%3fonly=true&visitor_id=736851146pi_opt_in=6&campaign_id=1564&account_id=473826title=Decoding%20Zeus%20Disguised%20as%20an%20.RTF%20File%20-%20PhishMe&url=http%3A%2F%2Fphishme.com%2Fdecoding-zeus-disguised-as-an-rtf-file%2Freferrer=http%3A%2F%2Ffnetsec: ASCII text
exported/analytiks%3fver=3&visitor_id=6pi_opt_in=6&campaign_id=1564&account_id=473826title=Decoding%20Zeus%20Disguised%20as%20an%20.RTF%20File%20-%20PhishMe&url=http%3A%2F%2Fphishme.com%2Fdecoding-zeus-disguised-as-an-rtf-file%2Freferrer=http%3A%2F%2Ffnetsec: ASCII text
exported/analytiks%3fver=3&visitor_id=6pi_opt_in=6&campaign_id=1564&account_id=473826title=Decoding%20Zeus%20Disguised%20as%20an%20.RTF%20File%20-%20PhishMe&url=http%3A%2F%2Fphishme.com%2Fdecoding-zeus-disguised-as-an-rtf-file%2Freferrer=http%3A%2F%2Ffnetsec: ASCII text
exported/hls%3fversion=4.12:
exported/hls%3fversion=4.12(1):
exported/6i=196126linocookies=undefined:
exported/6i=196126linocookies=undefined(1):
exported/iframe(1).html:
exported/iframe.html:
exported/object6689(1).application%2fjavascript:
exported/object6689.application%2fjavascript:
exported/ogli5PA(1).gifv:
exported/ogli5PA.gifv:
exported/pd%3fcc=16plm=56ph=d14b94c9-e278-4d1d-87a0-a6a729350974:
exported/pd%3fcc=16plm=56ph=d14b94c9-e278-4d1d-87a0-a6a729350974(1):
exported/port%2folio-slideshow.min(1).css:
exported/port%2folio-slideshow.min(1).js:
exported/port%2folio-slideshow.min.css:
exported/port%2folio-slideshow.min.js:
exported/push%3fclient=ca-pub-6854507968848272:
exported/push%3fclient=ca-pub-6854507968848272(1):
```

The results did not identify any recovered object as an RTF document, executable binary, archive, PDF, or Microsoft/OLE document. Objects returned by the keyword-based examination were instead identified by the file utility as HTML documents, XML documents or ASCII text. For example, several objects containing the historical page title "Decoding ZeuS Disguised as an .RTF File" were classified as ASCII text associated with analytics requests, while other matches were identified as HTML or XML web responses.

This distinction is significant from a forensic perspective. Although the historical browsing session clearly involved a PhishMe webpage whose URL and metadata referenced a ZeuS sample disguised as an RTF file, the recovered HTTP objects examined in this stage do not provide direct evidence that an RTF document or ZeuS executable was transferred to the internal endpoint. The evidence therefore supports the conclusion that the endpoint accessed and retrieved resources associated with the webpage, but it does not support attributing a malware download solely

Recovered HTTP Object Inventory

The HTTP object extraction produced a total of 222 recovered files within the exported directory, occupying approximately 4.3 MB of storage. The recovered material represents application-layer content reconstructed from the decrypted HTTP traffic contained in the historical wireless capture.

A terminal window screenshot from a Kali Linux system. The prompt is (kali@kali)-[~/SBT-DF203-Lab9]. The first command is \$ find exported -maxdepth 1 -type f | wc -l, which returns 222. The second command is \$ du -sh exported, which returns 4.3M. The third command is \$, followed by a cursor. In the top right corner of the terminal, the text "HTML document" is visible in red.

```
(kali@kali)-[~/SBT-DF203-Lab9]
$ find exported -maxdepth 1 -type f | wc -l
222
(kali@kali)-[~/SBT-DF203-Lab9]
$ du -sh exported
4.3M  exported
(kali@kali)-[~/SBT-DF203-Lab9]
$
```

The recovered objects include HTML documents, CSS stylesheets, JavaScript files, PNG and GIF images, web fonts, XML responses and small analytics or tracking responses. The inventory also contains duplicate exports of several resources, where the extraction process preserved repeated occurrences using filename suffixes such as (1), (2) and (3) rather than overwriting previously recovered objects. The earlier SHA-256 analysis confirmed that several of these duplicate files are byte-for-byte identical.

The inventory provides a measurable record of the application-layer evidence recovered from the decrypted capture and establishes the scope of the subsequent file examination. Combined with the file-type identification and targeted search for executable or document formats, the results indicate that the recovered objects predominantly represent web-browsing resources rather than standalone executable or document payloads.

From a forensic perspective, the inventory preserves the distinction between what was present in the network capture and what can be established about the user's activity from those objects. The 222 recovered objects demonstrate substantial HTTP content transfer, but the quantity of recovered objects alone does not establish that malicious software was downloaded or executed.

SHA-256 Integrity Verification of the Decrypted Capture

Following the successful offline decryption of the historical wireless capture using the recovered WEP40 key, a SHA-256 hash was generated for the decrypted capture derivative `working/file_working-dec`. The purpose of this step was to establish a cryptographic integrity reference for the decrypted evidence used throughout the subsequent network, endpoint, conversation and HTTP object analysis.



```
(kali@kali)-[~/SBT-DF203-Lab9]
$ sha256sum working/file_working-dec | tee reports/decrypted_capture_sha256.txt
167c91994c269777f9048227deb89882caf3cf3c763977f2059604f9a6a40b04  working/file_working-dec
```

The SHA-256 calculation produced the hash `167c91994c269777f9048227deb89882caf3cf3c763977f2059604f9a6a40b04`. The hash was recorded in `reports/decrypted_capture_sha256.txt`, creating a verifiable reference for the exact decrypted capture examined during the forensic analysis.

This integrity verification is important because the decrypted capture represents a derived evidence artifact rather than the original compressed acquisition. The original historical evidence was preserved and separately hashed before processing, while the decrypted capture was generated as a working derivative through the documented offline decryption process. Maintaining a separate hash for the derivative allows subsequent analysis to be performed against a known, reproducible evidence state without altering the original acquisition.

The verified decrypted capture was subsequently used as the source for protocol analysis, endpoint and conversation inventories, HTTP examination and recovered-object extraction. The SHA-256 value therefore provides an integrity checkpoint for the final evidence derivative used throughout these analytical stages.

Final Forensic Findings and Conclusion

Forensic Findings

The examination of the historical wireless capture established a complete forensic workflow from evidence preservation through offline WEP decryption and application-layer analysis. The original compressed evidence was preserved and hashed before processing, producing the SHA-256 value `dd54144caef34f228bfb4b87a9101ca7173969376f7ed357bd068061d4f4b8d6`. A preserved working copy was subsequently created and decompressed for analysis. The decompressed file was confirmed as a PCAP containing IEEE 802.11 wireless traffic.

The capture contained 45,169 packets associated with the historical wireless network. Analysis identified the wireless network as `cgnetwork`, with BSSID `00:26:66:55:97:D6`, and confirmed the presence of WEP-protected traffic. A total of 15,713 protected frames were identified through packet analysis, while Aircrack-ng identified 15,477 WEP data packets suitable for key-recovery processing. Analysis of the WEP initialization vectors also demonstrated repeated IV values, with some IVs occurring multiple times within the capture. This provides direct evidence of the IV reuse characteristic associated with the historical WEP environment.

Aircrack-ng successfully recovered the WEP40 key `A4:3D:F6:F3:74` from the supplied historical capture and reported that the traffic was decrypted correctly at 100%. The recovered key was then used with Airdecap-ng to perform offline decryption. Airdecap-ng reported 45,169 packets read, 15,477 WEP data packets, 15,477 decrypted WEP packets, and zero corrupted WEP packets. This provides strong validation that the recovered key was correct and that the resulting decrypted capture was suitable for subsequent forensic examination.

The decrypted derivative contained 15,477 Ethernet frames, including IPv4, IPv6, ARP, TCP and UDP traffic. Application-layer protocols identified in the decrypted traffic included

HTTP, TLS, DNS, DHCP, mDNS, SSDP and other network services. The primary internal IPv4 endpoint observed in the communication inventory was 192.168.0.15, which communicated extensively with external systems, particularly 198.90.20.111 and 199.27.79.193 over HTTP.

The HTTP analysis identified extensive web activity, including requests to Imgur, Reddit-related resources, advertising and analytics services, and a PhishMe webpage with the path /decoding-zeus-disguised-as-an-rtf-file/. The capture also contained numerous supporting resources associated with the webpage, including JavaScript, CSS, PNG images and other web content. The successful HTTP object extraction recovered 222 objects occupying approximately 4.3 MB.

File-type analysis showed that the recovered objects were predominantly HTML, CSS, JavaScript, PNG, GIF, XML, ASCII text and web-font resources. A targeted examination for RTF documents, executable binaries, archives, PDF files, Microsoft/OLE documents and similar formats did not identify a recovered object as one of those file types. In particular, the objects containing references to the PhishMe page title and the terms "ZeuS" and ".RTF" were identified as text, HTML or analytics-related responses rather than as RTF or executable files.

The recovered objects were also subjected to SHA-256 hashing. Matching hashes were observed among several duplicate exports, demonstrating that repeated filenames generated during HTTP object extraction represented byte-for-byte identical content in those cases. The hashes therefore provide integrity references for the recovered application-layer evidence.

Security Significance

The forensic evidence demonstrates the practical weakness of the historical WEP configuration. The capture contained sufficient WEP-protected traffic and repeated IV values for the supplied offline Aircrack-ng process to recover the 40-bit WEP key. Once the key was recovered, the complete set of suitable WEP data packets was successfully decrypted without corruption.

The significance of this finding is that WEP does not provide the level of confidentiality expected from modern wireless security. Its limited key space and weaknesses in the RC4-based protection mechanism, combined with IV reuse, allow sufficiently large captured traffic sets to support key-recovery attacks. The exercise therefore demonstrates why WEP should not be used to protect contemporary wireless networks.

Modern wireless environments should use current security mechanisms such as WPA2 with AES-CCMP or WPA3, with strong authentication and appropriately managed credentials. Enterprise environments should additionally consider 802.1X-based authentication, certificate-based access where appropriate, network segmentation, monitoring and centralized wireless security controls. Legacy wireless protocols and weak encryption configurations should be identified through regular security assessments and removed or isolated where they remain necessary for legacy systems.

CONCLUSION

The forensic examination successfully preserved, validated, decrypted and analysed the supplied historical WEP wireless capture. The recovered WEP40 key successfully decrypted all 15,477 WEP data packets identified by Airdecap-ng, with no corrupted WEP packets reported. The decrypted evidence subsequently enabled detailed analysis of network endpoints, communications, HTTP activity and application-layer objects.

The examination also demonstrated the security consequences of WEP's design weaknesses, particularly the reuse of initialization vectors and the ability to recover the WEP40 key from captured traffic. The recovered HTTP objects provided substantial evidence of historical web activity, including access to a PhishMe webpage concerning a Zeus sample disguised as an RTF file. However, the recovered-object analysis did not identify an RTF document or executable payload, and therefore the evidence does not support a conclusion that such a malware sample was directly transferred or executed based solely on this capture.

The final decrypted evidence derivative was preserved with the SHA-256 value 167c91994c269777f9048227deb89882caf3cf3c763977f2059604f9a6a40b04, providing an integrity reference for the artifact used throughout the subsequent forensic analysis.